A project report on

A CROSS-ATTENTION DEEP LEARNING FRAMEWORK FOR MULTIMODAL NEUROIMAGING FUSION IN VISION DEFECT DETECTION AND CLASSIFICATION

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)

by

MISHTHI JAIN ([Reg. No.])

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

[November, 2026]

A CROSS-ATTENTION DEEP LEARNING FRAMEWORK FOR MULTIMODAL NEUROIMAGING FUSION IN VISION DEFECT DETECTION AND CLASSIFICATION

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)

by

MISHTHI JAIN

Registration No.: [Reg. No.]

Course: BCSE497J

Guide: [Guide name, Designation]



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Vellore Institute of Technology, Chennai

Submission date: [November, 2026]

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

DECLARATION

I hereby declare that the thesis entitled "A Cross-Attention Deep Learning Framework for Multimodal Neuroimaging Fusion in Vision Defect Detection and Classification" submitted by MISHTHI JAIN ([Reg. No.]), for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of [Guide name].

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: [dd Month yyyy] Signature of the Candidate

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled "A Cross-Attention Deep Learning Framework for Multimodal Neuroimaging Fusion in Vision Defect Detection and Classification" is prepared and submitted by MISHTHI JAIN ([Reg. No.]) to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)** is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:

Name: [Dr./Prof. Guide name]

Date:

Signature of the Examiner	Signature of the Examiner
Name:	Name:
Date:	Date:

Approved by the Head of Department,
Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

Name: Dr. Tulasi Prasad Sariki

Date:

[Seal of the School]

ABSTRACT

Vision depends on an intact pathway from the retina to the visual cortex. Disorders such as glaucoma, optic neuritis and cortical visual impairment alter both its structure, visible on magnetic resonance imaging (MRI), and the timing of its electrical response, recorded as the visual evoked potential (VEP). This project presents a research prototype, the Cross-Attention Fusion Network (CAFN), that combines a three-dimensional MRI volume with a one-dimensional electrophysiological signal. A squeeze-and-excitation 3D residual network and a multi-scale 1D convolutional network map the two inputs to 64 spatial and 16 temporal tokens; gated bidirectional cross-attention fuses them, and a learned attention pool and a multi-task loss with two auxiliary unimodal heads complete the model (1,955,140 parameters; 2.60 G multiply-accumulates per sample at 64^3). VEP conditioning uses zero-phase filtering so that P100 latency is preserved: on 150 synthetic recordings it kept the latency within one sample (4 ms) of the ground truth in 149 cases and attenuated 50 Hz interference by 22 dB on average. Because no public dataset offers paired MRI, VEP and diagnosis labels, the architecture is studied on a controlled synthetic benchmark whose generator sets how much class information each modality carries. In this benchmark fusion improved test accuracy by 10.7 percentage points over the better single modality (78.1% against 68.0% for MRI only and 64.0% for VEP only; chance 33.3%), but cross-attention was statistically indistinguishable from concatenation (77.5%) and element-wise sum (79.6%), and none of seven component ablations produced a reliable change. The pipeline is also run on two public OpenNeuro cohorts (95 subjects with both modalities: major depressive disorder with MEG, and zoster-associated neuralgia with sleep EEG). These are not eye-disease data; the label is patient versus control and the resting log power spectrum stands in for the VEP. On them neural fusion is at chance level (cross-validated ROC-AUC 0.51 for CAFN; test accuracy 0.45 to 0.52 in pooled and leave-one-cohort-out runs), a weak spectral signal is detectable (ROC-AUC 0.56 to 0.62, the best being a spectrum-only network), and nothing transfers consistently between cohorts. A 340-test suite and a one-command, seeded pipeline make the work reproducible. No clinical or diagnostic accuracy is claimed.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to [Guide name], [Designation], School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for [his/her] constant guidance, continual encouragement, understanding; more than all, [he/she] taught me patience in my endeavor. My association with [him/her] is not confined to academics only, but it is a great opportunity on my part of work with an intellectual and expert in the field of [area of expertise].

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Mr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G V Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

Special mention to Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Associate Dean, Dr. Suganya G, Associate Dean, Dr. Sweetlin Hemalatha C, Associate Dean, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai for spending their valuable time and efforts in sharing their knowledge and for helping us in every aspect.

In jubilant state, I express ingeniously my whole-hearted thanks to Dr. Tulasi Prasad Sariki, Head of the Department, B.Tech. Computer Science and Engineering (Artificial Intelligence and Machine Learning) and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

My sincere thanks to all the faculties and staffs at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai

Date: [dd Month yyyy] Mishthi Jain

CONTENTS

ABSTRACT ..... **i**

ACKNOWLEDGEMENT ..... **ii**

CONTENTS ..... **iii**

LIST OF TABLES ..... **vi**

LIST OF FIGURES ..... **viii**

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE ..... **x**

CHAPTER 1 INTRODUCTION **1**

1.1 BACKGROUND 1

1.2 MOTIVATION AND CHALLENGES 2

1.2.1 UNIMODAL ISOLATION 2

1.2.2 CROSS-MODAL HETEROGENEITY 2

1.2.3 LIMITED INTERPRETABILITY 2

1.2.4 THE LABELLING GAP 3

1.3 PROBLEM STATEMENT 3

1.4 OBJECTIVES AND RESEARCH QUESTIONS 4

1.4.1 OBJECTIVES 4

1.4.2 RESEARCH QUESTIONS 5

1.4.3 WORKING HYPOTHESES 5

1.5 SCOPE OF THE PROJECT 6

1.6 LIMITATIONS OF THE STUDY 6

1.7 CONTRIBUTIONS AND DISTINCTION FROM EXISTING WORK 7

1.8 ORGANISATION OF THE REPORT 8

CHAPTER 2 LITERATURE REVIEW **10**

2.1 INTRODUCTION 10

2.2 CLINICAL BACKGROUND 10

2.2.1 GLAUCOMA, OPTIC NEURITIS AND CORTICAL VISUAL IMPAIRMENT 10

2.2.2 THE VISUAL EVOKED POTENTIAL AND THE P100 COMPONENT 11

2.3 DEEP LEARNING FOR VOLUMETRIC NEUROIMAGING 11

2.4 DEEP LEARNING FOR ONE-DIMENSIONAL ELECTROPHYSIOLOGICAL SIGNALS 12

2.5 MULTIMODAL DATA FUSION 12

2.6 ATTENTION AND CROSS-ATTENTION 14

2.7 EXPLAINABILITY 14

2.8 PUBLIC DATA RESOURCES AND THE LABELLING GAP 15

2.9 RESEARCH GAP AND POSITION OF THIS PROJECT 15

CHAPTER 3 METHODOLOGY **18**

3.1 OVERVIEW OF THE APPROACH 18

3.2 DESIGN PRINCIPLES 20

3.3 TOOLS AND ENVIRONMENT 21

3.4 DATA SOURCES 21

3.4.1 PUBLIC RESOURCES SURVEYED....21
3.4.2 REAL COHORTS USED IN STUDY B....22
3.5 SYNTHETIC DATA GENERATION....23
3.5.1 SYNTHETIC MRI VOLUMES (ORIGINAL GENERATOR)....24
3.5.2 SYNTHETIC VEP WAVEFORMS (ORIGINAL GENERATOR)....25
3.5.3 DATASET BEHAVIOUR....25
3.5.4 CONTROLLABLE CLASS INFORMATIVENESS (CURRENT GENERATOR)....26
3.6 DATA INTERFACE....27
3.7 MRI PREPROCESSING....28
3.8 VEP PREPROCESSING....30
3.9 EVALUATION STRATEGY....32
3.9.1 STUDY A: CONTROLLED SYNTHETIC BENCHMARK....33
3.9.2 STUDY B: REAL COHORTS....34
3.9.3 METRICS AND REPORTING....35
CHAPTER 4 SYSTEM DESIGN AND IMPLEMENTATION....36
4.1 ARCHITECTURE OVERVIEW....36
4.2 SPATIAL ENCODER (3D RESIDUAL CNN)....38
4.3 TEMPORAL ENCODER (1D CNN)....39
4.4 CROSS-ATTENTION FUSION MODULE....41
4.5 CLASSIFICATION HEADS AND MULTI-TASK LOSS....43
4.6 TRAINING PROCEDURE....44
4.7 EXPLAINABILITY....47
4.7.1 3D GRAD-CAM ON THE FUSED LOGIT....47
4.7.2 CROSS-ATTENTION HEATMAP....47
4.7.3 P100 LATENCY ESTIMATOR....48
4.8 SOFTWARE STRUCTURE AND CONFIGURATION....48
4.9 TEST DESIGN....51
CHAPTER 5 RESULTS....54
5.1 INTRODUCTION....54
5.2 SIGNAL CONDITIONING RESULTS....54
5.2.1 FREQUENCY RESPONSE....54
5.2.2 MAINS-INTERFERENCE ATTENUATION....55
5.2.3 P100 LATENCY....56
5.2.4 CONDITIONED-WAVEFORM STATISTICS....57
5.2.5 NOMINAL TEMPORAL-TOKEN BINS....58
5.3 SYNTHETIC MRI CHARACTERISATION....58
5.4 MODEL COMPLEXITY....58
5.5 INTERFACE AND SHAPE VERIFICATION....59
5.6 SOFTWARE VERIFICATION RESULTS....60
5.7 TRAINING ACCEPTANCE CRITERIA....61
5.8 SUMMARY OF RESULTS....62
5.9 CONTROLLED SYNTHETIC BENCHMARK AND ABLATION....63
5.10 INFORMATIVENESS SWEEP....65

5.11 REAL COHORTS: CROSS-VALIDATION AND PERMUTATION TESTS 66
5.12 POOLED AND LEAVE-ONE-COHORT-OUT CAFN 68
CHAPTER 6 DISCUSSION 70
6.1 INTRODUCTION 70
6.2 INTERPRETATION OF THE RESULTS 70
6.2.1 SIGNAL CONDITIONING 70
6.2.2 SYNTHETIC DATA AND THE FOREGROUND MASK 71
6.2.3 MODEL SIZE AND COST 72
6.2.4 VERIFICATION 72
6.3 ASSESSMENT AGAINST THE OBJECTIVES, QUESTIONS AND HYPOTHESES 73
6.4 DESIGN OBSERVATIONS 74
6.4.1 COARSE TEMPORAL TOKENS 74
6.4.2 SYNTHETIC SEPARABILITY 74
6.4.3 MULTI-TASK SUPERVISION AND ATTENTION 75
6.4.4 PITFALLS DISCOVERED DURING THE PROJECT 75
6.5 COMPARISON WITH THE LITERATURE 76
6.6 CONSTRAINTS AND THREATS TO VALIDITY 77
CHAPTER 7 CONCLUSION, RECOMMENDATIONS AND FUTURE WORK 78
7.1 CONCLUSION 78
7.2 CONCLUSIONS AGAINST THE OBJECTIVES 80
7.3 RECOMMENDATIONS 80
7.4 FUTURE WORK 81
7.5 CLOSING REMARKS 82
REFERENCES 83
APPENDIX 1 DEFAULT CONFIGURATION 88
A1.1 CONFIGURATION PARAMETERS 88
APPENDIX 2 SOURCE CODE LISTINGS 89
A2.1 OVERVIEW 89
A2.2 LISTINGS 89
APPENDIX 3 TEST INVENTORY 105
A3.1 TEST FILES 105
A3.2 TEST FUNCTIONS 105
APPENDIX 4 REPRODUCTION NOTES 108
A4.1 SOFTWARE ENVIRONMENT 108
A4.2 COMMANDS 108
A4.3 MEASUREMENTS REPORTED IN CHAPTER 5 108
A4.4 ITEMS NOT REPRODUCED HERE 109

LIST OF TABLES

Table 1.1 Organisation of the report....8

Table 2.1 Clinical conditions considered and the way each is encoded in the original synthetic generator (the current generator, Section 3.5.4, adds random lesion position and per-sample jitter)....11

Table 2.2 Canonical fusion strategies and the position of this project....13

Table 2.3 Key references and where each is used in the project....16

Table 2.4 Published cross-attention fusion studies versus this work. The numbers are those reported by the authors on their own data; they are not comparable with the results of this report (different tasks, modalities and data)....17

Table 3.1 Tensor contract at the default configuration....19

Table 3.2 Software stack. nilearn is listed in requirements.txt but is not imported by the current source or tests....21

Table 3.3 Public data resources identified in the project blueprint (descriptions as stated there; not independently re-verified for this report)....22

Table 3.4 Real cohorts used in Study B (subjects with both a T1-weighted MRI and a usable recording)....23

Table 3.5 Class-conditioned parameters of the original synthetic generators.....25

Table 3.6 MRI preprocessing parameters (*MRIConfig*)....30

Table 3.7 VEP preprocessing parameters (*VEPConfig*)....31

Table 3.8 Evaluation strategy: what is measured, on which data, how, and which objective it supports....32

Table 3.9 Model configurations of Study A. All share the encoders, the auxiliary loss and the training recipe unless stated....34

Table 4.1 Differences between the original prototype (v1) and the final design (v2/v3)....38

Table 4.2 Spatial encoder layers (batch size 1) and trainable parameters (default model; the squeeze-and-excitation gates are included)....38

Table 4.3 Temporal encoder layers (batch size 1) and trainable parameters....40

Table 4.4 Fusion strategies selectable with ModelConfig.fusion....43

Table 4.5 Training configuration (TrainConfig, left) and model configuration (ModelConfig, right) at the default values. The studies override several of them (see the text)....45

Table 4.6 Source modules, line counts and responsibilities (current working tree). 49

Table 4.7 Test files and what they verify (current suite: 340 tests in 29 files).....51

Table 5.1 Effective magnitude response (dB) at selected frequencies....55

Table 5.2 Detected P100 latency (ms; mean ± SD over 50 trials per class, original generator) and mean error against the ground truth.....56
Table 5.3 Group delay of the single-pass (causal) implementation of the same filters.....57
Table 5.4 Synthetic MRI (original generator): mean intensity of regions of interest and effect of the 60th-percentile foreground mask.....58
Table 5.5 Trainable parameters and multiply-accumulates (millions, at 64^3) per sample for the default (v2/v3) model.....59
Table 5.6 Statement coverage by module of the original code base (reconstructed from the recorded coverage data; not re-measured for the current code). 60
Table 5.7 Training-loop acceptance criteria asserted by *tests/integration/test_training_loop.py*.....61
Table 5.8 Summary of the objective results.....63
Table 5.9 Study A (synthetic, three classes, μMRI = 1.0, μVEP = 0.3): test performance of the fused head, mean ± SD over seeds.....64
Table 5.10 Study A sweep (synthetic): test accuracy (%, mean ± SD over 2 seeds) versus VEP informativeness, MRI strength fixed at 1.0.....65
Table 5.11 Real cohorts (95 subjects): out-of-fold ROC-AUC, bootstrap 95% interval, balanced accuracy and within-cohort permutation *p*. Neural models were cross-validated once and have no permutation test.....67
Table 5.12 CAFN on the real cohorts (mean ± SD over 3 repeats; test subjects only).....68
Table 5.13 Leave-one-cohort-out for the classical models (real cohorts): ROC-AUC / balanced accuracy on the named test cohort after training on the other.....69
Table 6.1 Assessment of research questions and hypotheses against the evidence. 73
Table 6.2 Constraints of the study and their consequence for the conclusions.…77
Table 7.1 Status of each objective.....80
Table 7.2 Prioritised recommendations.....81
Table A1.1 Default configuration values (current src/config.py).....88
Table A2.1 Source files reproduced in this appendix (original versions).....89
Table A3.1 Test files of the original suite, number of test functions and the component each exercises (86 tests; the current suite has 340, see Table 4.7).....105

LIST OF FIGURES

Figure 1.1 Conceptual motivation: MRI answers *where*, the VEP answers *when*, and cross-attention fusion links the two while producing material that can be inspected.....3

Figure 3.1 End-to-end data flow of the proposed framework (final v2/v3 design). Each token stream queries the other through gated bidirectional cross-attention; a learned attention pool feeds the fused head and two auxiliary heads read the unimodal tokens. Dashed arrows: explainability outputs.....19

Figure 3.2 Axial slices of synthetic MRI volumes from the original generator (seeded). Dashed boxes mark the class-specific region: lesion proxy (Optic neuritis / CVI, index 20) and atrophy proxy (Glaucoma, index 60).....24

Figure 3.3 Synthetic VEP waveforms (original generator) for the three classes, before (left) and after (right) the project's conditioning function *clean_vep_signal*. Dotted line: ground-truth P100 latency.....26

Figure 3.4 The two dataset classes share one preprocessing path. The manifest-driven class serves the real public cohorts (and a future labelled clinical cohort); the in-memory synthetic class serves the controlled benchmark.....28

Figure 3.5 Preprocessing chains for the MRI stream (top) and the VEP stream (bottom), with the function responsible for each step.....28

Figure 4.1 CAFN architecture (v2/v3). Preprocessed MRI and signal streams are encoded into spatial and temporal tokens, exchanged through gated bidirectional cross-attention, pooled by a learned attention pool and classified; two auxiliary heads read the unimodal tokens. In the real-cohort experiments the 1-D stream is the log power spectrum of the resting recording and the head has two classes.....36

Figure 4.2 CAFN architecture (v2/v3) with tensor shapes (default configuration; B = batch size). The two encoders produce 64 spatial and 16 temporal tokens of width 128, the gated bidirectional fusion updates both, a learned attention pool of the fused spatial tokens feeds the fused head, and two auxiliary heads read the mean-pooled unimodal tokens. The original (v1) design is compared in Table 4.1.....37

Figure 4.3 Nominal mapping of the 16 temporal tokens onto the 2 s epoch (125 ms per token) over an example conditioned VEP. The shaded region is token 0.....40

Figure 4.4 One gated bidirectional block of the cross-attention fusion module (CAFM), final design (Equations 4.4 and 4.5). The temporal tokens first query the spatial tokens (attention weights of shape (B, 4, 16, 64)); the updated temporal tokens then serve as keys and values for the spatial queries (weights (B, 4, 64, 16)). Each cross-attention output is scaled by a learned scalar gate initialised at zero, added residually and followed by a feed-forward block; both attention maps are returned for inspection.....42

Figure 4.5 Training and evaluation flow. A seeded permutation splits the data; each epoch is followed by validation; the trained model can be checkpointed and passed to the explainability tools.....45

Figure 4.6 Generation of the two explanation artefacts: the Grad-CAM map (A) and the cross-attention heatmap (B).....48

Figure 4.7 Module structure and import dependencies of the current code base (26 modules, with the test suite at the bottom). Solid outlines: the twelve modules of the original code base; dashed shaded outlines: the fourteen modules added since (listed in Table 4.6). Arrows run from the providing group to the importing group.....49

Figure 5.1 Magnitude response of the conditioning filters in the effective (forward-backward) form.....55

Figure 5.2 Amplitude spectrum of one synthetic VEP (original generator) before and after conditioning. The dotted line marks 50 Hz.....56

Figure 5.3 Left: group delay of the single-pass filters versus the zero-phase pipeline (shaded band: ±1 sample, 4 ms). Right: mean P100 latency error by class (original generator; shaded band: ±4 ms).....57

Figure 5.4 Distributions over 50 trials per class from the original synthetic generator: detected P100 latency after conditioning (left) and peak value after z-scoring (right).....57

Figure 5.5 Left: trainable parameters by component of the final model (1,955,140 in total). Right: multiply-accumulate operations (MACs) per sample at 64^3 by component group (2.60 G in total; Table 5.5).....59

Figure 5.6 Left: number of test functions per test file in the original suite (86 tests; blue: unit; green: integration). Right: statement coverage per module of the original code base (red: below 90%).....61

Figure 5.7 Study A (synthetic): test accuracy by configuration (bars: mean; whiskers: SD over seeds; dotted line: chance). Grey bars are the unimodal references.....64

Figure 5.8 Study A sweep (synthetic): test accuracy versus VEP informativeness (MRI fixed at 1.0). Dashed lines: unimodal models; solid lines: fusion models.....66

Figure 5.9 Real cohorts: cross-validated ROC-AUC (dot: mean; bar: 95% bootstrap interval). Grey: classical models; blue: neural models. The dotted line is chance.....67

Figure 5.10 Pooled CAFN on the real cohorts, repeat 0: training versus validation loss and accuracy. The validation curve stays near chance while the training loss vanishes.....68

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

1D / 3D	One- / three-dimensional
AUC	Area under the receiver operating characteristic curve
BN	Batch normalisation
CAFN	Cross-Attention Fusion Network (the model developed in this project)
CAMF	Cross-attentive multi-modal MRI fusion framework of Zhou et al. (2024)
CNN	Convolutional neural network
CSV	Comma-separated values (file format)
CV	Cross-validation
CVI	Cortical (cerebral) visual impairment
DRIFA-Net	Dual-attention multimodal fusion network of Dhar et al. (2025)
EEG	Electroencephalography
ERG	Electroretinography
FFN	Position-wise feed-forward network
Grad-CAM	Gradient-weighted class activation mapping
HCP	Human Connectome Project
IIR	Infinite impulse response (filter)
ISCEV	International Society for Clinical Electrophysiology of Vision
LN	Layer normalisation
LODO	Leave-one-cohort-out (train on the other cohort, test on the held-out one)
MAC	Multiply–accumulate operation
MDD	Major depressive disorder
MEG	Magnetoencephalography
MONAI	Medical Open Network for AI (software framework)
MRI / sMRI	(Structural) magnetic resonance imaging
N75, P100, N135	Components of the pattern-reversal VEP: negative peak near 75 ms, positive peak near 100 ms, negative peak near 135 ms
NIfTI	Neuroimaging Informatics Technology Initiative (file format)
OASIS	Open Access Series of Imaging Studies
PSD	Power spectral density
PSG	Polysomnography
RAS	Right–anterior–superior (canonical image orientation)
ReLU	Rectified linear unit
ROI	Region of interest
SD	Standard deviation

SE	Squeeze-and-excitation (channel recalibration block)
VEP	Visual evoked potential
A	Amplitude of the synthetic P100 component (Equation 3.1)
τ	Latency of the P100 component; also the percentile threshold τ_p in Equation 3.3
$\mathbf{d_{model}}$	Width of the token embedding (128 by default)
H, $\mathbf{d_h}$	Number of attention heads (4) and dimension per head
$\mathbf{X_s}$**,** $\mathbf{X_t}$	Spatial and temporal token matrices
Q, K, V	Query, key and value matrices of the cross-attention module
$\mathbf{W_Q}$**,** $\mathbf{W_K}$**,** $\mathbf{W_V}$**,** $\mathbf{W_O}$	Learned projection matrices of the cross-attention module
$\mathbf{A_h}$**,** $\mathbf{O_h}$	Attention weights and output of head h
$\mathbf{L_f}$**,** $\mathbf{L_s}$**,** $\mathbf{L_t}$	Cross-entropy losses of the fused, spatial and temporal heads
λ	Weight of the auxiliary losses (0.3)
μ, σ, ε	Mean, standard deviation and stabilising constant of z-score normalisation
$\mathbf{\alpha_k}$**, M**	Grad-CAM channel weights and resulting activation map
$\mathbf{g_s}$**,** $\mathbf{g_t}$	Learned scalar gates of the two cross-attention directions; σ(g) is the logistic sigmoid, σ(0) = 0.5 at initialisation
$\mathbf{\alpha_i}$**,** $\boldsymbol{f_i}$	Attention-pool weight of fused token *i* and the fused token itself (Equation 4.7)
$\mathbf{\mu_{MRI}}$**,** $\mathbf{\mu_{VEP}}$	Generator strengths in [0, 1] controlling how much class information the MRI and the VEP carry (1: default generator, 0: none)

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Vision depends on an uninterrupted chain of structures: the retina, the optic nerve, the optic chiasm, the lateral geniculate nucleus, the optic radiations and, finally, the primary and associated visual cortices (areas V1, V2 and V3). A lesion at any point of this chain can degrade sight, and the clinical picture depends on where along the pathway the damage lies and on whether it disturbs the anatomy, the conduction of the electrical signal, or both.

Three conditions illustrate the range. Glaucoma is a progressive optic neuropathy; a systematic review and meta-analysis estimated its global prevalence at 3.54% among people aged 40 to 80 years, corresponding to about 64.3 million people in 2013 and a projected 111.8 million by 2040 (Tham et al., 2014). Histological work has shown that the degeneration in human glaucoma is not confined to the eye but extends to the intracranial optic nerve, the lateral geniculate nucleus and the visual cortex (Gupta et al., 2006). Optic neuritis is an inflammatory, usually demyelinating, disorder of the optic nerve whose electrophysiological hallmark is a delayed P100 peak of the visual evoked potential (Halliday et al., 1972). Cortical or cerebral visual impairment (CVI) denotes reduced vision that arises from damage to the brain's visual pathways rather than the eye; the literature uses a variety of terms and definitions for it (Sakki et al., 2018).

Two investigations are routinely used to examine this pathway, and they answer different questions. Structural magnetic resonance imaging (sMRI) provides a three-dimensional picture of anatomy, so that atrophy and focal lesions can be localised. The visual evoked potential (VEP) is a one-dimensional electrophysiological recording of the cortical response to a visual stimulus; its latency and amplitude, in particular those of the P100 component, reflect the integrity and speed of conduction along the visual pathway, and its recording and interpretation are standardised by the International Society for Clinical Electrophysiology of Vision (ISCEV) (Odom et al., 2016). Neither investigation alone tells the whole story: MRI carries no timing information and the VEP carries no spatial localisation.

Multimodal data fusion has long been proposed as a way to exploit such complementary information (Calhoun & Sui, 2016). Deep learning has made it possible to learn fusion functions directly from data, and the attention mechanism introduced for sequence modelling (Vaswani et al., 2017) offers a particularly flexible way to let one stream of features select what it needs from another. This project investigates that idea for the pairing of 3D structural MRI and 1D visual evoked potentials.

Published studies that apply cross-attention to multimodal medical data (Zhou et al., 2024; Dhar et al., 2025) usually report a single headline accuracy, which makes it hard to tell whether the attention mechanism, or simply the presence of a second modality, is responsible for a gain. This project therefore treats the fusion design as a question to be tested under controlled conditions and not as an assumption.

1.2 MOTIVATION AND CHALLENGES

Automated, data-driven support for the diagnosis and classification of visual-pathway disorders faces four principal bottlenecks. They motivate the design choices described in the rest of this report.

1.2.1 UNIMODAL ISOLATION

Most computational pipelines analyse either imaging or electrophysiology, not both. A model that only sees the MRI volume can detect structural change but is blind to a conduction delay that has not yet produced visible atrophy; a model that only sees the VEP can detect the delay but cannot say where in the pathway it originates.

1.2.2 CROSS-MODAL HETEROGENEITY

A volumetric MRI array of shape (1, D, H, W) and a waveform of shape (1, L) differ in dimensionality, sampling, noise structure and units. Simple fusion strategies, such as concatenating pooled features or averaging unimodal predictions, give every feature the same standing and cannot express which spatial feature is relevant to which temporal feature. Subtle pathology markers can be diluted as a result.

1.2.3 LIMITED INTERPRETABILITY

A black-box classifier is difficult to trust in a clinical setting. A clinician needs to see which parts of the image and which parts of the waveform drove a decision, so that the output can be checked against clinical knowledge.

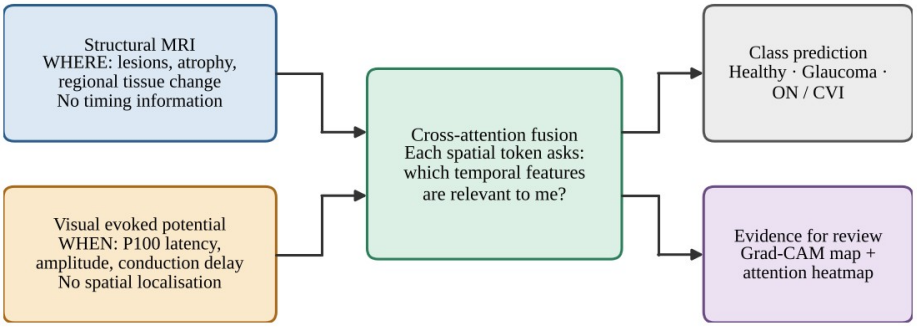


Figure 1.1: Conceptual motivation: MRI answers *where*, the VEP answers *when*, and cross-attention fusion links the two while producing material that can be inspected.

1.2.4 THE LABELLING GAP

No public dataset offers paired MRI, VEP and diagnosis labels for the conditions of interest (Section 2.8). The model and its pipeline can therefore be developed and verified on synthetic data, and exercised on public cohorts from other disorders, but a claim about glaucoma or optic neuritis needs a labelled clinical cohort that does not yet exist in the public domain.

1.3 PROBLEM STATEMENT

To design, implement and verify an end-to-end deep-learning framework that ingests a paired 3D structural MRI volume and a 1D electrophysiological signal (the visual evoked potential in the target application), fuses them with a gated bidirectional cross-attention mechanism in which each stream can query the other, classifies the pair (Healthy Control, Glaucoma or Optic Neuritis / Cortical Visual Impairment in the synthetic benchmark; control versus patient in the public cohorts), and exposes spatial and spatial–temporal explanations of each decision.

The problem is therefore both a modelling problem (how to relate two incompatible feature spaces) and an engineering problem (how to build a pipeline whose interfaces, preprocessing and training behaviour can be tested before a clinical cohort is available).

A third question, which the original prototype left open, is empirical: when does cross-attention fusion actually help compared with simpler fusion by concatenation or summation, or with either modality alone, and does a pipeline that works on synthetic data say anything about real cohorts? The project answers it with a controlled synthetic benchmark and with a reproducible evaluation on two public real cohorts, and reports what it finds even when the finding is negative.

1.4 OBJECTIVES AND RESEARCH QUESTIONS

1.4.1 OBJECTIVES

The project pursues the following objectives. They are revisited, one by one, in the conclusions (Chapter 7).

1. **O1: Data interface.** Provide a single dataset abstraction that reads paired MRI and 1D-signal records from a manifest (synthetic files or the real public cohorts) and standardises them to fixed-shape tensors ($1 \times 64 \times 64 \times 64$, or $1 \times 32 \times 32 \times 32$ for the cross-validation study, and 1×500).
2. **O2: Signal conditioning.** Implement MRI standardisation (RAS orientation, 1 mm isotropic resampling, percentile or Otsu foreground masking, min–max or foreground z-score scaling, centre crop or bounding-box resize) and VEP conditioning (zero-phase 1–100 Hz band-pass, 50 Hz notch, resampling, z-score), plus a spectrum mode for resting recordings, so that clinically meaningful timing is not distorted.
3. **O3: Dual encoders.** Build a 3D residual CNN with squeeze-and-excitation and a multi-scale 1D CNN that map the two modalities to 64 spatial and 16 temporal tokens in a shared 128-dimensional space.
4. **O4: Cross-attention fusion.** Implement gated bidirectional multi-head cross-attention (each stream queries the other) with a learned attention pool, implement the fusion baselines used in the literature (concatenation, element-wise sum, unimodal), and train them with a fused head plus two auxiliary unimodal heads (multi-task supervision).
5. **O5: Explainability.** Provide 3D Grad-CAM and Grad-CAM++ activation maps, spatial–temporal cross-attention heatmaps for both directions and each head, and a P100 latency estimator.
6. **O6: Verification.** Verify interfaces and behaviour with an automated unit and integration test suite (340 tests), and quantify what can be quantified without clinical data (filter behaviour, model size, test results).
7. **O7: Honest scoping.** State clearly what has and has not been demonstrated, and define the path from prototype to clinically credible evaluation.
8. **O8: Controlled benchmark.** Establish, on a synthetic generator whose modality informativeness can be set independently, when cross-attention fusion helps

compared with concatenation, element-wise sum and unimodal models and which design components matter (ablation), using several seeds and a held-out test set.

9. **O9: Real-data protocol.** Provide a one-command, seeded and resumable pipeline for two public real cohorts and evaluate it with per-cohort splits, pooled and leave-one-cohort-out training, repeated cross-validation, bootstrap intervals, label-permutation tests and classical baselines.

1.4.2 RESEARCH QUESTIONS

1. **RQ1.** Can paired 3D MRI and 1D signal data be brought into a common token representation through one manifest-driven pipeline?
2. **RQ2.** Does the conditioning pipeline preserve the P100 latency, the landmark that carries diagnostic meaning in the VEP?
3. **RQ3.** Can the cross-attention fusion network be optimised end-to-end under multi-task supervision, and what is its computational footprint?
4. **RQ4.** Can the system emit explanation artefacts (spatial maps and attention matrices) from the same forward pass that produces the prediction?
5. **RQ5.** When does cross-attention fusion help compared with concatenation, element-wise sum and either modality alone, as a function of how much class information each modality carries?
6. **RQ6.** Does the pipeline generalise across real cohorts: is any within-cohort signal distinguishable from chance by a permutation test, and does a model trained on one cohort work on another?

1.4.3 WORKING HYPOTHESES

The hypotheses are stated at the level at which the evidence can test them. H1 to H3 are engineering hypotheses that can be tested on synthetic data: (H1) zero-phase filtering keeps the detected P100 latency within one sample period (4 ms at 250 Hz) of the ground truth; (H2) the multi-task training loop reduces training loss substantially on a class-conditioned synthetic task and exceeds chance-level validation accuracy; and (H3) all modules respect a single shape contract so that the preprocessing, encoders, fusion module and heads cannot silently disagree. Two further working hypotheses concern the empirical questions: (H4) cross-attention fusion helps relative to concatenation, element-wise sum and unimodal models only

when both modalities carry complementary but individually imperfect class information, and not when one modality alone is almost sufficient; and (H5) on the available public cohorts, where the label is pan-disorder and only 95 subjects have both modalities, the signal is at most weak and a model trained on one cohort does not transfer to the other. H4 and H5 may be refuted by the experiments; they are assessed in Chapter 6.

1.5 SCOPE OF THE PROJECT

In scope. The project covers (i) the data-loading and preprocessing layer for MRI (NIfTI) and 1D signals (VEP CSV or NumPy vectors), including a loader that discovers two public OpenNeuro cohorts in BIDS layout and turns each resting MEG or EEG recording into a log power spectrum; (ii) a synthetic, class-conditioned data generator that exposes the same interface as real data and whose modality informativeness is controllable; (iii) the CAFN model (3D residual encoder with squeeze-and-excitation, multi-scale 1D encoder, gated bidirectional cross-attention, learned attention pooling and three classification heads) together with the concatenation, element-wise sum and unimodal baselines; (iv) a multi-task training loop (AdamW, warm-up and cosine schedule, gradient clipping, early stopping, mixed precision, deterministic algorithms, checkpointing); (v) the explainability tools; (vi) the evaluation protocols, namely a controlled synthetic benchmark with ablations and a real-cohort study with pooled and leave-one-cohort-out training, repeated cross-validation, bootstrap intervals, permutation tests and classical baselines; and (vii) an automated test suite of 340 tests and a one-command reproduction script.

Out of scope. The project does not include clinical data acquisition, expert labelling, clinical-grade brain extraction, anatomical registration of features to named cortical areas, prospective clinical validation or regulatory work. It makes no claim of glaucoma or optic-neuritis diagnosis, of clinical accuracy, of generalisation across cohorts or of state-of-the-art performance. The two real cohorts are not eye-disease data (Section 3.4.2): they test the pipeline, not the clinical hypothesis.

1.6 LIMITATIONS OF THE STUDY

- **No labelled clinical cohort.** The public repositories surveyed for the project provide structural or retinotopic scans and generic electrophysiology but not the Healthy / Glaucoma / Optic Neuritis-CVI diagnosis labels required for training.

The three-class experiments therefore use a synthetic benchmark, which is deliberately simple and cannot support any claim about clinical performance. The real public cohorts that were used instead are not eye-disease data (major depressive disorder with MEG; zoster-associated neuralgia with sleep EEG), carry a pan-disorder patient-versus-control label, and replace the VEP by a log power spectrum.

- **Prototype preprocessing.** Foreground masking is an intensity-percentile threshold or an Otsu threshold with morphological clean-up, not validated skull-stripping, and no anatomical registration is carried through to the model.
- **Learned, uncalibrated tokens.** Spatial and temporal tokens are learned feature positions, not anatomically registered brain regions or calibrated millisecond bins.
- **Small real samples and shortcuts.** The real study has 95 subjects (47 and 48 per cohort) and per-cohort test splits of about seven subjects, so single-split metrics are very noisy; and the spectrum alone identifies the cohort with 100% accuracy, so a cohort-identity shortcut is always possible. Within-cohort permutation tests, repeated cross-validation and leave-one-cohort-out training are used to limit, not remove, this problem.
- **Validity of the controlled benchmark.** In the synthetic generator the two modalities are independent given the label, so their evidence is complementary by construction. The benchmark shows when fusion can help in that setting, not how often real modalities behave that way. No calibration analysis or missing-modality evaluation is included.

1.7 CONTRIBUTIONS AND DISTINCTION FROM EXISTING WORK

It is important to separate what this project contributes from what it reuses. The reused building blocks are standard: scaled dot-product and multi-head attention (Vaswani et al., 2017), residual convolutional blocks (He et al., 2016), squeeze-and-excitation (Hu et al., 2018), Grad-CAM (Selvaraju et al., 2017), Butterworth and notch filtering through SciPy (Virtanen et al., 2020), and MONAI transforms for volumetric resampling and cropping (Cardoso et al., 2022). The contributions of the project are the following.

- **Architecture.** CAFN couples a squeeze-and-excitation 3D residual encoder and a multi-scale 1D encoder through gated bidirectional cross-attention (each stream

queries the other), with learned attention pooling, positional embeddings and two auxiliary unimodal heads. All adaptive pooling uses deterministic matrix products, so that GPU training is reproducible, and the implementation is configuration-driven and covered by 340 tests.

- **A controlled benchmark of when fusion helps.** A synthetic generator with independent controls for the class information carried by each modality allows a like-for-like comparison of cross-attention with concatenation, element-wise sum and unimodal models, a component ablation, and a sweep over modality informativeness, all with several seeds and a held-out test set.
- **A reproducible real-data protocol.** A one-command pipeline downloads two public OpenNeuro cohorts, preprocesses them, and evaluates pooled and leave-one-cohort-out models with repeated cross-validation, bootstrap intervals, label-permutation tests and classical baselines, under a single fixed random seed.
- **Candid findings.** The report states where the proposed design helps and where it does not, that on the available real cohorts CAFN is at chance level and the classical spectral signal is weak and does not transfer between cohorts, and the pitfalls found on the way (Section 6.4.4).

Not claimed. The project does not claim clinical accuracy, diagnosis of glaucoma or optic neuritis, generalisation across cohorts, or state-of-the-art performance. The figures that the published studies report (Table 2.4) refer to other tasks and data and are not comparable with the results here.

1.8 ORGANISATION OF THE REPORT

Table 1.1: Organisation of the report.

Chapter	Content
1 Introduction	Background, motivation, problem statement, objectives, research questions, hypotheses, scope, limitations and contributions.
2 Literature Review	Clinical background, deep learning for MRI and electrophysiology, multimodal fusion, published cross-attention studies (CAMF, DRIFA-Net), attention, explainability, public data resources and the research gap.
3 Methodology	Overall approach, tools, data sources (including the two real cohorts), controllable synthetic data, data interface, MRI and VEP/spectrum preprocessing, and the evaluation strategy (Studies A and B).
4 System Design and Implementation	CAFN architecture (v2/v3), gated bidirectional cross-attention, fusion baselines, multi-task loss, training procedure, explainability, software structure and test design.

Chapter	Content
5 Results	Objective presentation of the measurements: filter behaviour, synthetic data statistics, model size, tests, the controlled synthetic benchmark and ablation, the informativeness sweep, real-cohort cross-validation and permutation tests, and pooled and leave-one-cohort-out CAFN.
6 Discussion	Interpretation of the results, design observations and pitfalls, comparison with the literature and threats to validity.
7 Conclusion, Recommendations and Future Work	Conclusions against the objectives, prioritised recommendations and future work.
	References (APA) and Appendices (configuration, source listings, test inventory, reproduction notes).

CHAPTER 2
LITERATURE REVIEW

2.1 INTRODUCTION

This chapter summarises the research and technical background on which the project rests. It first describes the clinical conditions and the two investigations considered, then reviews deep learning for volumetric imaging and for one-dimensional signals, multimodal fusion strategies and published cross-attention fusion studies, attention and cross-attention, explainability, and the public data resources that were surveyed. It closes by stating the gap that the project addresses and the position it takes.

2.2 CLINICAL BACKGROUND

2.2.1 GLAUCOMA, OPTIC NEURITIS AND CORTICAL VISUAL IMPAIRMENT

Glaucoma is a progressive optic neuropathy and a leading cause of irreversible visual loss; its burden is large and growing (Tham et al., 2014). Although it is usually described through damage to the optic nerve head, post-mortem evidence indicates that degeneration also affects the intracranial optic nerve, the lateral geniculate nucleus and the visual cortex (Gupta et al., 2006). Electrophysiological abnormalities have been reported in ocular hypertension and glaucoma with pattern electroretinography and visual evoked potentials (Parisi et al., 2006). The synthetic data used in this project encode glaucoma by a focal loss of MRI intensity (an atrophy proxy) and a reduced VEP amplitude with near-normal latency.

Optic neuritis is inflammation of the optic nerve. Its classical electrophysiological signature is a prolonged P100 latency of the pattern-reversal VEP with a relatively preserved waveform (Halliday et al., 1972), and the lesion is typically visible on MRI as a focal signal abnormality. Cortical or cerebral visual impairment arises from damage to the posterior visual pathways and visual cortex; the terminology and definitions used for it are not uniform across the literature (Sakki et al., 2018). In this project, optic neuritis and CVI are grouped into a single class and encoded synthetically by a focal MRI hyperintensity (a lesion proxy) and a delayed P100.

Table 2.1: Clinical conditions considered and the way each is encoded in the original synthetic generator (the current generator, Section 3.5.4, adds random lesion position and per-sample jitter).

Condition	Structural (MRI) signature	Electrophysiological (VEP) signature	Synthetic encoding in this project
Healthy control	No focal abnormality	P100 near 100 ms with normal amplitude	Background-like volume; P100 at 100 ms, amplitude 6.0
Glaucoma	Atrophy along the visual pathway (Gupta et al., 2006)	Amplitude and/or latency abnormalities reported (Parisi et al., 2006)	Focal intensity reduction (×0.4); P100 at 108 ms, amplitude 2.5
Optic neuritis / CVI	Focal lesion; posterior-pathway damage in CVI	Delayed P100 (Halliday et al., 1972)	Focal hyperintensity (×1.8); P100 at 140 ms, amplitude 4.0

2.2.2 THE VISUAL EVOKED POTENTIAL AND THE P100 COMPONENT

The VEP is the scalp-recorded cortical response to a visual stimulus, most commonly a pattern reversal. In the ISCEV standard, the principal landmarks of the pattern-reversal response are a negative N75, a positive P100 and a negative N135 component, and the timing of the P100 is the key quantitative measure (Odom et al., 2016). Because a clinically significant delay may amount to a few tens of milliseconds, any signal-processing step applied before analysis must not shift the waveform in time. This requirement is the reason the project uses zero-phase filtering, discussed in Section 3.8.

2.3 DEEP LEARNING FOR VOLUMETRIC NEUROIMAGING

Convolutional neural networks extended to three dimensions are the standard way to learn directly from volumetric scans. Efficient multi-scale 3D CNNs were developed for brain-lesion segmentation (Kamnitsas et al., 2017), and the 3D U-Net demonstrated dense volumetric segmentation from sparse annotation (Çiçek et al., 2016). Residual connections, which make deep networks easier to optimise (He et al., 2016), are routinely carried over to the 3D setting, and squeeze-and-excitation blocks re-weight feature channels from global context at negligible cost (Hu et al., 2018). The spatial encoder of this project follows that tradition: a small 3D residual network with squeeze-and-excitation gates that reduces a 64 × 64 × 64 volume to a 4 × 4 × 4 grid of 128-dimensional features.

Volumetric pipelines depend on preprocessing. Brain extraction (skull-stripping) is a prerequisite for many analyses, and dedicated tools exist: HD-BET, a

neural-network brain extractor for multi-sequence MRI (Isensee et al., 2019), and SynthStrip, a skull-stripping method designed to work across image types (Hoopes et al., 2022). The MONAI framework provides open-source, domain-specific transforms and training components for medical imaging (Cardoso et al., 2022), and nibabel provides access to NIfTI files and their orientation metadata (NiBabel Developers, n.d.). The present project uses nibabel and MONAI for loading, orientation, resampling and cropping, and replaces dedicated skull-stripping with a lightweight intensity-percentile mask or an Otsu threshold with morphological clean-up, both explicitly labelled placeholders (Section 3.7).

2.4 DEEP LEARNING FOR ONE-DIMENSIONAL ELECTROPHYSIOLOGICAL SIGNALS

One-dimensional convolutional networks learn local temporal patterns, such as the rise and fall of a waveform component, with learned filters; stacked strided convolutions then expand the receptive field. Before learning, evoked-potential recordings are conventionally conditioned to remove baseline drift, muscle (EMG) noise and mains interference. A Butterworth filter has a maximally flat pass-band (Butterworth, 1930), and a narrow IIR notch removes the mains frequency; applying either filter forward and backward over the record (as implemented by SciPy's sosfiltfilt and filtfilt, following Gustafsson, 1996) cancels the phase response so that waveform timing is unchanged (Oppenheim & Schafer, 2010). This project adopts exactly this conditioning chain for the VEP and a multi-scale 1D CNN, with parallel convolution kernels of different lengths, as the temporal encoder. For the resting MEG and EEG recordings of the public cohorts no time-domain filtering is applied: each recording is reduced to its log power spectrum (Section 3.8).

2.5 MULTIMODAL DATA FUSION

Calhoun and Sui (2016) argued that combining brain-imaging modalities can reveal relationships invisible to any single modality because each modality carries complementary information. Surveys of multimodal machine learning (Baltrušaitis et al., 2019) and of deep multimodal learning (Ramachandram & Taylor, 2017) distinguish where in the processing chain the modalities are combined. Table 2.2 summarises the three canonical strategies and the position of the present work.

Table 2.2: Canonical fusion strategies and the position of this project.

Strategy	How modalities are combined	Advantages	Disadvantages for MRI + VEP
Early (input-level)	Raw or minimally processed inputs are joined before learning	Simple; allows low-level interaction	A 3D volume and a 1D waveform have no natural common input space; severe misalignment
Late (decision-level)	Unimodal predictions are averaged or voted	Modular; tolerant of a missing modality	No interaction between modalities before the decision; subtle cross-modal evidence is lost
Intermediate (feature-level)	Learned features are combined, by concatenation, gating or attention	Learns task-specific interactions	Concatenation weights all features equally; attention is needed to express relevance
This project	Intermediate fusion by gated bidirectional cross-attention (each stream queries the other), compared with concatenation, element-wise sum and unimodal baselines, plus auxiliary unimodal heads	Selective, inspectable interaction in both directions; unimodal supervision limits modality collapse	Requires paired data; learned tokens are not calibrated anatomy or time; the benefit over simpler fusion has to be tested (Chapter 5)

Cross-attention fusion in published studies. Two recent studies apply cross-attention to multimodal medical data and are the closest published work; Table 2.4 summarises them. CAMF fuses structural and functional MRI for schizophrenia diagnosis with self- and cross-attention (Zhou et al., 2024). It was developed on 620 subjects pooled from three cohorts (COBRE, FBIRN and MPRC) and checked on an independent cohort (BSNIP, 442 subjects); the authors report an accuracy of 74.4% (67.9% on BSNIP) and that cross-attention with adaptive weights outperforms concatenation, element-wise sum and single-modality models. DRIFA-Net combines dual attention modules with iterative alignment and is evaluated on five public medical-imaging datasets (dermoscopy, cytology, CT and MRI) for classification with three to seven classes and for tumour segmentation, with accuracies of 95.6% to 99.7% on the four classification datasets; the authors report that both attention modules are better than either alone (Dhar et al., 2025). Both studies tend to report one headline number per dataset, which makes it difficult to see when, and how much, the attention mechanism contributes. Their results are quoted for context only: the tasks, data and modalities differ from those of this project, so the figures are not comparable with the results of this report.

2.6 ATTENTION AND CROSS-ATTENTION

The Transformer replaced recurrence with scaled dot-product attention, in which queries are compared with keys and the resulting weights are used to average values (Vaswani et al., 2017). Attention is *self*-attention when queries, keys and values come from one sequence and *cross*-attention when the queries come from one sequence and the keys and values from another. Vision Transformers showed that images can be treated as sequences of patch tokens (Dosovitskiy et al., 2021), which is the same device used here to turn a volume into 64 spatial tokens. Cross-modal attention has been used to relate modalities that are not time-aligned: the multimodal transformer lets one modality attend to another without requiring explicit alignment (Tsai et al., 2019), and ViLBERT uses co-attentional layers to exchange information between visual and linguistic streams (Lu et al., 2019).

The central design choice of this project follows from that line of work. Rather than flattening and concatenating the two modalities, the streams are allowed to query each other. In the original design each spatial token was a query that asks which temporal tokens are relevant to it; the final design is bidirectional, so that each temporal token also queries the spatial tokens, and learned gates set how much each direction contributes. In both directions the softmax yields an attention matrix that can be inspected. Because the published evidence that cross-attention beats simpler fusion is limited to specific tasks (Zhou et al., 2024), the benefit is tested here against concatenation, element-wise sum and unimodal baselines (Chapters 3 to 5) and not assumed.

Two normalisation layers that appear in the model are also drawn from the literature: batch normalisation in the convolutional encoders (Ioffe & Szegedy, 2015) and layer normalisation in the fusion block (Ba et al., 2016). The optimiser is AdamW, a variant of Adam (Kingma & Ba, 2015) with decoupled weight decay.

2.7 EXPLAINABILITY

Gradient-weighted class activation mapping (Grad-CAM) uses the gradients of a class score with respect to the activations of a convolutional layer to produce a coarse localisation map (Selvaraju et al., 2017). The project applies the same idea in three dimensions, starting the back-propagation from the fused logit so that the map reflects the multimodal decision.

The literature also warns against over-interpreting explanation artefacts. Attention weights are not automatically faithful explanations of a model's output (Jain & Wallace, 2019), although whether they can serve as explanations depends on how that question is defined and tested (Wiegreffe & Pinter, 2019). Saliency methods can also produce visually plausible maps that are insensitive to the model's learned parameters, which motivates sanity checks such as randomising weights (Adebayo et al., 2018). These cautions shape how the outputs of this project are described: as inspection aids, not as clinical evidence.

2.8 PUBLIC DATA RESOURCES AND THE LABELLING GAP

Several public repositories were identified in the project blueprint as sources of structural and functional neuroimaging and of electrophysiology: OpenNeuro datasets (the project lists ds003787 and ds004496) (Gong et al., 2023; Himmelberg et al., 2026), the Human Connectome Project including its 7 T retinotopy data (Van Essen et al., 2013; Benson et al., 2018), the OASIS-3 database (LaMontagne et al., 2019), the MNE-Python sample dataset (Gramfort et al., 2013) and PhysioNet (Goldberger et al., 2000). They are valuable for developing preprocessing and for retinotopic mapping of the visual cortex, but, as the project blueprint itself notes, none of them provides the Healthy / Glaucoma / Optic Neuritis-CVI diagnosis labels that this model targets. A labelled, paired clinical cohort, or a clinical partner to label a subset, is still needed before real training is possible. This labelling gap is the reason for the synthetic data generator described in Section 3.5.

Because the labelling gap cannot be closed with public data, the project also searched for public cohorts that pair a T1-weighted MRI with a resting electrophysiological recording and have a patient group and a control group, so that the whole pipeline could at least be tested on real data. Two OpenNeuro datasets met these conditions: ds003568, major depressive disorder versus healthy adolescents with MEG (Liuzzi et al., 2023), and ds007181, zoster-associated neuralgia versus healthy controls with polysomnographic EEG (Li et al., 2026). Neither contains VEPs or eye disease; they are described in Section 3.4.2.

2.9 RESEARCH GAP AND POSITION OF THIS PROJECT

The literature reviewed above establishes the ingredients: complementary information in MRI and VEP, 3D and 1D convolutional encoders, cross-modal

attention and Grad-CAM. Published cross-attention fusion studies (Zhou et al., 2024; Dhar et al., 2025) report that attention-based fusion outperforms simpler alternatives on their own tasks, but they combine imaging data, rely on datasets with diagnostic labels, and tend to report a single headline accuracy, so it is hard to tell when fusion is responsible for a gain. To our knowledge no published study fuses MRI with an electrophysiological signal for visual-pathway disorders, and no public dataset with paired MRI, VEP and these diagnoses exists. The project therefore takes the following position. First, it provides a transparent, tested, configuration-driven pipeline that connects the ingredients for the pairing of 3D structural MRI and a 1D electrophysiological signal. Second, it asks the question that the published studies leave open, namely when cross-attention fusion helps compared with concatenation, element-wise sum and unimodal models, by means of a controlled synthetic benchmark. Third, it tests the pipeline candidly on public real cohorts, with leave-one-cohort-out training and permutation tests. It does not claim a new learning algorithm, and no clinical performance claim is made.

Table 2.3: Key references and where each is used in the project.

Reference	Contribution	Use in this project
Vaswani et al. (2017)	Scaled dot-product and multi-head attention	Cross-attention fusion module (Section 4.4)
He et al. (2016)	Residual learning	3D residual blocks of the spatial encoder (Section 4.2)
Selvaraju et al. (2017)	Grad-CAM	3D Grad-CAM on the fused logit (Section 4.7)
Odom et al. (2016)	ISCEV standard for clinical VEP	P100 as the timing landmark; epoch and filter choices (Section 3.8)
Butterworth (1930); Gustafsson (1996)	Maximally flat filter; forward-backward filtering	Zero-phase 1–100 Hz band-pass (Section 3.8)
Cardoso et al. (2022)	MONAI medical imaging framework	Resize, ScaleIntensity, ResizeWithPadOrCrop (Section 3.7)
Paszke et al. (2019)	PyTorch	All neural-network modules and training
Kingma & Ba (2015)	Adam optimiser (AdamW is its decoupled-weight-decay variant)	Training loop (Section 4.6)
Jain & Wallace (2019); Adebayo et al. (2018)	Limits of attention and saliency as explanations	Interpretation of explainability outputs (Chapter 6)
Zhou et al. (2024)	CAMF: cross-attentive fusion of sMRI and fMRI for schizophrenia; cross-attention reported to beat concatenation and sum	Fusion baselines and comparison design (Sections 3.9 and 4.4)

Reference	Contribution	Use in this project
Dhar et al. (2025)	DRIFA-Net: dual-attention multimodal fusion on five public medical-imaging datasets	Related work and context for the benchmark (Section 2.5)
Hu et al. (2018)	Squeeze-and-excitation networks	Channel recalibration in the 3D residual blocks (Section 4.2)
Liuzzi et al. (2023); Li et al. (2026)	OpenNeuro ds003568 (MDD, MEG) and ds007181 (zoster-associated neuralgia, PSG EEG)	The two real cohorts of Study B (Section 3.4.2)

Table 2.4: Published cross-attention fusion studies versus this work. The numbers are those reported by the authors on their own data; they are not comparable with the results of this report (different tasks, modalities and data).

Study	Modalities	Data (size)	Task	Reported result
CAMF (Zhou et al., 2024)	sMRI + fMRI (both 3D imaging)	COBRE + FBIRN + MPRC (620 subjects); BSNIP (442) as independent test	Schizophrenia vs control	Accuracy 74.4%, F1 0.705, MCC 0.491 (BSNIP: 67.9%, 0.684, 0.361); cross-attention better than concatenation, sum and single modality
DRIFA-Net (Dhar et al., 2025)	Dermoscopy, cytology, CT, MRI slices; BraTS MRI	HAM10000 (10,015), SIPaKMeD (4,049), brain-tumour MRI (7,023), IQ-OTHNCCD (1,098), BraTS2020 (369)	3 to 7-class classification; segmentation	Accuracy 95.6–99.7% (D1–D4); both attention modules better than either alone
This work	3D T1w MRI + 1D electrophysiology (heterogeneous)	Controlled synthetic benchmark; two OpenNeuro cohorts (95 subjects)	3-class (synthetic); control vs patient (real)	See Chapter 5

CHAPTER 3
METHODOLOGY

3.1 OVERVIEW OF THE APPROACH

The project follows an iterative engineering methodology in which the interfaces are fixed first and each stage is then implemented and tested against them. Because no labelled clinical data were available, the methodology deliberately separates software correctness (which can be established now) from clinical validity (which cannot), and it separates a controlled synthetic benchmark from a test of the pipeline on real public cohorts. The work proceeded in the following steps.

1. **Define the shape contract.** All tensor shapes, sampling parameters and hyper-parameters are fixed in a single configuration module before any model code is written (Section 4.8).
2. **Specify the data interface.** A manifest of paired records (subject, MRI path, VEP path, sampling rate, label) drives a PyTorch dataset that returns standardised tensors (Section 3.6).
3. **Implement preprocessing.** MRI standardisation and VEP conditioning (or, for resting recordings, spectrum extraction) are implemented as separate, validated, side-effect-free functions (Sections 3.7 and 3.8).
4. **Generate class-conditioned synthetic data.** Because labelled data are missing, synthetic volumes and waveforms with the same interface as real data are produced, with controls for how informative each modality is, so that the entire pipeline can be exercised (Section 3.5).
5. **Build the model.** A 3D residual spatial encoder with squeeze-and-excitation, a multi-scale 1D temporal encoder, gated bidirectional cross-attention with attention pooling and three classification heads are implemented, together with concatenation, element-wise sum and unimodal baselines (Chapter 4).
6. **Train with multi-task supervision.** A fused loss and two auxiliary unimodal losses are optimised jointly with AdamW, a warm-up and cosine schedule and early stopping (Section 4.6).
7. **Add explainability.** 3D Grad-CAM and the cross-attention heatmap are generated from the trained model (Section 4.7).

8. **Verify.** A unit and integration test suite (340 tests), plus measurements on the synthetic data, establish what the prototype does and does not demonstrate (Section 3.9 and Chapter 5).
9. **Evaluate.** Study A, a controlled synthetic benchmark, asks when cross-attention fusion helps; Study B runs the pipeline on two public real cohorts with pooled and leave-one-cohort-out training, repeated cross-validation and permutation tests (Section 3.9).

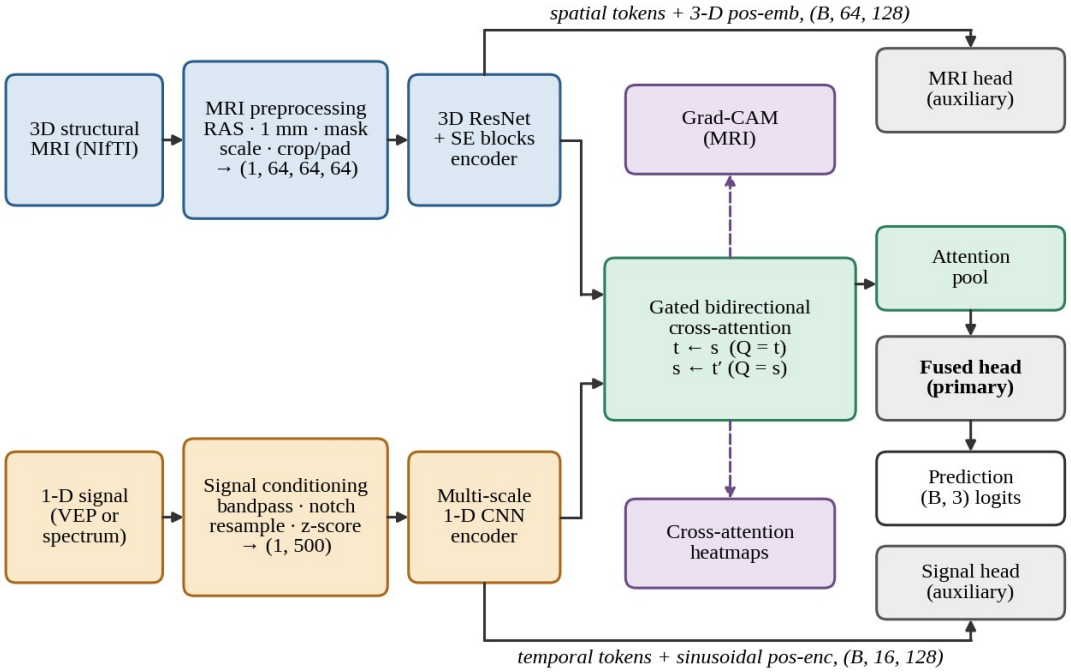


Figure 3.1: End-to-end data flow of the proposed framework (final v2/v3 design). Each token stream queries the other through gated bidirectional cross-attention; a learned attention pool feeds the fused head and two auxiliary heads read the unimodal tokens. Dashed arrows: explainability outputs.

At the default configuration a paired sample moves through the tensor contract of Table 3.1. The batch dimension B is added by the data loader; the channel dimension is part of each sample.

Table 3.1: Tensor contract at the default configuration.

Stage	MRI stream	VEP stream
Raw / loaded	3D array from NIfTI	1D numeric waveform (VEP), or a log power spectrum for resting recordings
After preprocessing	(1, 64, 64, 64); (1, 32, 32, 32) in Study A and the cross-validation study	(1, 500)
Encoder output (tokens)	(B, 64, 128)	(B, 16, 128)

Stage	MRI stream	VEP stream
Cross-attention weights	(B, 4, 64, 16) forward (spatial queries, temporal keys) and (B, 4, 16, 64) backward (temporal queries, spatial keys): 4 heads	
Fused tokens	(B, 64, 128), pooled by a learned attention pool to (B, 128)	(B, 16, 128) updated temporal tokens (not read by the head)
Logits	Fused (B, C); spatial auxiliary (B, C)	Temporal auxiliary (B, C); C = 3 (synthetic) or 2 (real cohorts)

3.2 DESIGN PRINCIPLES

- **Single source of truth.** Every shape and hyper-parameter lives in frozen dataclasses in `src/config.py`; the preprocessing stream, both encoders, the fusion module and the training loop read from it, so they cannot silently disagree.
- **Fail loudly on invalid input.** Preprocessing functions validate dimensionality, finiteness, sampling limits (for example, a band-pass upper edge above the Nyquist frequency is rejected) and signal length, and raise explicit errors.
- **Same interface for real and synthetic data.** The synthetic generator writes real NIfTI and CSV files and a manifest, so the exact code path that a clinical cohort would use is tested.
- **Replaceable placeholders.** Components known to be prototype-grade, notably the foreground mask, are isolated behind one function so a validated tool can be substituted without changing anything downstream.
- **Test every module and the whole system.** Each module has unit tests (340 tests in all); three integration suites test the raw-file-to-optimiser-step path, the training loop and the v2 training pipeline.
- **Reproducible by construction.** One constant seed (42; repeat k uses 42 + k), deterministic algorithms, matrix-product adaptive pooling that is deterministic on CUDA, an environment snapshot recorded with each run, and one resumable script for download, preprocessing, training, validation and testing.

3.3 TOOLS AND ENVIRONMENT

Table 3.2: Software stack. nilearn is listed in requirements.txt but is not imported by the current source or tests.

Component	Role in the project
Python 3	Implementation language: Python 3.12 (Ubuntu 22.04 on WSL2) and 3.14 (macOS); the version is recorded in each run's environment snapshot
PyTorch (Paszke et al., 2019)	Tensors, neural-network modules, automatic differentiation, optimisation, data loading; the device is chosen automatically (CUDA, Apple MPS or CPU), with mixed precision on CUDA and deterministic algorithms requested
MONAI (Cardoso et al., 2022)	`Resize` (trilinear), `ScaleIntensity`, `ResizeWithPadOrCrop` for the MRI stream
NiBabel (NiBabel Developers, n.d.)	NIfTI reading and writing; reorientation to the closest canonical (RAS) frame
SciPy (Virtanen et al., 2020)	`butter`, `sosfiltfilt`, `iirnotch`, `filtfilt` for VEP conditioning
NumPy (Harris et al., 2020)	Array computation; synthetic data generation
Matplotlib (Hunter, 2007)	Grad-CAM slice overlays and the attention heatmap
pytest, pytest-cov	Unit and integration tests and coverage measurement
MNE-Python (Gramfort et al., 2013), scikit-learn (Pedregosa et al., 2011)	MNE-Python reads the CTF MEG and EDF polysomnography files; scikit-learn provides the classical baselines, cross-validation, bootstrap and permutation tests. (nilearn is listed as a dependency but not used.)

The source code is held in a Git repository (NeuroImaging). It now comprises about 5,700 lines of Python source in 26 modules (including the cohort loaders, the experiment runners and the deterministic pooling), 29 test files with about 3,700 lines of tests plus a shared fixtures file, a dataset downloader script and the `run_all.sh` reproduction script. The original code base, on which the measurements of Sections 5.2 to 5.4 and the coverage figure of Section 5.6 were made, had 1,054 lines of source in twelve modules and 1,004 lines of tests in twelve test files.

3.4 DATA SOURCES

3.4.1 PUBLIC RESOURCES SURVEYED

The project blueprint identifies the public repositories in Table 3.3. They are useful for developing and testing preprocessing, and for retinotopic mapping of visual cortex, but none carries the diagnostic labels required for supervised training.

Table 3.3: Public data resources identified in the project blueprint (descriptions as stated there; not independently re-verified for this report).

Modality	Resource	Stated utility
3D sMRI / fMRI	OpenNeuro ds004496	High-resolution visual cortex mapping under retinotopic stimulation
3D sMRI / fMRI	OpenNeuro ds003787	Multimodal structural and functional visual-pathway benchmarks
3D sMRI / fMRI	Human Connectome Project 7 T retinotopy (Van Essen et al., 2013; Benson et al., 2018)	7 T structural and retinotopic functional scans of cortical visual areas V1, V2, V3
3D sMRI	OASIS (LaMontagne et al., 2019)	Large-scale structural MRI for normalisation and grey-matter segmentation baselines
1D VEP / EEG	MNE-Python sample dataset (Gramfort et al., 2013)	Visual and auditory stimulus responses on occipital sensors
1D VEP / ERG	PhysioNet (Goldberger et al., 2000)	Standardised clinical electrophysiology recordings

The labelling gap. None of the resources in Table 3.3 ships Healthy / Glaucoma / Optic Neuritis-CVI diagnoses. Real training therefore requires a labelled, paired clinical cohort or a clinical partner able to label a subset. The data interface described in Section 3.6 is ready for such a cohort: a manifest with the columns `subject_id, mri_path, vep_path, fs, label` is all that is needed.

3.4.2 REAL COHORTS USED IN STUDY B

Because Table 3.3 contains no resource with diagnostic labels for the target conditions, the pipeline was also tested on the closest public data that could be found: OpenNeuro cohorts that provide a T1-weighted MRI, a resting electrophysiological recording, and a patient group and a control group. Two cohorts are used (Table 3.4). ds003568 is a study of major depressive disorder (MDD) in adolescents with healthy volunteers as controls; its recording is resting-state MEG acquired on a CTF system (Liuzzi et al., 2023). ds007181 contains zoster-associated neuralgia patients and healthy controls with a T1-weighted MRI, resting-state fMRI and whole-night polysomnographic (PSG) EEG, of which the T1-weighted scan and the EEG are used (Li et al., 2026).

Table 3.4: Real cohorts used in Study B (subjects with both a T1-weighted MRI and a usable recording).

Cohort	Patients vs controls	1-D signal	Controls	Patients	Total
ds003568	MDD vs healthy adolescents	Resting-state MEG (CTF)	18	29	47
ds007181	Zoster-associated neuralgia vs healthy controls	Whole-night PSG EEG	27	21	48
Total			45	50	95

Label and selection. The label is pan-disorder: 0 for a control and 1 for a patient (MDD in ds003568, zoster-associated neuralgia in ds007181). What "patient" means therefore differs between the cohorts, which is a deliberate and important limitation of pooling them. Only subjects with both a T1-weighted scan and a usable recording are kept: 95 of the 117 listed subjects, namely 47 in ds003568 (18 controls, 29 MDD) and 48 in ds007181 (27 controls, 21 patients).

What these cohorts are not. They are not eye-disease data, no VEP is available in them, and no claim about glaucoma or optic neuritis can follow from them. They serve to test, on real scans and real recordings, whether the pipeline runs end to end, whether any signal can be detected, and whether it transfers from one cohort to the other. The 1-D stream is the channel-averaged log power spectrum of the resting recording (Section 3.8).

Acquisition. `scripts/download_datasets.py` downloads only the files that the pipeline uses (the T1-weighted scan and the recording) in a deterministic order, with resumable transfers and a check of each file against the size reported by the server, and writes a manifest of every file (size and entity tag) so that two machines can compare what they downloaded. The core download is about 68 GB. `python -m src.data.cohorts` then standardises each T1-weighted volume once (Section 3.7), caches it as a NumPy array, extracts the spectrum (Section 3.8) and writes a manifest in the same format as that of the synthetic dataset.

3.5 SYNTHETIC DATA GENERATION

To close the labelling gap for software development, `src/data/synthetic.py` generates class-conditioned MRI-like volumes and VEP-like waveforms. The conditioning is deliberately simple but structured so that a model

can learn from it. The data are useful for software validation and for testing the learning loop; they are not evidence of clinical performance.

Two versions of the generator exist. Sections 3.5.1 to 3.5.3 describe the original generator, with which the signal-conditioning and mask measurements of Sections 5.2 and 5.3 were made. Section 3.5.4 describes the current generator, which draws lesion position and size and the VEP latency and amplitude at random for every sample and exposes controls for how informative each modality is; it is the generator of Study A (Section 3.9.1).

3.5.1 SYNTHETIC MRI VOLUMES (ORIGINAL GENERATOR)

A volume of shape (80, 96, 64) is drawn from a normal distribution with mean 100 and standard deviation 15, clipped at zero. Class conditioning is applied to one octant-like region: for **Glaucoma** the intensities in the block with indices [D/2:, :H/2, :W/2] are multiplied by 0.4 (a proxy for atrophy), and for **Optic Neuritis / CVI** the block [:D/2, H/2:, W/2:] is multiplied by 1.8 (a proxy for a hyperintense lesion). Healthy volumes are left unstructured. Figure 3.2 shows example axial slices.

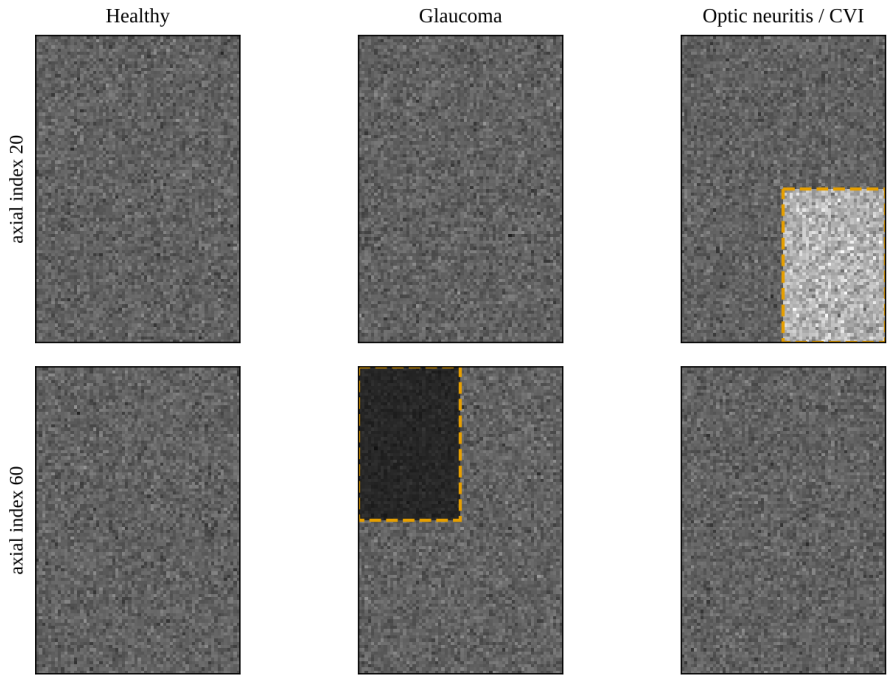


Figure 3.2: Axial slices of synthetic MRI volumes from the original generator (seeded). Dashed boxes mark the class-specific region: lesion proxy (Optic neuritis / CVI, index 20) and atrophy proxy (Glaucoma, index 60).

3.5.2 SYNTHETIC VEP WAVEFORMS (ORIGINAL GENERATOR)

A waveform sampled at 250 Hz for 2 s (500 samples) is built from three Gaussian components that mimic the N75-P100-N145 complex, with latency τ and amplitude A that depend on the class (Table 3.5):

$$s(t) = A\,e^{-\frac{(t-\tau)^2}{2\sigma_1^2}} - 0.5\,A\,e^{-\frac{(t-\tau+0.025)^2}{2\sigma_2^2}} - 0.4\,A\,e^{-\frac{(t-\tau-0.045)^2}{2\sigma_3^2}} \tag{3.1}$$

with σ_1 = 12 ms, σ_2 = 10 ms and σ_3 = 15 ms. Three artefacts are then added: a slow baseline drift 0.3 sin(2π · 0.3 t), mains interference 0.15 sin(2π · 50 t) and Gaussian muscle noise with standard deviation 0.3. These are exactly the artefacts that the conditioning pipeline of Section 3.8 is designed to remove.

Table 3.5: Class-conditioned parameters of the original synthetic generators.

Class (label)	MRI conditioning	P100 latency τ	P100 amplitude A
Healthy Control (0)	None (background-like)	100 ms	6.0
Glaucoma (1)	Block × 0.4 (atrophy proxy)	108 ms	2.5
Optic Neuritis / CVI (2)	Block × 1.8 (lesion proxy)	140 ms	4.0

3.5.3 DATASET BEHAVIOUR

`SyntheticNeuroDataset` creates a balanced list of labels and shuffles it deterministically from a seed. The raw sample for an index is generated when the item is requested. In the original implementation a shared random-number generator was used, so that the same index was not guaranteed to return identical noise on repeated access; the current version draws every item from its own generator seeded with the pair (seed, index), so that a given index always yields the same sample, whatever the access order or the number of data-loading workers. Items can be cached after preprocessing, and augmentation is applied after the cache. The function `write_synthetic_dataset` writes the samples to disk as `sub-XXXX_mri.nii.gz` and `sub-XXXX_vep.csv` files with a `manifest.csv`, which is how the end-to-end tests exercise the real file path.

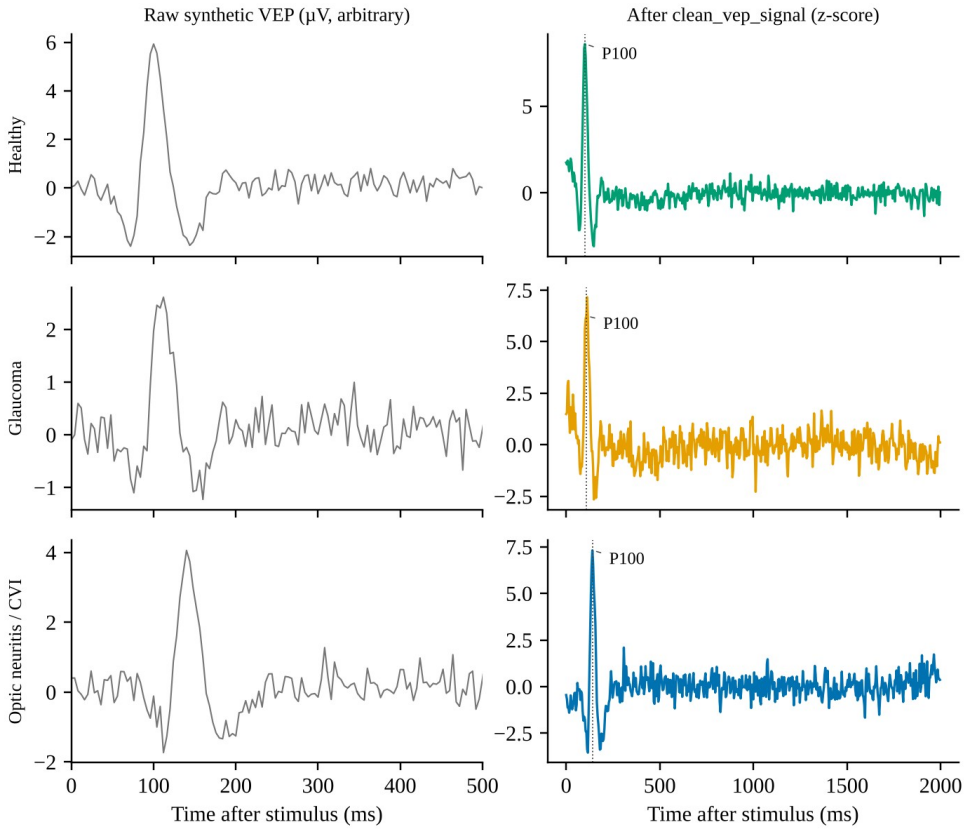


Figure 3.3: Synthetic VEP waveforms (original generator) for the three classes, before (left) and after (right) the project's conditioning function `clean_vep_signal`. Dotted line: ground-truth P100 latency.

3.5.4 CONTROLLABLE CLASS INFORMATIVENESS (CURRENT GENERATOR)

The current generator (`src/data/synthetic.py`) is more realistic than the original one and adds two controls. The MRI is an ellipsoidal brain with random centre and radii, smoothed tissue texture, a bright cortical rim, dark ventricles and a noisy background. Glaucoma volumes carry a focal hypointensity (contrast factor 0.35 to 0.55 relative to the surrounding tissue) and Optic Neuritis / CVI volumes a focal hyperintensity (1.6 to 2.2), each at a random position inside the brain, with a random radius of 3 to 6 voxels and a soft Gaussian fall-off; healthy volumes have no lesion. The VEP keeps the N75–P100–N145 complex, but latency and amplitude are drawn for every sample: latency 100 ± 3, 104 ± 4 and 138 ± 6 ms (mean ± SD) and amplitude 6.0 ± 0.8, 2.5 ± 0.6 and 4.0 ± 1.0 for the Healthy, Glaucoma and Optic Neuritis / CVI classes, with the same drift, mains and muscle artefacts as before and a random phase for the drift and mains components.

Two parameters, μ_{MRI} and μ_{VEP} in [0, 1] (`mri_strength` and `vep_strength`), scale how much class information each modality carries. For the

MRI, the lesion contrast is scaled as $1 + (c - 1) \cdot \mu_{\mathrm{MRI}} \cdot g$, where c is the sampled contrast factor and g the spatial fall-off, so that $\mu_{\mathrm{MRI}} = 0$ removes the visible lesion. For the VEP, the offsets of latency and amplitude from the healthy class are multiplied by μ_{VEP}, so that $\mu_{\mathrm{VEP}} = 0$ makes every class look healthy. A value of 1 recovers the default generator. Because the two modalities are generated independently given the label, their evidence is complementary by construction. This is the setting in which fusion should help, and the benchmark asks how reliably each architecture exploits it (Section 3.9.1).

3.6 DATA INTERFACE

`PairedNeuroDataset` reads a manifest CSV with the columns `subject_id`, `mri_path`, `vep_path`, `fs`, `label` and validates it when it is constructed: an empty manifest, missing columns, bad labels and missing files raise an error before training starts, and relative paths are resolved against the manifest directory. For each row, `__getitem__` performs the following steps: (1) load and standardise the MRI with `preprocess_mri_file`, or load a cached NumPy array of an already standardised volume; (2) read the 1-D signal from a CSV file with `numpy.loadtxt`, skipping the header, or from a `.npy` file; (3) take the sampling rate from the row, falling back to the configured 250 Hz when it is missing; (4) condition the waveform with `clean_vep_signal`, or, in spectrum mode, resample and z-score the stored spectrum; and (5) return a dictionary with the MRI tensor (1, D, H, W), the signal tensor (1, L), a long-integer label and the subject identifier. Items can be cached after preprocessing, and train-time augmentation is applied through a wrapper over the training indices so that validation data are never augmented. The same class serves the real public cohorts.

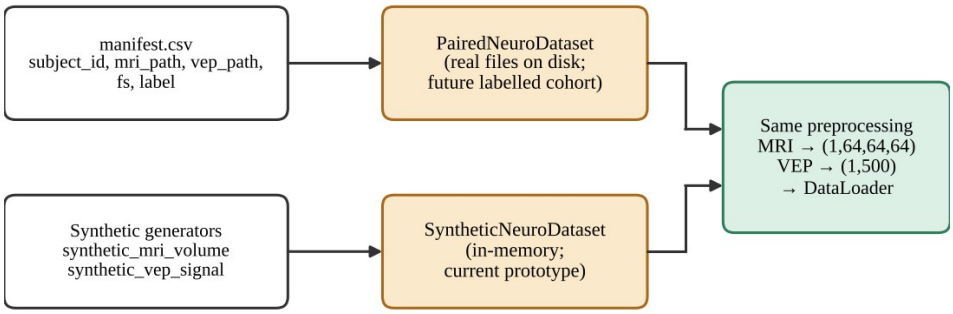


Figure 3.4: The two dataset classes share one preprocessing path. The manifest-driven class serves the real public cohorts (and a future labelled clinical cohort); the in-memory synthetic class serves the controlled benchmark.

3.7 MRI PREPROCESSING

MRI standardisation (`src/preprocessing/mri_transforms.py`) consists of five steps (Figure 3.5). The parameters are held in `MRIConfig`.

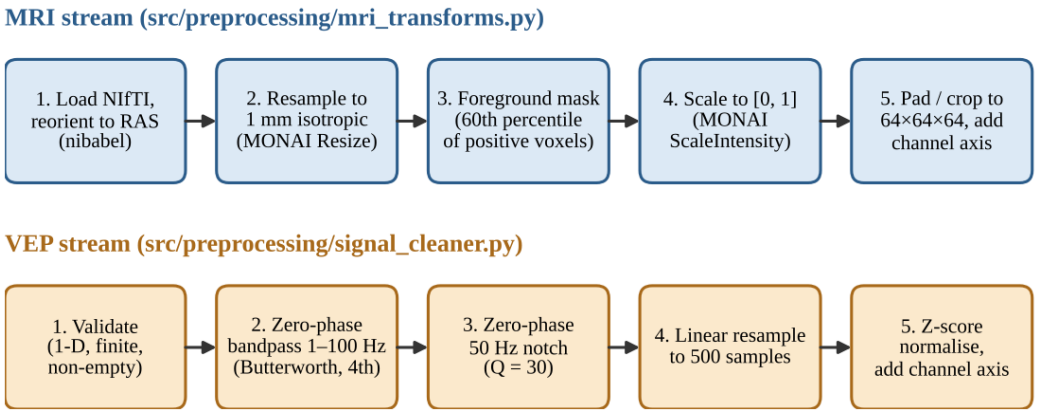


Figure 3.5: Preprocessing chains for the MRI stream (top) and the VEP stream (bottom), with the function responsible for each step.

1. **Load and canonicalise.** `load_nifti` reads the NIfTI file with NiBabel and reorients it to the closest canonical (RAS) frame. It verifies that the image is three-dimensional and that all voxel spacings are positive, and returns the data, the affine and the voxel zooms.
2. **Resample to isotropic spacing.** For voxel zooms z and target spacing s (1 mm), the new shape is computed from the original shape n (Equation 3.2) and the volume is resampled with MONAI's trilinear `Resize`.
3. **Foreground masking.** The p-th percentile (default 60) of the strictly positive voxels is computed and all voxels below it are set to zero (Equation 3.3).

4. **Intensity scaling.** If the masked volume has a non-zero range, MONAI `ScaleIntensity` maps it to [0, 1] (Equation 3.4); a constant volume is preserved as a float array.
5. **Pad or crop.** `ResizeWithPadOrCrop` brings the volume to (64, 64, 64) and the channel axis is added, giving (1, 64, 64, 64).

$$n_i' = round\left(\frac{n_i z_i}{s_i}\right) \tag{3.2}$$

$$\tau_p = P_p\left(\{x : x > 0\}\right), \tilde{x} = x \text{ if } x \geq \tau_p, \tilde{x} = 0 \text{ otherwise} \tag{3.3}$$

$$x' = \frac{\tilde{x} - \min(\tilde{x})}{\max(\tilde{x}) - \min(\tilde{x})} \tag{3.4}$$

Important caveat. The percentile mask is a lightweight, dependency-free stand-in for skull-stripping. It is *not* clinical-grade brain extraction and does not replace validated tools such as HD-BET (Isensee et al., 2019) or SynthStrip (Hoopes et al., 2022). It is isolated in the function `foreground_mask_threshold` so that it can be replaced without changing any downstream code.

Options for real whole-head scans. `MRIConfig` offers three further options (Table 3.6). `mask_method="otsu"` replaces the percentile threshold by the histogram-optimal Otsu threshold followed, with `mask_morphology`, by hole filling and retention of the largest connected component; `normalization="zscore"` standardises the foreground to zero mean and unit variance and sets the background to zero; and `fit="bbox_resize"` crops the volume to the bounding box of the foreground and resamples that box to the target shape (64^3, or 32^3 in the cross-validation study). The default centre crop to 64^3 suffices for the small synthetic volumes but would see only a sliver of a head about 170 mm across at 1 mm spacing. The real cohorts use Otsu, z-score and bounding-box resizing. The original percentile mask, min–max scaling and centre crop remain the defaults, and Study A uses them because the Otsu and z-score pipeline was found to erase the lesion contrast of the synthetic generator (in a single-seed diagnostic the MRI-only accuracy dropped from 73% to 41%). Neither mask is a substitute for validated skull-stripping.

Table 3.6: MRI preprocessing parameters (`MRIConfig`).

Parameter	Default	Meaning
target_shape	(64, 64, 64)	Output spatial size after pad or crop
target_spacing	(1.0, 1.0, 1.0) mm	Isotropic resampling target
orientation	RAS	Canonical orientation
foreground_percentile	60.0	Percentile of positive voxels used as the mask threshold
mask_method	"percentile" (alternative: "otsu")	Foreground mask: percentile threshold, or Otsu threshold
mask_morphology	True	Hole filling and largest connected component for the Otsu mask
normalization	"minmax" (alternative: "zscore")	Scaling to [0, 1], or foreground z-score with the background set to zero
fit	"crop" (alternative: "bbox_resize")	Centre pad/crop, or crop to the foreground bounding box and resample to the target shape

3.8 VEP PREPROCESSING

VEP conditioning (`src/preprocessing/signal_cleaner.py`) removes baseline drift, high-frequency muscle noise and mains interference without shifting the waveform in time. The parameters are held in `VEPConfig` (Table 3.7).

1. **Validate.** The input is converted to 64-bit floats and rejected if it is not one-dimensional, is empty, or contains NaN or infinite values.
2. **Band-pass.** `butterworth_bandpass` applies a fourth-order Butterworth band-pass between 1 Hz and 100 Hz in second-order-section form with `sosfiltfilt`, i.e. forward and backward. The upper edge must lie below the Nyquist frequency (125 Hz at 250 Hz sampling) and the record must be long enough for zero-phase padding; otherwise an error is raised.
3. **Notch.** `notch_filter` applies a zero-phase IIR notch at 50 Hz with quality factor 30 to suppress AC power-line interference.
4. **Resample.** `resample_to_length` linearly interpolates the record to exactly 500 samples.
5. **Normalise.** `zscore_normalize` subtracts the mean and divides by the standard deviation plus a small constant (Equation 3.6); the result is returned as 32-bit floats and a channel axis is added.

$$|H(j\omega)|^2 = \frac{1}{1+(\omega/\omega_c)^{2n}}(n=4), H_{zp}(\omega) = |H(j\omega)|^2 \tag{3.5}$$

$$\hat{x}_k = \frac{x_k - \mu}{\sigma + \epsilon}, \epsilon = 10^{-8} \tag{3.6}$$

Why zero-phase filtering. An ordinary single-pass (causal) filter delays different frequency components by different amounts (its group delay). Because the P100 latency is itself the clinical measurement, such a delay would corrupt the very quantity of interest. Filtering forward and then backward cancels the phase response, so the effective frequency response is the squared magnitude of the single-pass filter and has zero phase (Oppenheim & Schafer, 2010; Gustafsson, 1996). The price is that the filter is non-causal and the effective roll-off is steeper; both are acceptable for offline analysis.

Spectrum mode for resting recordings. The public cohorts contain resting MEG or EEG and no VEP. With `signal_mode="spectrum"` the time-domain chain is bypassed: each recording is first reduced (`src/data/cohorts.py`) to the channel-averaged $\log_{10}$ Welch power spectral density between 1 and 45 Hz (4 s segments with 50% overlap; for ds003568, 120 s of MEG after the first 10 s; for ds007181, 120 evenly spaced 30 s windows of the whole-night EEG channels) and interpolated to 500 points; the dataset layer then resamples to the configured length and z-scores it, and no band-pass or notch filter is applied, because filtering a spectrum would be wrong. The temporal encoder then operates on a spectrum, so the temporal-token axis corresponds to frequency and the nominal millisecond mapping of Figure 4.3 does not apply.

Table 3.7: VEP preprocessing parameters (`VEPConfig`).

Parameter	Default	Meaning
length	500	Samples after resampling
sample_rate	250 Hz	500 samples span a 2.0 s sweep
bandpass_low / bandpass_high	1.0 Hz / 100.0 Hz	Edges of the fourth-order Butterworth band-pass
notch_freq	50 Hz	Mains frequency
notch_quality	30	Q factor; notch bandwidth = f_0 / Q ≈ 1.67 Hz
baseline_window_ms	None	Optional pre-stimulus baseline correction (disabled: synthetic sweeps have no pre-stimulus interval)

Parameter	Default	Meaning
signal_mode	"waveform" (alternative: "spectrum")	Time-domain conditioning chain, or an input that is already a spectrum (only resampling and z-score)

3.9 EVALUATION STRATEGY

The evaluation has three parts, and every result in Chapter 5 is labelled by the kind of data it comes from. Study A is a controlled benchmark on synthetic data; its results are a controlled comparison of architectures and not clinical performance. Study B uses two real public cohorts; its results are reported as found, including where they are at chance. The engineering evidence (filters, model size, tests) is independent of both. Table 3.8 relates each piece of evidence to the objectives, questions and hypotheses of Section 1.4.

Table 3.8: Evaluation strategy: what is measured, on which data, how, and which objective it supports.

Evidence	Data	Method	Supports
Signal-conditioning behaviour	Synthetic (original generator)	The project's own functions are run on 150 synthetic VEPs (50 per class, fixed seeds): frequency response, mains attenuation, group delay, detected P100 latency against ground truth	O2, RQ2, H1
Synthetic data characterisation	Synthetic (original generator)	Region-of-interest statistics of generated MRI volumes; effect of the foreground mask; per-class VEP statistics	O2, O7
Model size and cost	None	Trainable parameters and multiply-accumulate operations counted from the layer definitions	O3, O4, RQ3
Software verification	Synthetic test data	340 test functions (unit and integration) run on two platforms	O1, O6, H3
Training acceptance criteria	Synthetic	Thresholds asserted by the training-loop integration tests (loss reduction, above-chance validation accuracy, checkpoint round-trip)	O4, RQ3, H2
Study A: controlled benchmark and ablation	Synthetic (current generator)	12 model configurations on several seeds, held-out test set, fixed main setting (Section 3.9.1)	O8, RQ5, H4
Study A: informativeness sweep	Synthetic (current generator)	Sweep of the VEP strength for MRI-only, VEP-only, concatenation and CAFN	O8, RQ5, H4
Study B: repeated cross-validation, classical and neural	Real cohorts (95 subjects)	5-fold cross-validation, bootstrap intervals, within-cohort permutation tests (Section 3.9.2)	O9, RQ6, H5
Study B: pooled and leave-one-cohort-out CAFN	Real cohorts	Per-cohort 70/15/15 splits; training on one cohort and testing on the other	O9, RQ6, H5

3.9.1 STUDY A: CONTROLLED SYNTHETIC BENCHMARK

Each (setting, seed) draws 144 training, 48 validation and 120 test samples (the test set from a different seed range), with raw volumes of 48 × 56 × 40 voxels standardised to 32^3 and 500-point VEPs. The MRI preprocessing is the original one (percentile mask, min–max scaling, centre crop), which preserves the lesion contrast on this generator. Checkpoints are selected on validation balanced accuracy and all reported numbers are on the test set. The main setting ($\mu_{\mathrm{MRI}} = 1.0$, $\mu_{\mathrm{VEP}} = 0.3$) was fixed from a pilot that trained only the two unimodal models over a grid of VEP strengths and chose the difficulty at which both were clearly above chance and below saturation; it was not tuned on CAFN. Twelve model configurations (Table 3.9) are trained on 4 to 7 seeds each (Table 5.9): the full CAFN, five single-component ablations, a two-block fusion, a v1-style configuration with all new components off, and the fusion baselines of the literature. A sweep over μ_{VEP}, with the MRI strength fixed, compares MRI-only, VEP-only, concatenation and CAFN with 2 seeds per point. Training uses AdamW with batch size 16 and at most 40 epochs with early stopping (Section 4.6). Differences between configurations are paired over seeds.

Table 3.9: Model configurations of Study A. All share the encoders, the auxiliary loss and the training recipe unless stated.

Configuration	Change relative to the full CAFN	Setting
CAFN (full)	Gated bidirectional cross-attention, positional embeddings, attention pooling, SE blocks, auxiliary loss (λ = 0.3)	defaults
w/o bidirectional	One-way fusion (spatial tokens query temporal tokens)	`fusion="cross"`
w/o positional enc.	No positional embeddings	`pos_encoding=False`
w/o attention pooling	Mean pooling of the fused tokens	`pooling="mean"`
w/o SE blocks	Plain residual blocks	`se_reduction=None`
w/o auxiliary loss	No unimodal losses	auxiliary weight 0
2 fusion blocks	Two stacked fusion blocks	`fusion_blocks=2`
v1-style (all off)	One-way fusion, no positional embeddings, mean pooling, no SE	`fusion="cross"`, `pos_encoding=False`, `pooling="mean"`, `se_reduction=None`
Concatenation	Mean-pooled streams concatenated and projected	`fusion="concat"`
Element-wise sum	Mean-pooled streams added	`fusion="sum"`
MRI only	The fused head reads the spatial stream only	`fusion="mri_only"`
VEP only	The fused head reads the temporal stream only	`fusion="vep_only"`

3.9.2 STUDY B: REAL COHORTS

Study B uses the two cohorts of Table 3.4 (95 subjects with both modalities) and three protocols. *Pooled and leave-one-cohort-out training.* Each cohort is split 70/15/15 into training, validation and test subjects, stratified by label. The pooled model trains on the union of the training splits, selects its checkpoint on the union of the validation splits and is reported separately for every cohort and for the pool. In leave-one-cohort-out (LODO) training the model is trained on one cohort and tested on every subject of the other. These runs use 64^3 MRI volumes and are repeated three times (seeds 42, 43 and 44). Context baselines are computed on the same splits: logistic regression on the 500-point spectrum, the unimodal auxiliary heads, chance, and a cohort-identity probe that asks whether the spectrum alone can tell which cohort a subject belongs to.

Repeated cross-validation. The per-cohort test splits contain about seven subjects, which cannot separate skill from luck. Every subject is therefore also predicted out-of-fold in 5-fold cross-validation stratified on cohort and label, repeated

five times for the classical models and once for the neural models, so that the metrics use all 95 subjects. The classical baselines are logistic regression on the 500-point spectrum, on five band powers (delta 1–4, theta 4–8, alpha 8–13, beta 13–30 and gamma 30–45 Hz) and on PCA-reduced MRI (8^3 voxels reduced to 20 components), a random forest on band powers, and MRI plus band powers; classical models are also evaluated with leave-one-cohort-out. The neural variants (CAFN, concatenation, MRI only, spectrum only) use a regularised configuration fixed in advance ($d_{\text{model}} = 64$, dropout 0.3, weight decay 10^{-2}) with 32^3 MRI volumes.

Statistics. The metrics are ROC-AUC and balanced accuracy, with a 1000-resample bootstrap 95% interval over subjects and a 200-permutation test that shuffles the labels within each cohort. Permuting within cohorts preserves each cohort's base rate, so that a cohort-identity shortcut cannot by itself produce a significant result.

3.9.3 METRICS AND REPORTING

Accuracy, macro-F1 and balanced accuracy are reported for the three-class synthetic task, and ROC-AUC and balanced accuracy for the binary real task (chance level for balanced accuracy is 0.33 and 0.50 respectively). Every number comes from data that were used neither for training nor for checkpoint selection. Metrics are labelled synthetic or real-cohort wherever they appear, and they are never presented as diagnostic ability: on the synthetic task they are the outcome of a controlled comparison between architectures on a generator designed to be separable, and on the real cohorts they test a pipeline on disorders other than the target ones. Because the real cohorts are small, confidence intervals and permutation p-values accompany the real-cohort metrics.

CHAPTER 4
SYSTEM DESIGN AND IMPLEMENTATION

4.1 ARCHITECTURE OVERVIEW

The Cross-Attention Fusion Network (CAFN), implemented in `src/models/cafn.py`, has three parts (Figure 4.1): two modality-specific encoders that turn each input into a sequence of 128-dimensional tokens (a 3D residual encoder with squeeze-and-excitation gates and a multi-scale 1D encoder), a gated bidirectional cross-attention fusion module in which each stream queries the other, and three linear classification heads (fused, spatial-auxiliary and temporal-auxiliary); the fused head reads a learned attention pool of the fused spatial tokens. Learned positional information is added to both token streams before fusion. This is the v2/v3 design. The original v1 prototype (one-way fusion, mean pooling, no positional information) remains available as a configuration, which allows like-for-like ablations (Chapter 5). Table 4.1 lists what changed.

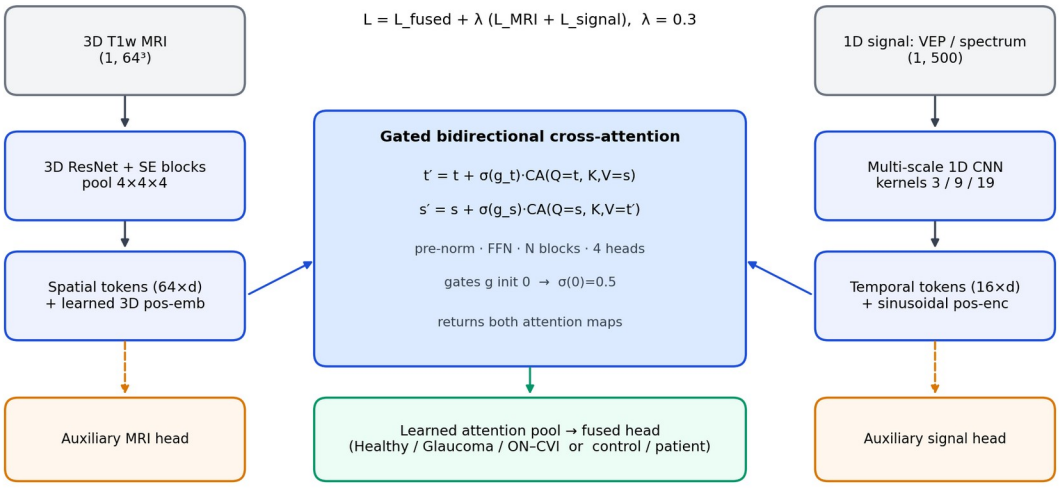


Figure 4.1: CAFN architecture (v2/v3). Preprocessed MRI and signal streams are encoded into spatial and temporal tokens, exchanged through gated bidirectional cross-attention, pooled by a learned attention pool and classified; two auxiliary heads read the unimodal tokens. In the real-cohort experiments the 1-D stream is the log power spectrum of the resting recording and the head has two classes.

Figure 4.2 shows the layer-by-layer structure of the final v2/v3 model with the tensor shape after each operation for a batch of size B. The encoder widths and the token shapes (64 × 128 and 16 × 128) are the same as in the original v1 model.

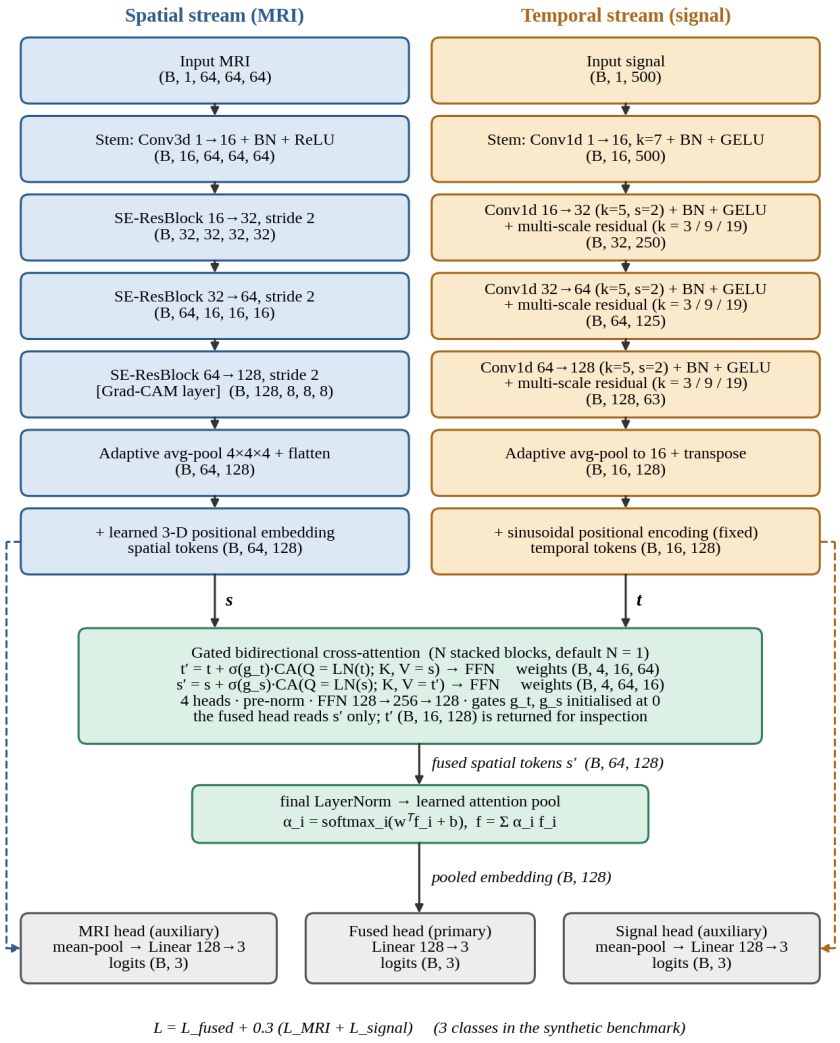


Figure 4.2: CAFN architecture (v2/v3) with tensor shapes (default configuration; B = batch size). The two encoders produce 64 spatial and 16 temporal tokens of width 128, the gated bidirectional fusion updates both, a learned attention pool of the fused spatial tokens feeds the fused head, and two auxiliary heads read the mean-pooled unimodal tokens. The original (v1) design is compared in Table 4.1.

The complete v2/v3 model has 1,955,140 trainable parameters for three classes, about 7.8 MB in 32-bit floating point (Section 5.4); the original v1 model had 1,055,081.

Table 4.1: Differences between the original prototype (v1) and the final design (v2/v3).

Aspect	v1 (original prototype)	v2/v3 (final design)
Fusion	One-way: spatial tokens query temporal tokens	Gated bidirectional: each stream queries the other; learned scalar gates ($\sigma(0) = 0.5$); pre-norm blocks; stackable
Positional information	None	Learned 3D embedding on the $4 \times 4 \times 4$ token grid; sinusoidal encoding on the temporal tokens
Pooling of the fused tokens	Mean	Learned attention pool (starts as mean pooling)
Spatial encoder	3D residual blocks	3D residual blocks with squeeze-and-excitation (reduction 4)
Temporal encoder	Four convolutional stages, one kernel per stage	Stem and three strided stages, each followed by a multi-scale residual block (kernels 3, 9 and 19)
Adaptive pooling	PyTorch adaptive average pooling	Deterministic matrix-product pooling (equal to the PyTorch operation)
Optimisation	Adam, constant learning rate, validation-loss checkpoint	AdamW, warm-up and cosine schedule, gradient clipping, early stopping, mixed precision, selectable checkpoint metric, class weights
Fusion baselines	None	Concatenation, element-wise sum, MRI only, VEP only
Trainable parameters	1,055,081	1,955,140
MACs per sample at 64^3	2.53 G	2.60 G
Automated tests	86	340

4.2 SPATIAL ENCODER (3D RESIDUAL CNN)

`SpatialEncoder3D` (`src/models/spatial_encoder.py`) converts a standardised MRI volume of shape (B, 1, 64, 64, 64) into a sequence of 64 tokens. It consists of a convolutional stem, three residual blocks that each halve the spatial resolution, increase the channel count and end with a squeeze-and-excitation gate, and a deterministic adaptive average pool to a $4 \times 4 \times 4$ grid (Table 4.2).

Table 4.2: Spatial encoder layers (batch size 1) and trainable parameters (default model; the squeeze-and-excitation gates are included).

Layer	Operation	Output shape	Parameters
Stem	Conv3d $1 \rightarrow 16$ (3×3×3, stride 1) + BatchNorm + ReLU	(16, 64, 64, 64)	464
Residual block 1	$16 \rightarrow 32$, stride 2, SE	(32, 32, 32, 32)	42,728
Residual block 2	$32 \rightarrow 64$, stride 2, SE	(64, 16, 16, 16)	170,448
Residual block 3	$64 \rightarrow 128$, stride 2, SE (Grad-CAM target `last_conv`)	(128, 8, 8, 8)	680,864
Pool and tokens	Deterministic adaptive average pool (4,4,4); flatten; transpose	(64, 128)	0

Layer	Operation	Output shape	Parameters
Total			**894,504**

Each residual block applies two 3 × 3 × 3 convolutions with batch normalisation (Ioffe & Szegedy, 2015); the first has the block's stride. A squeeze-and-excitation gate (Hu et al., 2018) recalibrates the channels of the second convolution's output before the shortcut is added: global average pooling gives one value per channel, two linear layers with a reduction factor of 4 and a ReLU between them followed by a sigmoid give a weight per channel, and the feature map is multiplied by these weights. The three gates add 11,032 parameters. The input is added to the output through an identity shortcut, or through a 1 × 1 × 1 convolution with batch normalisation when the stride or the channel count changes, and a ReLU follows the addition (He et al., 2016). The spatial size of a convolutional layer with kernel k, stride s and padding p follows Equation 4.1.

$$L_{out} = \lfloor \frac{L_{\text{in}} + 2p - k}{s} \rfloor + 1 \tag{4.1}$$

After the three stride-2 blocks the feature map is 8 × 8 × 8; adaptive average pooling reduces it to 4 × 4 × 4, so each of the 64 tokens summarises a 2 × 2 × 2 neighbourhood of feature-map positions, i.e. a nominal 16 × 16 × 16 voxel cube of the standardised (1 mm) volume. In CAFN a learned positional embedding of shape 64 × 128 (8,192 parameters), indexed like the token grid, is added to these tokens before fusion. The final residual block is stored under the name `last_conv` so that the Grad-CAM implementation can attach its hooks to it.

4.3 TEMPORAL ENCODER (1D CNN)

`TemporalEncoder1D` (`src/models/temporal_encoder.py`) maps a conditioned VEP, or a spectrum, of shape (B, 1, 500) to 16 tokens through a stem and three strided stages, each followed by a multi-scale residual block, and a deterministic adaptive average pool (Table 4.3). A multi-scale block applies three parallel 1D convolutions with kernels of 3, 9 and 19 samples to its input, concatenates their outputs, fuses them with a 1 × 1 convolution and batch normalisation, and adds the result to the input through a residual connection followed by a GELU. The short kernels see fine waveform detail such as the P100 peak and the long kernels the extent of the N75–P100–N145 complex.

Table 4.3: Temporal encoder layers (batch size 1) and trainable parameters.

Stage	Operation	Output length	Parameters
Stem	Conv1d 1→16, kernel 7, stride 1 + BatchNorm + GELU	500	144
Stage 1	Conv1d 16→32, kernel 5, stride 2 + BatchNorm + GELU	250	2,624
Multi-scale block 1	Kernels 3/9/19 on 32 channels, 1×1 fusion, residual	250	34,880
Stage 2	Conv1d 32→64, kernel 5, stride 2 + BatchNorm + GELU	125	10,368
Multi-scale block 2	Kernels 3/9/19 on 64 channels, 1×1 fusion, residual	125	139,392
Stage 3	Conv1d 64→128, kernel 5, stride 2 + BatchNorm + GELU	63	41,216
Multi-scale block 3	Kernels 3/9/19 on 128 channels, 1×1 fusion, residual	63	557,312
Pool	Deterministic adaptive average pool (16); transpose → (16, 128)	16	0
Total			**785,936**

The strided stages bring the 500 samples down to 63 positions (an output stride of 8 samples, 32 ms at 250 Hz), which the adaptive pool averages into 16 tokens, so that each token nominally summarises about 125 ms of the 2 s epoch (Figure 4.3). The tokens are learned feature positions: without further temporal registration they should not be read as calibrated millisecond bins. A fixed sinusoidal positional encoding is added to the temporal tokens in CAFN. In spectrum mode the same encoder operates on a spectrum, and the token axis corresponds to frequency, not time.

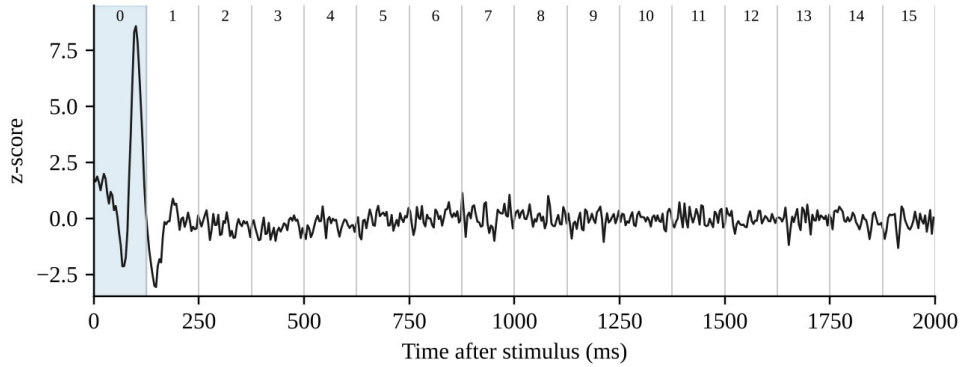


Figure 4.3: Nominal mapping of the 16 temporal tokens onto the 2 s epoch (125 ms per token) over an example conditioned VEP. The shaded region is token 0.

4.4 CROSS-ATTENTION FUSION MODULE

The fusion module (`src/models/cross_attention.py`) is the central component. Its building block is multi-head cross-attention in which the queries come from one token sequence and the keys and values from the other. In the original (v1) design only the spatial tokens X_s (64 × 128) supplied the queries and the temporal tokens X_t (16 × 128) supplied the keys and values (Equation 4.2), so that each spatial token could attend to any of the 16 temporal tokens.

$$Q = X_s W_Q, K = X_t W_K, V = X_t W_V \tag{4.2}$$

With d_{model} = 128 and H = 4 heads, each head has dimension d_h = 32. For head h the scaled dot-product attention is computed (Vaswani et al., 2017), with a softmax over the 16 temporal tokens, followed by dropout on the attention weights (p = 0.1):

$$A_h = softmax\left(\frac{Q_h K_h^T}{\sqrt{d_h}}\right), O_h = A_h V_h \tag{4.3}$$

Gated bidirectional block (final design). The final module updates both streams in each of N pre-norm blocks (N = `fusion_blocks`, 1 by default), the temporal stream first so that the spatial read, which feeds the fused head, consumes the enriched temporal tokens. With LN layer normalisation, CA(q, kv) the attention of Equations 4.2 and 4.3 with queries from q and keys and values from kv, and FFN_t, FFN_s position-wise feed-forward blocks (128 → 256 → 128 with GELU and dropout), the updates are

$$t' = t + \sigma(g_t) CA(\ln(t), s), t' \leftarrow t' + FFN_t(\ln(t')) \tag{4.4}$$

$$s' = s + \sigma(g_s) CA(\ln(s), t'), s' \leftarrow s' + FFN_s(\ln(s')) \tag{4.5}$$

Here g_t and g_s are learned scalar gates, initialised at zero so that σ(g) = σ(0) = 0.5: both directions start half open, and each can learn to open or close. The gates are shared by all blocks. A final layer normalisation is applied to the spatial tokens that the fused head reads. The module returns the attention weights of both directions: the forward map (B, H, 64, 16), in which spatial tokens read temporal tokens, and the backward map (B, H, 16, 64), in which temporal tokens read spatial tokens. Before fusion, a learned positional embedding is added to the 4 × 4 × 4 spatial token grid and a fixed sinusoidal encoding to the temporal tokens, so that a token can carry the identity of the region or latency window it represents; without positions the attention

map would only correlate bags of features. Optionally, with modality dropout, the temporal tokens of a random subset of training samples are zeroed inside the fusion path only, while the auxiliary heads still see clean inputs. Figure 4.4 shows one such block.

For comparison, the original v1 block updated only the spatial stream and normalised after the residual connections: the head outputs are concatenated and projected, added to the original spatial tokens through a residual connection, normalised (Ba et al., 2016), passed through a position-wise feed-forward block (128 → 256 → 128 with ReLU and dropout) and normalised again after a second residual connection (Equation 4.6). The configuration `fusion="cross"` of the current code keeps the one-way direction but uses pre-norm blocks like the bidirectional module, so that the "w/o bidirectional" ablation of Study A isolates the direction of attention.

$$Z = \ln\left(X_s + Concat\left(O_1, \ldots, O_H\right) W_O\right), F = \ln\left(Z + FFN\left(Z\right)\right) \tag{4.6}$$

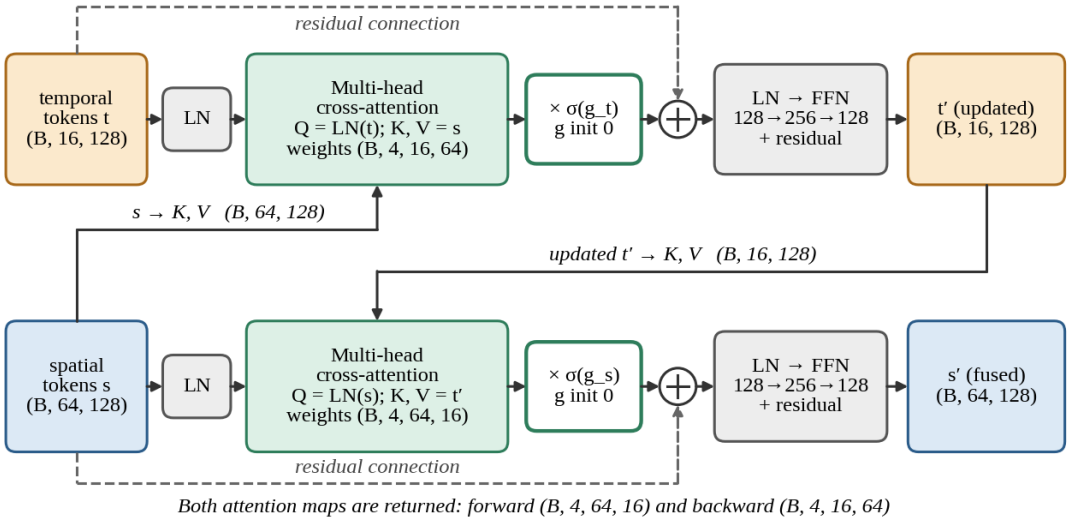


Figure 4.4: One gated bidirectional block of the cross-attention fusion module (CAFM), final design (Equations 4.4 and 4.5). The temporal tokens first query the spatial tokens (attention weights of shape (B, 4, 16, 64)); the updated temporal tokens then serve as keys and values for the spatial queries (weights (B, 4, 64, 16)). Each cross-attention output is scaled by a learned scalar gate initialised at zero, added residually and followed by a feed-forward block; both attention maps are returned for inspection.

The v1 output F keeps the shape (B, 64, 128): the spatial-token sequence is preserved and each token is enriched with temporal context. The module accepts queries and keys of different lengths and rejects configurations in which d_{model} is not divisible by the number of heads. The v1 module (projection matrices, output projection, feed-forward block and two layer norms) has 132,480 trainable

parameters; the default gated bidirectional module has 265,218: two attention modules of 66,048 parameters each, two feed-forward blocks of 65,920, five layer norms (1,280) and the two gates (Table 5.5).

Fusion baselines. For like-for-like comparison, other fusion strategies can be selected with `ModelConfig.fusion` (Table 4.4) while the encoders, the auxiliary heads and the training recipe stay the same. The concatenation and element-wise sum baselines are those used in the literature (Zhou et al., 2024), and the two unimodal references show what each modality achieves alone.

Table 4.4: Fusion strategies selectable with ModelConfig.fusion.

fusion	Input of the fused head	Attention maps returned
`"bidirectional"` (default)	Learned attention pool of the gated, bidirectionally fused spatial tokens	Forward and backward
`"cross"`	One-way cross-attention (spatial tokens query temporal tokens), pooled according to `pooling`	Forward only
`"concat"`	Mean-pooled spatial and temporal embeddings concatenated and projected (256 → 128, GELU, dropout)	None
`"sum"`	Element-wise sum of the mean-pooled spatial and temporal embeddings	None
`"mri_only"`	Mean-pooled spatial tokens	None
`"vep_only"`	Mean-pooled temporal tokens	None

4.5 CLASSIFICATION HEADS AND MULTI-TASK LOSS

`CAFN.forward` returns three sets of logits of shape (B, C), with C = 3 classes for the synthetic task and 2 for the real cohorts. The fused head reads a learned attention pool of the fused spatial tokens: a linear scorer gives one score per token, a softmax turns the scores into weights α_i, and the pooled embedding is their weighted sum (Equation 4.7), where f_i are the fused tokens, and w and b the scorer's weights and bias. The scorer is initialised with small weights, so that the weights start almost uniform and the pool starts as mean pooling; it can then learn to emphasise pathology-bearing tokens, and the weights are returned for explainability.

$$\alpha_i = \frac{\exp\left(w^\top f_i + b\right)}{\sum_j \exp\left(w^\top f_j + b\right)}, z = \sum_i \alpha_i f_i \tag{4.7}$$

The two auxiliary heads read the mean-pooled spatial tokens and the mean-pooled temporal tokens. The fused head is the primary prediction used at evaluation

time; the auxiliary heads provide unimodal supervision: they encourage each encoder to carry class information on its own and reduce the chance that the fused pathway learns to ignore one modality entirely. With cross-entropy L_f, L_s and L_t for the fused, spatial and temporal heads, the objective is

$$L_{total} = L_f + \lambda(L_s + L_t), \lambda = 0.3 \tag{4.8}$$

The function `CAFN.multi_task_loss` (implemented in `src/models/losses.py`) returns the total together with the three component values, so that each can be logged. Optional extensions are inverse-frequency class weights in every head, label smoothing, and a learnable task-uncertainty weighting; the experiments of this report use the fixed weight λ = 0.3, with class weights in the real-cohort runs (Section 4.6).

4.6 TRAINING PROCEDURE

Training is implemented in `src/train.py` and summarised in Figure 4.5. For each epoch, `train_model` performs the following steps.

1. Put the model in training mode and iterate over a shuffled `DataLoader` (batch size 8 by default), with train-time augmentation applied to the training indices only (MRI flips, intensity jitter and noise; VEP amplitude scaling and noise; the waveform is not rolled or substantially shifted, because latency carries class information).
2. Move the MRI, signal and label tensors to the selected device (CUDA, Apple MPS or CPU, chosen automatically).
3. Clear gradients, run the forward pass (with mixed precision when enabled) and compute the multi-task loss with the configured auxiliary weight and, optionally, class weights.
4. Back-propagate, clip the gradient norm (1.0) and take an AdamW step (learning rate 10^{-3}, weight decay 10^{-4} by default); the learning rate follows a linear warm-up over the first 10% of the steps and then a cosine decay, stepped per batch.
5. If a validation loader is supplied, evaluate the fused head with gradients disabled and the model in evaluation mode, recording loss, accuracy, balanced accuracy and macro-F1. The checkpoint that is restored at the end is chosen by `select_metric` (validation loss by default; validation balanced accuracy in

the experiments of this report), and training stops early when that quantity has not improved for `patience` epochs.

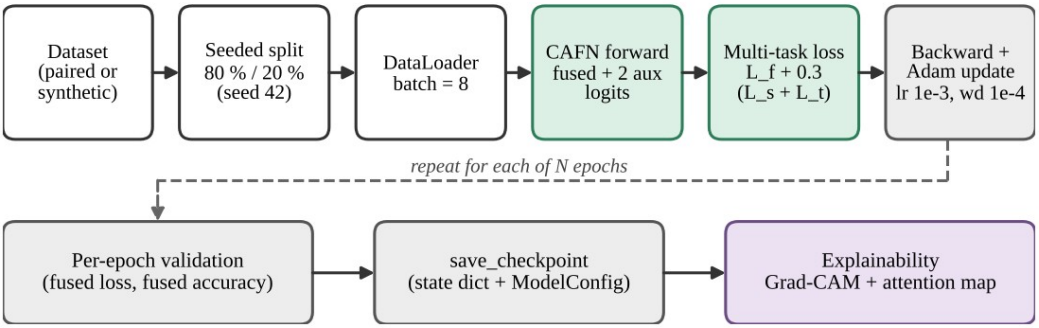


Figure 4.5: Training and evaluation flow. A seeded permutation splits the data; each epoch is followed by validation; the trained model can be checkpointed and passed to the explainability tools.

Table 4.5: Training configuration (TrainConfig, left) and model configuration (ModelConfig, right) at the default values. The studies override several of them (see the text).

TrainConfig	Default	ModelConfig	Default
`batch_size`	8	`d_model`	128
`epochs`	20	`num_heads`	4
`lr`	1e-3	`spatial_tokens` / `spatial_pool`	64 / (4, 4, 4)
`weight_decay`	1e-4	`temporal_tokens`	16
`aux_loss_weight` (λ)	0.3	`ffn_hidden`	256
`val_split`	0.2	`dropout`	0.1
`seed`	42	`num_classes`	3
`optimizer`	"adamw"	`fusion`	"bidirectional"
`scheduler`	"cosine"	`pos_encoding`	True
`warmup_fraction`	0.1	`pooling`	"attn"
`max_grad_norm`	1.0	`modality_dropout`	0.0
`patience`	None	`fusion_blocks`	1
`amp`	False	`se_reduction`	4
`class_weights`	None		
`label_smoothing`	0.0		
`deterministic`	False		
`select_metric`	"val_loss"		
`augment`	True		

Splitting. `train_val_split` draws a deterministic random permutation with a seeded generator, assigns the first max(1, int(n · val_split)) indices to validation and the rest to training, and falls back to using all samples for training when a very small

dataset would otherwise leave none. Label-stratified and subject-grouped splits (`src/data/splits.py`) are also available: they preserve the class proportions and keep all scans of one subject on one side. **Checkpoints.** `save_checkpoint` stores the model's state dictionary together with its `ModelConfig`; `load_checkpoint` rebuilds the model from the stored configuration, upgrading older checkpoints to the current field set, and loads the weights. Each run writes the best and last checkpoints, the training history, the final metrics and the training curves to its own directory, and training can resume from the last checkpoint. **Command line.** `python -m src.train` accepts `--manifest` for real data (or `--synthetic-samples` for synthetic data) and options for the optimiser, scheduler, warm-up, fusion mode, positional encoding, patience, mixed precision, class weights, subject grouping, a held-out test split, evaluation of a saved checkpoint (`--eval-only`) and deterministic algorithms.

Recipes of the studies. Study A (Section 3.9.1) uses AdamW with learning rate 10^{-3} and weight decay 10^{-4}, batch size 16, at most 40 epochs with early stopping (patience 12), mixed precision, deterministic algorithms, augmentation and checkpoint selection on validation balanced accuracy, with fixed seeds. The pooled and leave-one-cohort-out runs of Study B use AdamW with learning rate 5×10^{-4} and weight decay 10^{-2}, batch size 8 (halved automatically after a GPU out-of-memory error), at most 120 epochs with early stopping (patience 25), an inverse-frequency class-weighted loss, augmentation, mixed precision, deterministic algorithms and checkpoint selection on validation balanced accuracy. The neural variants of the cross-validation study use the regularised configuration of Section 3.9.2 ($d_{\text{model}} = 64$, feed-forward width 128, dropout 0.3), the same optimiser settings, at most 60 epochs and patience 15. Checkpoint selection on validation balanced accuracy replaced the default selection on validation loss after short runs were seen to restore an untrained first-epoch model whose loss happened to be lowest (Section 6.4.4).

Reproducibility. All randomness derives from one constant seed (`SEED = 42` in the real-cohort scripts; repeat k uses $42 + k$); deterministic algorithms are requested and the cuBLAS workspace and `PYTHONHASHSEED` are fixed; data-loader workers are seeded; and the adaptive pooling is the deterministic matrix-product version of `src/models/pooling.py`, which equals PyTorch's operation, is deterministic on

CUDA and runs on Apple MPS. `run_all.sh` records the date, git commit, seed and the full package list of every run, and downloads are resumable and size-verified. Neural models were trained on an NVIDIA RTX 3050 Laptop GPU (4 GB, thermally throttled during long runs); to finish in time, some synthetic-benchmark runs were distributed over an Apple M3 Max. Every run is reproducible on its own device at a fixed seed, but results are not bit-identical across devices; all configurations of a given seed ran on the same device, so the paired comparisons between configurations are not affected. The model needs under 1 GB of memory at batch size 8.

4.7 EXPLAINABILITY

4.7.1 3D GRAD-CAM ON THE FUSED LOGIT

`GradCAM3D` (`src/explainability/grad_cam_3d.py`) registers a forward hook and a backward hook on the spatial encoder's last residual block. It back-propagates from the selected *fused* logit, by default that of the predicted class, so that the gradients flow through the cross-attention module into the spatial encoder and the map reflects the multimodal decision. With activations A^k of channel k and class score y^c, the channel weights are the spatially averaged gradients and the map is a rectified weighted sum (Selvaraju et al., 2017):

$$\alpha_k = \frac{1}{Z}\sum_{i,j,l}\frac{\partial y^c}{\partial A^k_{ijl}}, \quad M = ReLU\left(\sum_k \alpha_k A^k\right) \tag{4.9}$$

The map is upsampled by trilinear interpolation to the input shape and normalised per sample to [0, 1]. The helper `plot_gradcam_slices` overlays it on the mid axial, coronal and sagittal slices and saves a PNG. The model is returned to its original training or evaluation mode and the hooks are removed after use.

A Grad-CAM++ variant of the map, which weights the gradients per location, can be selected instead.

4.7.2 CROSS-ATTENTION HEATMAP

`cross_attention_heatmap` averages the attention tensor (B, H, S, T) over the batch and the heads to a matrix of shape (S, T), 64 × 16 at the default configuration, and `plot_cross_attention` renders it as a heatmap with spatial tokens on one axis and temporal tokens on the other. With the bidirectional module both directions are returned, the forward map (spatial tokens read temporal tokens)

and the backward map (temporal tokens read spatial tokens), `plot_attention_by_head` renders the maps head by head, and the weights of the attention pool give a token-salience map.

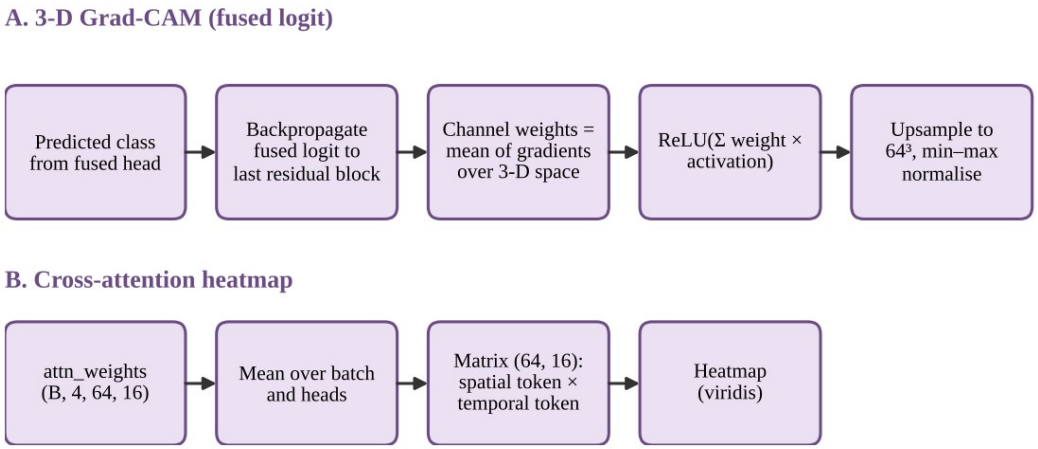


Figure 4.6: Generation of the two explanation artefacts: the Grad-CAM map (A) and the cross-attention heatmap (B).

Interpretation caveat. Both artefacts are inspection aids. The Grad-CAM map shows which feature locations influenced the selected logit; it does not by itself identify V1, V2, V3, the optic nerve or any other anatomical structure. The heatmap axes are learned token indices, not registered brain regions or calibrated milliseconds. Chapter 6 discusses why attention and saliency maps must be validated before being offered as clinical evidence.

4.7.3 P100 LATENCY ESTIMATOR

`src/explainability/p100.py` estimates the P100 latency of a conditioned sweep as the time of the maximum-amplitude sample inside an ISCEV-aligned search window (70 to 180 ms), after a defensive z-score, and also reports the amplitude, the peak index and a rough half-maximum width for quality control. It turns the VEP stream into a number that a clinician can check against the waveform, and it relies on the zero-phase conditioning for the timing to be trustworthy. It is an inspection aid, not clinical evidence.

4.8 SOFTWARE STRUCTURE AND CONFIGURATION

The code base is organised by responsibility (Figure 4.7 and Table 4.6). The configuration module is the only place in which shapes and hyper-parameters are defined; the remaining modules import it. Figure 4.7 shows all 26 modules of the

current code base: the twelve modules of the original code base have solid outlines and the fourteen modules added since have dashed outlines. Table 4.6 lists them with their line counts; the additions include the deterministic pooling, attention pool, positional embeddings and loss module, the cohort loaders (`src/data/cohorts.py`), the two real-cohort experiment runners (`src/cohort_experiments.py` and `src/cv_real.py`), the synthetic benchmark (`src/ablation_synthetic.py`), the dataset downloader (`scripts/download_datasets.py`) and the one-command pipeline `run_all.sh`.

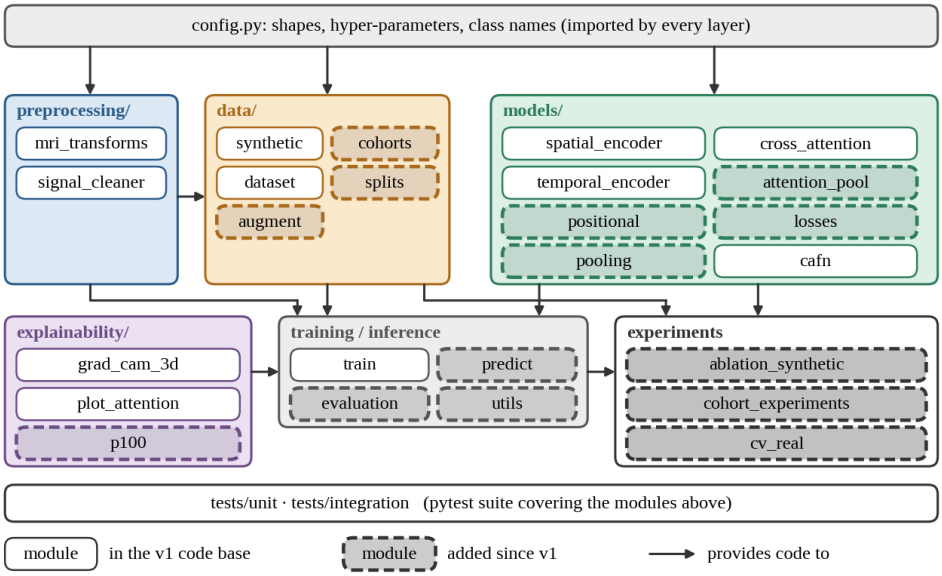


Figure 4.7: Module structure and import dependencies of the current code base (26 modules, with the test suite at the bottom). Solid outlines: the twelve modules of the original code base; dashed shaded outlines: the fourteen modules added since (listed in Table 4.6). Arrows run from the providing group to the importing group.

Table 4.6: Source modules, line counts and responsibilities (current working tree).

Module	Lines	Responsibility
`src/config.py`	202	Frozen dataclasses (MRI, VEP, model, training, augmentation); class names; upgrade of legacy checkpoint configurations
`src/data/synthetic.py`	215	Synthetic MRI and VEP generators with controllable strengths; on-disk dataset writer
`src/data/dataset.py`	352	PairedNeuroDataset (validated manifest), SyntheticNeuroDataset, augmentation wrapper, class weights
`src/data/cohorts.py`	354	Real-cohort discovery (BIDS), log-PSD spectrum extraction, T1-weighted caching, manifest
`src/data/splits.py`	291	Stratified and subject-grouped train/validation/test splits
`src/data/augment.py`	142	Train-time MRI and VEP augmentation

Module	Lines	Responsibility
src/preprocessing/ mri_transforms.py	229	NIfTI loading, resampling, percentile or Otsu masking, scaling, centre crop or bounding-box resize
src/preprocessing/ signal_cleaner.py	132	Validation, band-pass, notch, baseline correction, resampling, z-score, spectrum mode
src/models/ spatial_encoder.py	92	3D residual CNN with squeeze-and-excitation producing spatial tokens
src/models/ temporal_encoder.py	70	Multi-scale 1D CNN producing temporal tokens
src/models/ cross_attention.py	197	Multi-head cross-attention; one-way and gated bidirectional fusion
src/models/ attention_pool.py	41	Learned attention pooling
src/models/ positional.py	104	Learned 3D spatial and sinusoidal temporal positional encodings
src/models/pooling.py	55	Deterministic adaptive average pooling
src/models/losses.py	86	Multi-task cross-entropy and uncertainty-weighted loss
src/models/cafn.py	207	Full model, fusion modes (bidirectional, one-way, concatenation, sum, unimodal) and heads
src/train.py	940	Training, evaluation, split, checkpoints, command line
src/evaluation.py	251	Detailed metrics, confusion matrix, training-curve plots
src/utils.py	300	Seeding, device selection, label and group utilities
src/predict.py	149	Single-subject inference with an evidence bundle
src/explainability/ grad_cam_3d.py	213	3D Grad-CAM and Grad-CAM++ and slice plots
src/explainability/ plot_attention.py	105	Attention reduction and per-direction, per-head heatmaps
src/explainability/ p100.py	137	P100 latency and amplitude estimator
src/ ablation_synthetic.py	160	Study A runner: fusion baselines, ablations, informativeness sweep
src/ cohort_experiments.py	423	Study B runner: pooled and leave-one-cohort-out training
src/cv_real.py	280	Study B runner: repeated cross-validation, baselines, bootstrap, permutation tests
scripts/ download_datasets.py	224	Resumable, size-verified downloader for the public cohorts
run_all.sh	155	One-command pipeline: environment, self-check, download, preprocess, train, validate, test

One-command pipeline. `./run_all.sh` performs, in order, the environment set-up (virtual environment, dependencies, GPU check and an environment snapshot with the git commit and the seed), a pre-flight test of the cohort loaders and the experiment runner, the download of the two cohorts, the preprocessing (T1-weighted volumes to 64^3 arrays, resting recordings to 500-point log power spectra, manifest)

and the training, validation and testing of the pooled and leave-one-cohort-out models. It is safe to interrupt and re-run, because downloads resume, preprocessed subjects are cached and finished training runs are skipped. `./run_all.sh --smoke` runs one subject per diagnostic group for two epochs as a plumbing check. The repeated cross-validation study and the synthetic benchmark are run with `python -m src.cv_real` and `python -m src.ablation_synthetic` (Appendix 4).

4.9 TEST DESIGN

The test suite (`tests/`) now contains 340 test functions in 29 files: 310 unit tests and 30 integration tests (the original suite had 86 tests in 12 files). A shared `tiny_config` fixture scales the model down (MRI shape 32^3, VEP length 200, $d_{\text{model}} = 32$, two heads, eight spatial and eight temporal tokens, three epochs) so that tests run quickly while exercising the same interfaces. Fixtures also provide a seeded random generator, a factory that writes synthetic NIfTI files with non-isotropic spacing, and a VEP-like array with known 10 Hz and 50 Hz components. All tests run with fixed seeds. Besides the original modules, the suite covers the v2/v3 additions: bidirectional fusion and gating, positional embeddings, attention pooling, the fusion baselines, the controllable generator strengths, deterministic pooling, Otsu and bounding-box preprocessing, the cohort loaders, the experiment runners, augmentation, evaluation metrics and the command-line tools (Table 4.7).

Table 4.7: Test files and what they verify (current suite: 340 tests in 29 files).

Test file	Tests	Properties verified
`unit/ test_utils_extra.py`	31	Device-resolution fallbacks; label and group extraction on subsets; split edge cases; baseline-correction guards
`unit/ test_upgrades_v3.py`	26	Subject-grouped splitting; per-index deterministic synthetic data; manifest validation; Otsu and z-score preprocessing; command-line smoke tests
`unit/test_splits.py`	20	Stratified, grouped and three-way splits: partitions, class ratios, determinism, degenerate cases
`unit/ test_cafn_extras.py`	19	Positional embeddings; attention pooling (a zeroed scorer equals mean pooling); modality dropout; class weights; label smoothing; uncertainty-weighted loss
`unit/ test_signal_cleaner.py`	18	Output shape; attenuation of drift and mains hum; preservation of in-band content; rejection of invalid input; mean 0 / std 1; P100-like peak latency is not shifted
`unit/ test_evaluation.py`	16	Balanced accuracy, macro-F1, ROC-AUC, confusion matrix, input validation, plots

Test file	Tests	Properties verified
unit/test_guards.py	15	Configuration validation; legacy checkpoint merging; embedding API; attention-plot input variants; degenerate splits
unit/test_cafn_v2.py	14	Both fused streams and both attention directions; gates; positional information; gradient flow to all parameters; fusion-mode selection
unit/test_mri_transforms.py	14	RAS orientation; voxel zooms; masking; anisotropic resampling; output shape and range; constant volumes; bounding-box fit of a whole head
unit/test_p100.py	14	P100 latency recovery; neuritis later than healthy; window handling; input validation; effective-rate correction
unit/test_utils.py	12	Seeding; device selection; label extraction; stratified split on small cohorts
unit/test_augment.py	11	Flips, rotations, intensity jitter, noise, bounded VEP shift; determinism given the generator; disabled means identity
unit/test_cohort_experiments.py	10	Per-cohort splits; binary metrics; bootstrap interval; end-to-end pooled and leave-one-cohort-out run; resumption; out-of-memory fallback; cohort-identity probe; checkpoint selection
unit/test_cohorts.py	10	Log-PSD extraction; window placement; EEG channel selection; BIDS discovery; manifest round-trip; spectrum mode skips filtering
unit/test_fusion_variants.py	10	Every fusion mode in forward and backward passes; unimodal references ignore the other modality; generator strength 0 removes class signal
unit/test_cross_attention.py	8	Output and attention shapes; attention rows sum to one; mismatched query/key lengths; divisibility check; finite outputs; gradients reach all parameters
unit/test_explainability_v2.py	8	Grad-CAM++ shapes and differences from Grad-CAM; direction-aware attention heatmaps; per-head plots
unit/test_grad_cam_3d.py	8	Map shape and [0, 1] range; explicit target class respected; hooks removed; training mode restored; PNG written
unit/test_pooling.py	8	Deterministic pooling equals PyTorch's pooling in forward and backward passes; backward is bitwise repeatable
unit/test_cafn_model.py	7	Output keys and shapes; batch size one; finite logits; loss equals the weighted sum of parts; backward reaches every parameter
unit/test_dataset.py	7	Item shapes and types; length; balanced labels; data-loader batching; seed dependence; manifest reading
unit/test_cli_v2.py	6	Command line: held-out test split with report and plots; evaluation of a saved checkpoint; manifest mode; failure isolation
unit/test_plot_attention.py	5	Reduction over batch and heads; non-negative values; PNG written; matches manual reduction
unit/test_spatial_encoder.py	5	Output shape; batch size one; gradient flow; hook target participates; eval-mode determinism

Test file	Tests	Properties verified
`unit/ test_temporal_encoder .py`	4	Output shape; batch size one; gradient flow; robustness to input length
`unit/ test_train_utils.py`	4	Split partitions the dataset; single-sample fallback; determinism for a fixed seed; empty loader
`integration/ test_train_upgrade.py`	21	Rich metrics; class weighting; augmentation views; scheduler; caching; early stopping; resume; checkpoint round-trip; prediction CLI
`integration/ test_pipeline_end_to_ end.py`	5	Raw files → preprocessing → model → loss → backward → optimiser step; shape compatibility; Grad-CAM and attention plots; command-line entry point; glaucoma and neuritis VEP peaks distinguishable
`integration/ test_training_loop.py`	4	Training loss falls substantially; separation of synthetic classes above a minimum validation accuracy; training without a validation set; checkpoint round-trip

Verification on two platforms. All 340 tests pass on macOS (Python 3.14) and on Ubuntu 22.04 under WSL2 (Python 3.12, CUDA), where one test is skipped. Running on the second platform exposed three latent defects that the first platform had masked: a type annotation in `utils.py` that raised a `NameError` on Python 3.13 and earlier, a device mismatch in the evaluation-only path (`--eval-only`) on CUDA, and a unit test whose assertion was mathematically vacuous. All three were fixed. The two real-file readers (CTF MEG and EDF polysomnography) were also checked on real recordings.

CHAPTER 5
RESULTS

5.1 INTRODUCTION

This chapter presents the measurements obtained for the project. Following the evaluation strategy of Section 3.9, the results concern (i) the behaviour of the signal-conditioning pipeline, (ii) the properties of the synthetic data, (iii) the size and computational cost of the model, (iv) the results of the automated verification (test suite), and (v) the experiments with trained models: the controlled synthetic benchmark of Study A (Sections 5.9 and 5.10) and the real-cohort experiments of Study B (Sections 5.11 and 5.12). They are presented without interpretation; the interpretation follows in Chapter 6.

Every result is labelled as synthetic or real-cohort. Synthetic results come from a controlled benchmark and are not clinical performance; real-cohort results come from public cohorts that are not eye-disease data (Section 3.4.2) and are reported as found. The measurements of Sections 5.2 and 5.3 were produced by running the project's own functions on seeded synthetic data from the original generator (Section 3.5; 50 trials per class, 150 in total; sampling rate 250 Hz; 500 samples per record). Section 5.4 reports the model size of the final design and Section 5.6 the test results; the coverage figure there belongs to the original code base. Training and evaluation of the model were carried out after the first prototype, and their results are reported in Sections 5.9 to 5.12. No clinical or diagnostic accuracy is claimed anywhere in this report.

5.2 SIGNAL CONDITIONING RESULTS

Generator used in Sections 5.2 and 5.3: all measurements below were made on the original synthetic generator (Sections 3.5.1 to 3.5.3; P100 latencies 100, 108 and 140 ms; block-shaped lesions), not on the current generator of Section 3.5.4.

5.2.1 FREQUENCY RESPONSE

The combined frequency response of the zero-phase conditioning filters (fourth-order Butterworth band-pass, 1–100 Hz, followed by a 50 Hz notch with Q = 30, each applied forward and backward) is shown in Figure 5.1 and sampled at selected

frequencies in Table 5.1. The −6 dB bandwidth of the notch (the −3 dB point of a single pass) extends from 49.18 to 50.83 Hz.

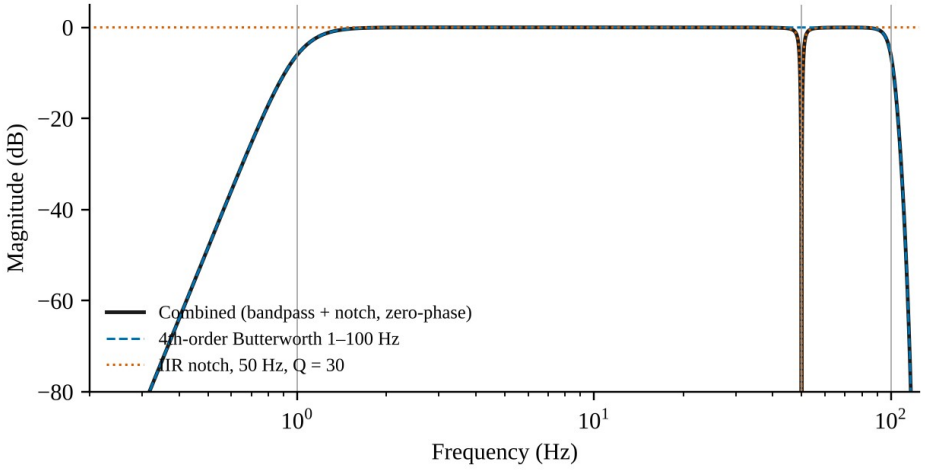


Figure 5.1: Magnitude response of the conditioning filters in the effective (forward-backward) form.

Table 5.1: Effective magnitude response (dB) at selected frequencies.

Frequency (Hz)	Band-pass	Notch	Combined
0.3	-82.73	0.00	-82.73
1.0	-5.78	0.00	-5.78
10.0	0.00	0.00	0.00
49.0	0.00	-4.52	-4.52
50.0	0.00	-97.45	-97.45
51.0	0.00	-4.65	-4.65
100.0	-6.03	0.00	-6.03

5.2.2 MAINS-INTERFERENCE ATTENUATION

A least-squares sinusoid fit at 50 Hz was used to measure the amplitude of the injected mains component before and after band-pass and notch filtering. Averaged over the 150 trials, the fitted 50 Hz amplitude fell from 0.152 to 0.0125 (arbitrary units), a mean reduction of 22.0 dB. For reference, the residual amplitude fitted at 47 Hz after filtering, a frequency at which no component was injected, was 0.0212. Figure 5.2 shows the amplitude spectrum of one example record.

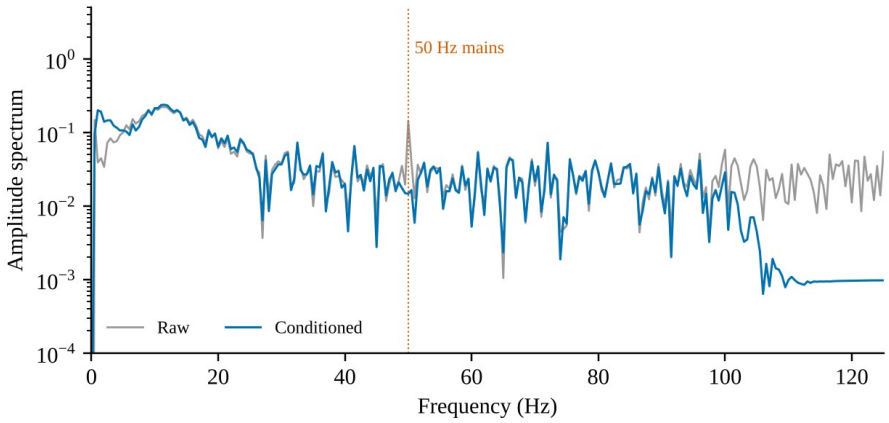


Figure 5.2: Amplitude spectrum of one synthetic VEP (original generator) before and after conditioning. The dotted line marks 50 Hz.

5.2.3 P100 LATENCY

The P100 latency was estimated as the time of the maximum within 60–220 ms. Table 5.2 compares the estimates from the raw signal, from the project's zero-phase conditioning, and from an ordinary single-pass (causal) implementation of the same filters, with the ground-truth latency of the generator. Estimates are quantised by the sampling interval of 4 ms.

Table 5.2: Detected P100 latency (ms; mean ± SD over 50 trials per class, original generator) and mean error against the ground truth.

Class	True	Raw	Zero-phase	Single-pass	Err. zero-phase	Err. single-pass
Healthy Control	100	101.8 ± 2.3	100.6 ± 1.9	101.1 ± 1.8	+0.64	+1.12
Glaucoma	108	107.4 ± 2.7	108.0 ± 2.9	108.2 ± 2.3	+0.00	+0.24
Optic Neuritis / CVI	140	141.3 ± 2.5	140.8 ± 2.1	141.0 ± 2.7	+0.80	+1.04

With zero-phase conditioning, 149 of 150 trials (99.3%) gave a latency within one sample period (4 ms) of the ground truth; the largest absolute error was 8 ms, which occurred in one Glaucoma trial. For the single-pass implementation the corresponding count was 149 of 150, and the largest raw-signal error was 8 ms. The group delay of the single-pass filter, computed from its transfer function, is given in Table 5.3 (the zero-phase pipeline has zero group delay at all frequencies) and plotted in Figure 5.3.

Table 5.3: Group delay of the single-pass (causal) implementation of the same filters.

Frequency (Hz)	Group delay (ms)
1	593.1
2	120.7
5	18.8
10	6.1
20	3.1
30	2.8
40	3.9
60	4.8

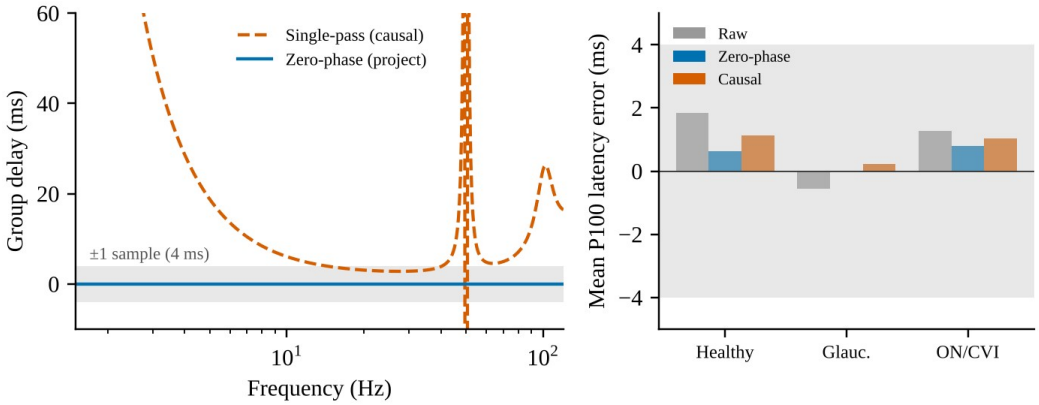


Figure 5.3: Left: group delay of the single-pass filters versus the zero-phase pipeline (shaded band: ±1 sample, 4 ms). Right: mean P100 latency error by class (original generator; shaded band: ±4 ms).

5.2.4 CONDITIONED-WAVEFORM STATISTICS

After conditioning and z-scoring, the peak value of the P100 window was 8.38 ± 0.27 for Healthy Control, 6.85 ± 0.41 for Glaucoma and 7.51 ± 0.54 for Optic Neuritis / CVI (mean ± SD, 50 trials per class). Figure 5.4 shows the distributions of the detected latency and of the peak value.

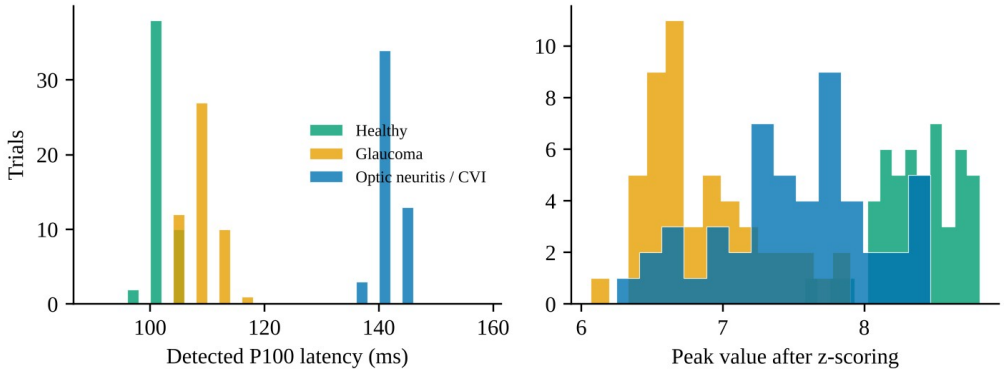


Figure 5.4: Distributions over 50 trials per class from the original synthetic generator: detected P100 latency after conditioning (left) and peak value after z-scoring (right).

5.2.5 NOMINAL TEMPORAL-TOKEN BINS

The temporal encoder reduces the 500-sample record to 16 tokens, so that each token nominally spans 125 ms of the 2.0 s record. The ground-truth P100 latencies of the three classes (100, 108 and 140 ms) fall in token 0 for Healthy Control, token 0 for Glaucoma and token 1 for Optic Neuritis / CVI (the token mapping is illustrated in Figure 4.3).

5.3 SYNTHETIC MRI CHARACTERISATION

One seeded synthetic volume of shape (80, 96, 64) was generated per class and the intensity statistics of the class-specific regions of interest (ROI) were measured, together with the effect of the project's foreground-mask function at its default 60th percentile (Table 5.4). The ROIs are the atrophy-proxy block [D/2:, :H/2, :W/2] and the lesion-proxy block [:D/2, H/2:, W/2:] described in Section 3.5. These measurements used the original generator; the current generator places lesions at random positions, so that fixed regions of interest no longer apply.

Table 5.4: Synthetic MRI (original generator): mean intensity of regions of interest and effect of the 60th-percentile foreground mask.

Class	Mean, rest of volume	Mean, atrophy ROI	Mean, lesion ROI	Voxels zeroed (%)	Atrophy ROI retained (%)	Lesion ROI retained (%)
Healthy Control	100.0	99.9	100.0	60.0	39.7	40.0
Glaucoma	100.0	40.0	100.0	60.0	0.0	45.7
Optic Neuritis / CVI	100.0	100.1	179.9	60.0	31.7	99.7

5.4 MODEL COMPLEXITY

The number of trainable parameters and the multiply-accumulate (MAC) operations per sample were counted from the layer definitions in `src/models/*.py` at the default configuration (Table 5.5). Batch-normalisation running statistics are buffers, not trainable parameters, and are excluded; MAC counts include convolutions, linear layers and the attention matrix products, and exclude normalisation, activation, pooling and the softmax. In total the final (v2/v3) model has 1,955,140 trainable parameters (three classes) and about 2.60 billion MACs per sample at 64^3 (0.39 billion at 32^3). For comparison, the original (v1) model had 1,055,081 trainable parameters and 2.53 billion MACs.

Table 5.5: Trainable parameters and multiply-accumulates (millions, at 64^3) per sample for the default (v2/v3) model.

Component	Parameters	MACs (millions)
Spatial encoder (3D ResNet + SE)	894,504	2,522.68
Temporal encoder (multi-scale 1D CNN)	785,936	65.89
Gated bidirectional fusion	265,218	11.01
Positional embeddings	8,192	0.00
Attention pool	129	0.00
Classification heads (×3)	1,161	0.00
Total	**1,955,140**	**2,599.60**

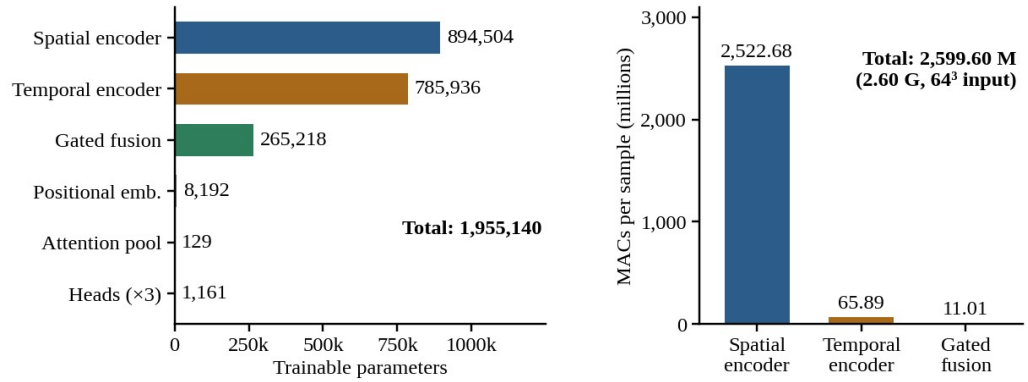


Figure 5.5: Left: trainable parameters by component of the final model (1,955,140 in total). Right: multiply-accumulate operations (MACs) per sample at 64^3 by component group (2.60 G in total; Table 5.5).

In the final model the spatial encoder accounts for 46% of the parameters and 97% of the MACs, the multi-scale temporal encoder for 40% of the parameters and 2.5% of the MACs, the gated bidirectional fusion for 14% of the parameters and under 0.5% of the MACs, and the positional embeddings, attention pool and three heads together for the remaining 0.5% of the parameters. The attention-score tensor of each direction has 4,096 elements per sample (4 heads × 64 × 16). Figure 5.5 shows this split. In the original v1 model the spatial encoder held 83.7% of the parameters, the fusion module 12.6%, the temporal encoder 3.6% and the three heads 0.11%.

5.5 INTERFACE AND SHAPE VERIFICATION

The tensor contract of Table 3.1 is asserted by the test suite: the preprocessing outputs (1, 64, 64, 64) and (1, 500) at the default configuration; the spatial and temporal encoders return (B, 64, 128) and (B, 16, 128); the fusion module returns fused spatial tokens (B, 64, 128), updated temporal tokens (B, 16, 128) and attention weights for both directions, (B, 4, 64, 16) forward and (B, 4, 16, 64) backward, whose

rows sum to one; the model returns three logits tensors of shape (B, 3); Grad-CAM returns (B, D, H, W) values in [0, 1]; and the attention heatmap has shape (64, 16). A scaled-down configuration (the `tiny_config` fixture) is used for speed in the tests and exercises the same code.

5.6 SOFTWARE VERIFICATION RESULTS

The repository's automated suite now contains 340 test functions in 29 files (310 unit and 30 integration tests). All 340 pass on macOS (Python 3.14) and on Ubuntu 22.04 under WSL2 (Python 3.12, CUDA), where one test is skipped; the three defects that the second platform exposed are described in Section 4.9. Figure 5.6 (left) shows the original suite of 86 tests in 12 files. The statement coverage reported here, 521 of 548 statements (95.1%), applies to the original code base only (twelve modules): it was reconstructed from the recorded coverage data file of the development machine by matching the executed line numbers against the executable statements of each source file, and it has not been re-measured for the current code. No coverage claim is made for the code added since. Table 5.6 gives the per-module values for the original code; they may differ from the values printed by the coverage tool by a statement or so, because the executable-statement set was derived from the syntax tree rather than from the tool's own parser.

Table 5.6: Statement coverage by module of the original code base (reconstructed from the recorded coverage data; not re-measured for the current code).

Module	Statements	Executed	Coverage (%)
src/config.py	38	38	100.0
src/data/dataset.py	43	43	100.0
src/data/synthetic.py	62	62	100.0
src/explainability/grad_cam_3d.py	70	69	98.6
src/explainability/plot_attention.py	22	22	100.0
src/models/cafn.py	31	31	100.0
src/models/cross_attention.py	45	45	100.0
src/models/spatial_encoder.py	37	37	100.0
src/models/temporal_encoder.py	14	14	100.0
src/preprocessing/mri_transforms.py	42	41	97.6
src/preprocessing/signal_cleaner.py	51	49	96.1
src/train.py	93	70	75.3
Total	**548**	**521**	**95.1**

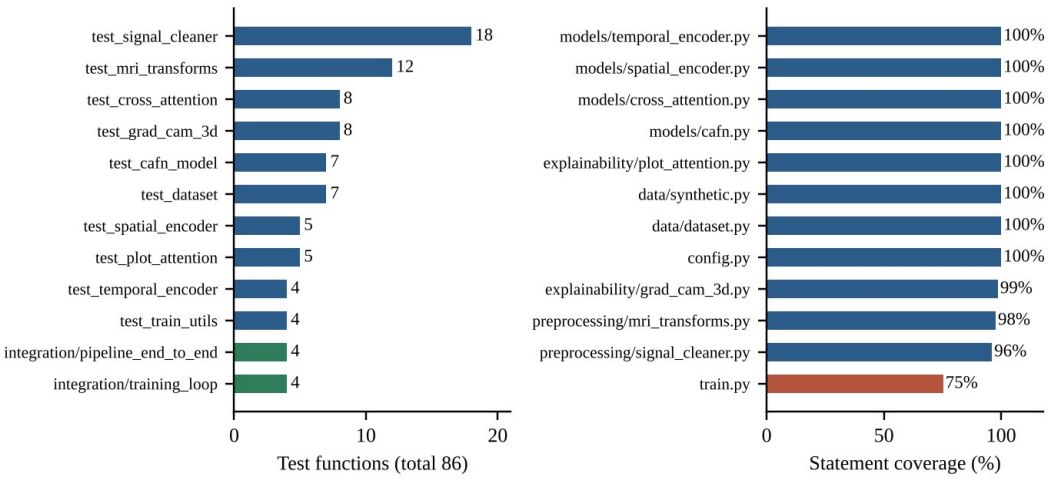


Figure 5.6: Left: number of test functions per test file in the original suite (86 tests; blue: unit; green: integration). Right: statement coverage per module of the original code base (red: below 90%).

Among the original modules the lowest-covered is `src/train.py` (75%). Its uncovered statements were mainly the body of the command-line `main()` function, which parses arguments, builds the synthetic dataset and writes a checkpoint, together with a few verbose-logging and edge-case branches. Command-line tests added since exercise this entry point, but the coverage has not been re-measured.

5.7 TRAINING ACCEPTANCE CRITERIA

The training loop is verified by the four integration tests in `tests/integration/test_training_loop.py`, which train a scaled-down model on synthetic data. The configuration and the thresholds that they assert are given in Table 5.7. The tests assert thresholds, not point values; the exact loss and accuracy reached are therefore not part of the recorded results.

Table 5.7: Training-loop acceptance criteria asserted by `tests/integration/test_training_loop.py`.

Item	Specification
Data	24 synthetic samples (raw volume 40 × 48 × 32), validation split 25%, seed 0
Model	d_model 32, 2 heads, 8 spatial and 8 temporal tokens, FFN 64, dropout 0.1; MRI shape 32^3; VEP length 200
Training	15 epochs, batch size 4, learning rate 1e-3, weight decay 1e-4, auxiliary weight 0.3; optimiser and schedule are the `TrainConfig` defaults, now AdamW with warm-up and cosine decay (the test was written when the default was Adam with a constant rate)
Criterion 1	Final training loss < 0.7 × initial training loss
Criterion 2	Validation accuracy of the fused head > 0.5 (chance level for three balanced classes is 0.33)
Criterion 3	Training runs without a validation set (history of validation loss is empty)

Item	Specification
Criterion 4	A saved and reloaded checkpoint reproduces the fused logits exactly (`assert_close`)

The four tests of `tests/integration/test_pipeline_end_to_end.py` exercise the raw-file path: synthetic NIfTI and CSV files are written and read through `PairedNeuroDataset`; the preprocessed tensors are passed through the model without shape errors; the loss is back-propagated and at least one parameter is shown to change after an optimiser step; Grad-CAM and attention plots are produced from a dataset sample; and the preprocessed Glaucoma and Optic Neuritis VEP peaks are shown to occur at different sample indices.

Measured training and evaluation results of the final model on synthetic and on real data are reported in Sections 5.9 to 5.12.

5.8 SUMMARY OF RESULTS

Table 5.8 summarises the results of Sections 5.2 to 5.7 together with the headline findings of Study B that are final; the detailed synthetic-benchmark and real-cohort results follow in Sections 5.9 to 5.12.

Table 5.8: Summary of the objective results.

Item	Result
H1: P100 latency preserved (synthetic, original generator)	149 of 150 synthetic trials within one sample (4 ms) of ground truth; one Glaucoma trial at 8 ms; mean errors between +0.00 and +0.80 ms
Mains attenuation (synthetic)	Mean 22.0 dB reduction of the 50 Hz component; notch −6 dB band 49.18–50.83 Hz
Foreground mask on synthetic MRI (original generator, percentile mask)	Exactly 60.0% of voxels zeroed in every class; atrophy-proxy ROI fully zeroed in the Glaucoma volume
Model size	1,955,140 trainable parameters (original v1: 1,055,081); 2.60 G MACs per sample at 64^3 (0.39 G at 32^3)
Test suite	340 test functions (310 unit, 30 integration) in 29 files; all pass on macOS (Python 3.14) and on Ubuntu 22.04 with CUDA (Python 3.12; one test skipped)
Statement coverage	95.1% for the original code base (twelve modules), not re-measured for the current code; no coverage claim for the new code
H2: training acceptance	Thresholds defined and asserted by integration tests (loss < 70% of initial; validation accuracy > 0.5); measured results in Sections 5.9 to 5.12
Study A: controlled synthetic benchmark and informativeness sweep (synthetic)	Fusion beats the better single modality by 10.7 points (78.1% against 68.0% MRI only and 64.0% VEP only; paired over 5 seeds, $p < 0.001$). CAFN (78.1%) is indistinguishable from concatenation (77.5%) and element-wise sum (79.6%); none of 7 component ablations differs reliably (−1.7 to 0.0 points). The fusion gain is largest at intermediate informativeness and vanishes when one modality dominates (Sections 5.9, 5.10)
Study B: classical baselines, 5 × 5-fold cross-validation on all 95 subjects (real cohorts)	ROC-AUC between 0.52 and 0.59 for all five baselines (mean over repeats). Only logistic regression on the spectrum (0.563, permutation $p = 0.025$) and on band powers (0.567, $p = 0.005$) reach $p < 0.05$, and only the band-power model survives Bonferroni correction (adjusted $p = 0.025$); the bootstrap intervals (0.45–0.68) include 0.5. Leave-one-cohort-out ROC-AUC 0.40–0.63: no consistent transfer (Sections 5.11, 5.12)
Study B: CAFN, pooled and leave-one-cohort-out, 3 repeats, 64^3 MRI (real cohorts)	Test accuracy 0.45–0.52 and balanced accuracy 0.46–0.54 in all settings: at chance level (Section 5.12). Cross-validated ROC-AUC: CAFN 0.51, concatenation 0.53, MRI only 0.48 (all at chance); spectrum-only network 0.62 (interval 0.50–0.72), the best (Section 5.11)
Study B: cohort-identity probe (real cohorts)	The spectrum alone identifies the cohort with 100% accuracy (majority baseline 50.5%)

5.9 CONTROLLED SYNTHETIC BENCHMARK AND ABLATION

This section reports Study A (synthetic data; Section 3.9.1). Table 5.9 and Figure 5.7 compare the twelve configurations at the preset difficulty ($\mu_{MRI} = 1.0$, $\mu_{VEP} = 0.3$; 120 test samples; chance accuracy 33.3%). The numbers are test performance of the fused head, mean ± standard deviation over seeds (4 to 7 seeds per

configuration, as the runs finished); the last column is the mean paired difference in accuracy to the full CAFN, in percentage points. They are results of a controlled synthetic benchmark and not clinical performance. Study A uses the current generator of Section 3.5.4 (VEP latency 100 ± 3, 104 ± 4 and 138 ± 6 ms and amplitude 6.0, 2.5 and 4.0 for the three classes, with random lesion position and size), not the original generator of Sections 5.2 and 5.3.

Table 5.9: Study A (synthetic, three classes, $\mu_{MRI} = 1.0$, $\mu_{VEP} = 0.3$): test performance of the fused head, mean ± SD over seeds.

Configuration	Seeds	Params (M)	Accuracy (%)	Macro-F1 (%)	Δ accuracy vs full CAFN (points)
CAFN (full)	7	1.96	78.1 ± 2.3	78.0 ± 2.4	—
w/o bidirectional	5	1.82	77.0 ± 5.5	76.7 ± 5.1	−1.7
w/o positional enc.	5	1.95	78.7 ± 3.9	78.5 ± 3.9	−0.0
w/o attention pooling	5	1.96	77.7 ± 2.1	77.2 ± 2.2	−1.0
w/o SE blocks	5	1.94	77.5 ± 5.9	76.8 ± 6.8	−1.2
w/o auxiliary loss	5	1.96	77.3 ± 5.7	77.2 ± 5.8	−1.3
2 fusion blocks	5	2.22	77.0 ± 1.6	76.5 ± 2.1	−1.7
v1-style (all off)	5	1.80	77.2 ± 7.0	76.7 ± 7.1	−1.5
Concatenation	6	1.72	77.5 ± 2.3	77.1 ± 2.4	−0.7
Element-wise sum	6	1.69	79.6 ± 3.4	79.4 ± 3.3	+1.4
MRI only	5	1.69	68.0 ± 4.7	66.0 ± 5.9	−10.7
VEP only	4	1.69	64.0 ± 4.3	63.5 ± 4.4	−14.6

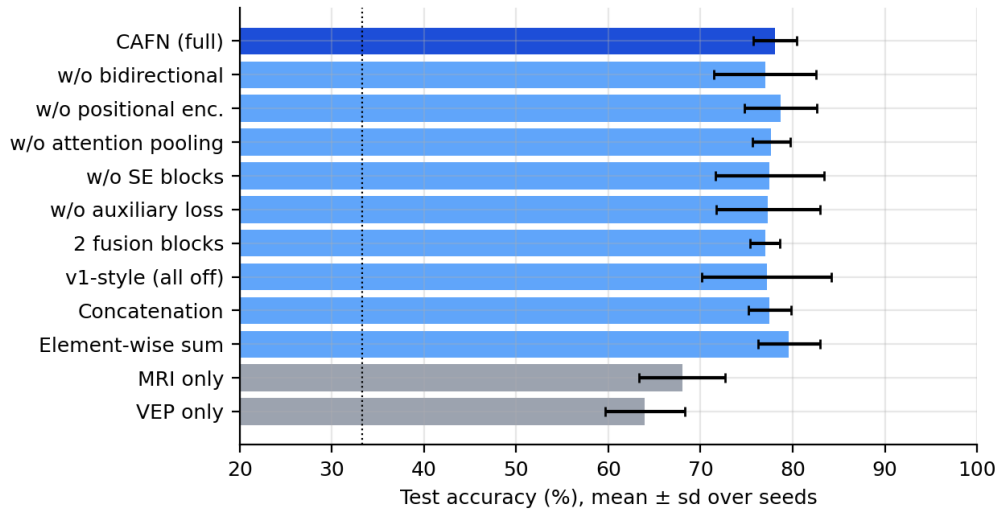


Figure 5.7: Study A (synthetic): test accuracy by configuration (bars: mean; whiskers: SD over seeds; dotted line: chance). Grey bars are the unimodal references.

Fusion helps, by a wide margin. The two unimodal references reach only 68.0% (MRI only, 5 seeds) and 64.0% (VEP only, 4 seeds), whereas every fusion

model reaches 77.5% to 79.6%. The full CAFN improves on the better unimodal model (MRI only) by 10.7 percentage points (paired over 5 seeds, SD 2.7, paired *t*-test $p < 0.001$). This confirms the premise of multimodal fusion in the setting in which the evidence in the two modalities is complementary.

The specific fusion mechanism does not matter at this scale. CAFN (78.1 ± 2.3%, 7 seeds) is statistically indistinguishable from concatenation (77.5 ± 2.3%; paired difference +0.7, $p = 0.63$) and from element-wise sum (79.6 ± 3.4%; −1.4, $p = 0.47$). Removing any single design choice (bidirectionality, positional embeddings, attention pooling, SE blocks, the auxiliary loss) or adding a second fusion block changes the mean accuracy by between −1.7 and 0.0 points, which is smaller than the seed-to-seed standard deviation (about 4.3 points); none of these seven differences is nominally significant ($p < 0.05$, uncorrected). The v1-style configuration with everything switched off scores 77.2% (−1.5 points) but with a much larger spread (SD 7.0 against 2.3 points), in line with the training instability observed earlier for the original recipe. No claim is therefore made that gated bidirectional cross-attention improves accuracy over simpler fusion on this benchmark.

5.10 INFORMATIVENESS SWEEP

Study A also varied the informativeness of the VEP (μ_{VEP}) with the MRI fixed at 1.0, for four methods with only 2 seeds per point (Table 5.10, Figure 5.8). With so few seeds the intervals are wide, and the results should be read as trends, not as precise estimates. All numbers are synthetic.

Table 5.10: Study A sweep (synthetic): test accuracy (%, mean ± SD over 2 seeds) versus VEP informativeness, MRI strength fixed at 1.0.

Method	VEP 0.15	VEP 0.3	VEP 0.5	VEP 1.0
MRI only	62.9 ± 4.1	65.0 ± 5.9	70.0 ± 4.7	62.9 ± 1.8
VEP only	45.0 ± 0.0	64.2 ± 1.2	85.4 ± 0.6	96.7 ± 2.4
Concatenation	69.6 ± 0.6	77.5 ± 7.1	90.8 ± 1.2	98.3 ± 0.0
CAFN (full)	67.5 ± 7.1	77.9 ± 11.2	90.0 ± 2.4	97.5 ± 2.4

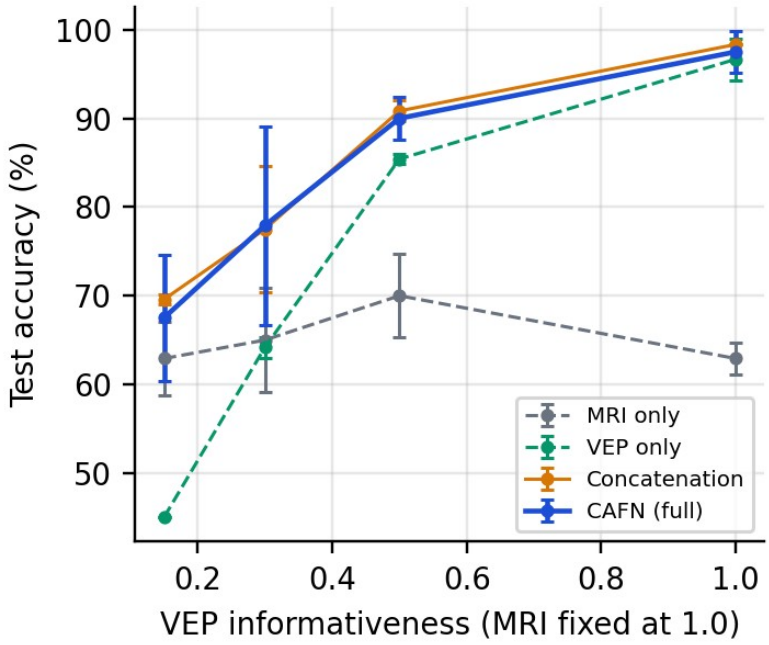


Figure 5.8: Study A sweep (synthetic): test accuracy versus VEP informativeness (MRI fixed at 1.0). Dashed lines: unimodal models; solid lines: fusion models.

The VEP-only model rises from 45% to 97% while the MRI-only model stays between 63% and 70%. The gain of the fused models over the better single modality is largest at intermediate informativeness and vanishes when one modality dominates: about +5 points (CAFN) and +7 points (concatenation) at $\mu_{\mathrm{VEP}} = 0.15$, +13 and +12 points at 0.3, +5 and +5 points at 0.5, and +1 and +2 points at 1.0. Fusion therefore did not hurt in this sweep, helps when both streams carry independent evidence, and converges to the dominant stream otherwise; CAFN and concatenation track each other throughout.

5.11 REAL COHORTS: CROSS-VALIDATION AND PERMUTATION TESTS

All 95 subjects (50 patients, 45 controls) are predicted out-of-fold (Study B, real cohorts; Section 3.9.2). Table 5.11 and Figure 5.9 give the out-of-fold ROC-AUC (mean over repeats: five repeats of 5-fold cross-validation for the classical models, one repeat for the neural models), the bootstrap 95% interval (repeat 0), the balanced accuracy and, for the classical models, the within-cohort permutation *p*-value (200 permutations).

Table 5.11: Real cohorts (95 subjects): out-of-fold ROC-AUC, bootstrap 95% interval, balanced accuracy and within-cohort permutation *p*. Neural models were cross-validated once and have no permutation test.

Model	ROC-AUC	95% CI	Balanced accuracy	Permutation *p*
Spectrum (500-point) · logistic regression	0.563	[0.45, 0.68]	0.590	0.025
Band powers (5) · logistic regression	0.567	[0.45, 0.68]	0.590	0.005
Band powers (5) · random forest	0.518	[0.36, 0.60]	0.539	0.726
MRI (8^3 → PCA-20) · logistic regression	0.587	[0.43, 0.67]	0.584	0.239
MRI + band powers · logistic regression	0.588	[0.43, 0.67]	0.580	0.234
CAFN (ours)	0.510	[0.40, 0.62]	0.518	—
Concatenation fusion	0.533	[0.42, 0.66]	0.553	—
MRI only (network)	0.479	[0.36, 0.60]	0.503	—
Spectrum only (network)	0.618	[0.50, 0.72]	0.561	—

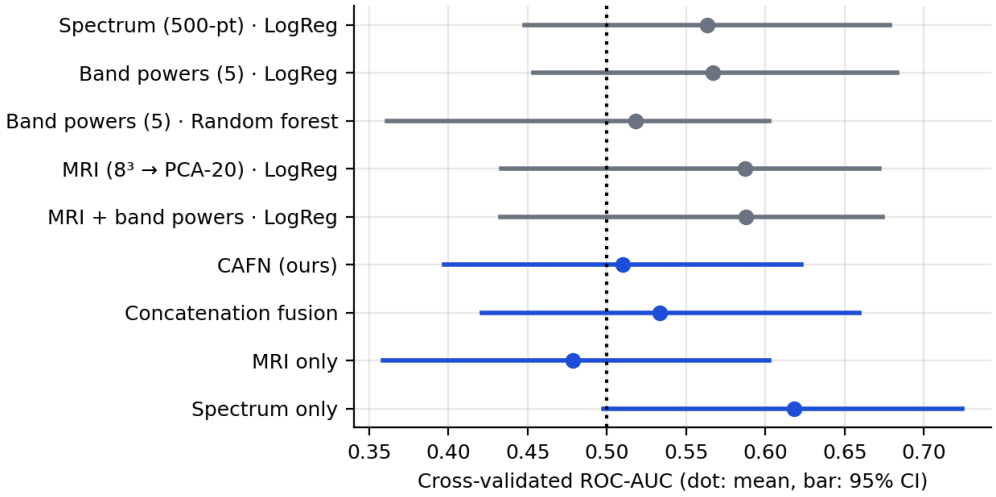


Figure 5.9: Real cohorts: cross-validated ROC-AUC (dot: mean; bar: 95% bootstrap interval). Grey: classical models; blue: neural models. The dotted line is chance.

The signal is weak and fragile. Linear models on spectral features reach a ROC-AUC of 0.56 to 0.57 (balanced accuracy 0.59). The within-cohort permutation test gives $p = 0.005$ for the five band powers and $p = 0.025$ for the full spectrum. This is nominally significant, but with five classical models tested only the band-power model survives a Bonferroni correction (adjusted $p = 0.025$), and its bootstrap interval ([0.45, 0.68]) still includes 0.5, so the evidence is at best suggestive. A random forest on the same features is at chance ($p = 0.73$), and MRI-derived features (PCA of an 8^3 down-sampled volume) are not distinguishable from chance either ($p = 0.24$). The neural models tell the same story: the best is the spectrum-only network (ROC-AUC

0.62, interval [0.50, 0.72]), while the full CAFN (0.51), the concatenation network (0.53) and the MRI-only network (0.48) are at chance. Adding the MRI stream therefore did not help on these cohorts and, if anything, hurt, which is consistent with an MRI branch that overfits the 76 training subjects of each fold.

5.12 POOLED AND LEAVE-ONE-COHORT-OUT CAFN

Table 5.12 shows the full CAFN (64^3 MRI) trained on the pooled cohorts and evaluated on each cohort's held-out test subjects (7 per cohort), and trained on one cohort and tested on every subject of the other (leave-one-cohort-out; Study B, real cohorts). Values are mean ± SD over 3 repeats, test subjects only.

Table 5.12: CAFN on the real cohorts (mean ± SD over 3 repeats; test subjects only).

Setting	Accuracy	Balanced accuracy	ROC-AUC
Pooled model, test subjects of both cohorts	0.52 ± 0.04	0.52 ± 0.04	0.61 ± 0.09
Pooled model, test subjects of ds003568	0.52 ± 0.08	0.46 ± 0.07	0.61 ± 0.17
Pooled model, test subjects of ds007181	0.52 ± 0.08	0.54 ± 0.07	0.53 ± 0.17
Leave-one-cohort-out, tested on ds003568	0.48 ± 0.09	0.46 ± 0.04	0.57 ± 0.08
Leave-one-cohort-out, tested on ds007181	0.45 ± 0.02	0.49 ± 0.05	0.44 ± 0.03

All accuracies lie between 0.45 and 0.52 and all balanced accuracies between 0.46 and 0.54: the model is at chance, with variation across repeats larger than any difference between settings. Figure 5.10 shows why: the training loss falls to nearly zero within about 30 epochs while the validation accuracy stays near 0.5, so the model memorises the roughly 67 training subjects.

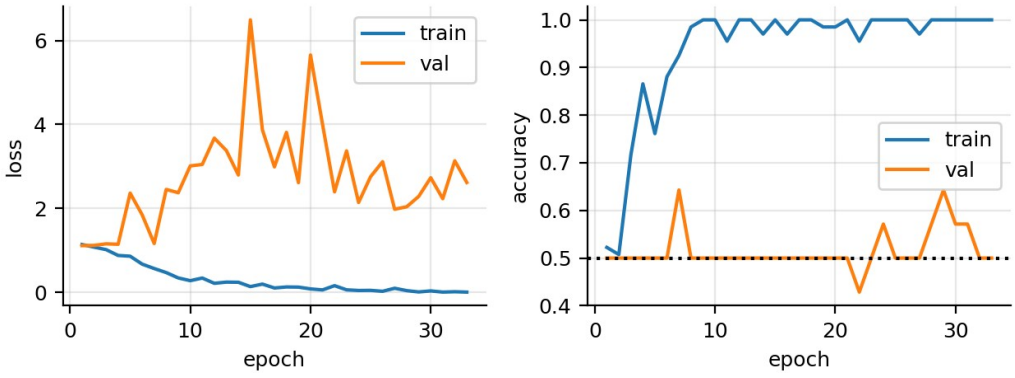


Figure 5.10: Pooled CAFN on the real cohorts, repeat 0: training versus validation loss and accuracy. The validation curve stays near chance while the training loss vanishes.

The classical models do not transfer either (Table 5.13): across cohorts the ROC-AUC ranges from 0.40 to 0.63 and the balanced accuracy is 0.43 to 0.51. The

spectrum model trained on one cohort reaches a ROC-AUC of 0.63 on ds003568 but a balanced accuracy of 0.50, which shows that the ranking transfers weakly while the decision threshold does not (different sensors and recording protocols shift the spectrum).

Table 5.13: Leave-one-cohort-out for the classical models (real cohorts): ROC-AUC / balanced accuracy on the named test cohort after training on the other.

Model	Test ds003568	Test ds007181
Spectrum (500-point) · logistic regression	0.63 / 0.50	0.57 / 0.50
Band powers (5) · logistic regression	0.50 / 0.50	0.48 / 0.50
Band powers (5) · random forest	0.50 / 0.50	0.50 / 0.50
MRI (8^3 → PCA-20) · logistic regression	0.40 / 0.43	0.44 / 0.47
MRI + band powers · logistic regression	0.41 / 0.43	0.46 / 0.51

Cohort-identity probe. A linear model on the spectrum alone identifies the cohort of a subject with 100% accuracy (majority baseline 50.5%), so cohort identity is an available shortcut. The within-cohort permutation test of Section 5.11 is designed not to reward it.

CHAPTER 6
DISCUSSION

6.1 INTRODUCTION

This chapter interprets the measurements of Chapter 5, relates them to the objectives, research questions and hypotheses of Section 1.4, compares the design with the literature of Chapter 2, and states the constraints under which the conclusions hold. The guiding principle is that every statement is limited to what the evidence supports: the project is a tested research prototype; the evidence on synthetic data concerns software behaviour and a controlled comparison of architectures, and the evidence on real data concerns a pipeline tested on cohorts that are not eye-disease data. None of it is evidence of diagnostic performance.

6.2 INTERPRETATION OF THE RESULTS

6.2.1 SIGNAL CONDITIONING

The frequency response confirms the intended behaviour of the conditioning chain (Table 5.1): the pass-band is flat to better than 0.001 dB at 10 Hz, the 50 Hz component is removed by about 97 dB in the ideal response, and the −6 dB band of the notch is only 1.65 Hz wide, so that neighbouring frequencies are barely affected. The effective response at 1 Hz is about −5.8 dB, which is the −3 dB corner of the single-pass filter applied twice; the 1 Hz edge therefore removes baseline drift without cutting into the region around 10 Hz that carries the main VEP components.

The measured reduction of the injected 50 Hz component was 22 dB on average, much smaller than the 97 dB of the ideal response. This is not a failure of the filter. After filtering, the fitted 50 Hz amplitude (0.0125) is of the same order as the fitted amplitude at 47 Hz (0.0212), where nothing was injected: the residual is dominated by the broadband noise of the synthetic record, and the measured figure is therefore a lower bound on the notch's attenuation.

For the P100 latency, hypothesis H1 is supported at the level of the class means (errors of +0.64, +0.00 and +0.80 ms against a 4 ms sampling interval) and in 149 of 150 individual trials. The one exception, a Glaucoma record with an 8 ms error, is not specific to the zero-phase pipeline: the raw and single-pass results for the same class also reach a maximum error of 8 ms, so it is most likely due to the noise realisation

and not to the filtering. The ordering of the three classes by detected latency (about 100, 108 and 141 ms) is preserved after conditioning, which is the property that matters for the downstream task.

A noteworthy observation is that the benefit of zero-phase over single-pass filtering cannot be demonstrated by these data. The mean difference in detected latency between the two implementations is below 0.5 ms in every class, i.e. much smaller than one sample, even though the single-pass filter has a group delay of 3 to 6 ms in the 10–30 Hz range (Table 5.3). The likely reason is that the synthetic P100 is a smooth, low-frequency peak whose maximum is insensitive to a small delay that is nearly constant across its bandwidth. The justification for zero-phase processing in this project therefore rests on the principle that latency is the clinical quantity (Odom et al., 2016) and on the absence of any cost to using it on offline records, not on an empirical gain at the project's sampling rate. At a higher sampling rate, or for sharper real waveforms, the difference could become measurable.

6.2.2 SYNTHETIC DATA AND THE FOREGROUND MASK

The original class-conditioned generators behave as specified: the mean intensity of the atrophy region in the Glaucoma volume is 40 against about 100 elsewhere, the lesion region in the Optic Neuritis / CVI volume is about 180, and the Healthy Control volume is homogeneous (Table 5.4). The synthetic VEP classes differ in latency and amplitude as intended (Table 5.2 and Figure 5.4).

The mask results expose a limitation of the placeholder preprocessing. By construction the 60th-percentile threshold zeroes 60% of the voxels in every volume, whatever their content. In the Glaucoma volume this removes the entire atrophy region, so that the structural signature survives only as a contiguous block of zeros and no longer as a reduced intensity, while the lesion region of the Optic Neuritis / CVI volume is retained almost completely (99.7%). The mask is therefore not neutral with respect to the classes. This does not affect the correctness of the software, which was designed with the mask isolated for replacement (Section 3.7), but it shows that conclusions about what the spatial encoder can learn from the synthetic MRI depend on the preprocessing and cannot be carried over to real, skull-stripped scans. The Otsu mask with hole filling and bounding-box resizing used for the real cohorts is a better stand-in for whole-head scans, but it is still not validated skull-stripping.

6.2.3 MODEL SIZE AND COST

With 1,955,140 parameters the final model is still small by the standards of current deep-learning systems. About 46% of the parameters sit in the 3D residual encoder, and the multi-scale temporal encoder holds 40%, because its parallel convolutions are widest in the last block (557,312 of the 785,936 parameters). The compute is concentrated differently: 97% of the 2.60 billion MACs per sample are in the 3D spatial encoder, because the early 3D convolutions operate on high-resolution feature maps (in the original model the first residual block alone accounted for 54% of the MACs and the first two for 82%). The gated bidirectional fusion module, which is the conceptual core of the project, accounts for 14% of the parameters and under 0.5% of the compute. A practical consequence is that the cost of the attention mechanism is negligible compared with the cost of reading the volume, and that any reduction in training time should target the early 3D stages, for example by lowering the input resolution (at 32^3 the cost falls from 2.60 to 0.39 billion MACs) or the width of the first blocks.

6.2.4 VERIFICATION

The test suite covers every module, and all 340 tests pass on two platforms (macOS with Python 3.14; Ubuntu 22.04 with Python 3.12 and CUDA, one test skipped). Running on the second platform was itself informative: it exposed three latent defects that the first platform had masked (a type annotation that fails on Python 3.13 and earlier, a device mismatch in the evaluation-only path on CUDA, and a unit test whose assertion was mathematically vacuous). A test count is not a measure of correctness, so it is best read together with the content of the tests, which assert shapes, value ranges, normalisation of attention rows, determinism, error handling, equality of the deterministic pooling with PyTorch's, and end-to-end gradient flow (Section 4.9). The line coverage of 95% was measured for the original code base only and has not been re-measured; no coverage claim is made for the code added since. The training acceptance criteria are meaningful in that they test learning (loss reduction and above-chance accuracy) and not only execution, but as thresholds on a toy configuration they say nothing about performance on real data.

6.3 ASSESSMENT AGAINST THE OBJECTIVES, QUESTIONS AND HYPOTHESES

Table 6.1: Assessment of research questions and hypotheses against the evidence.

Item	Evidence	Assessment
RQ1: common token representation through one pipeline	Shape contract asserted by tests; end-to-end tests from raw NIfTI and CSV files; the real cohorts are processed through the same manifest interface (Sections 3.4.2, 5.5, 5.6)	Supported (software level)
RQ2 / H1: P100 latency preserved	Class-mean errors below one sample; 149 of 150 trials within 4 ms (Table 5.2)	Supported on synthetic data
RQ3 / H2: end-to-end optimisation and cost	Loss-reduction and accuracy thresholds asserted by tests (Table 5.7); 1.96 M parameters and 2.60 G MACs (Table 5.5); synthetic benchmark: the fused head reaches 78.1 ± 2.3% against 33.3% chance (Table 5.9); real cohorts: the network trains (training loss falls to near zero) but validation stays at chance (Section 5.12)	Supported: the model optimises end to end and exceeds chance on synthetic data; on real cohorts it memorises the training subjects
RQ4: explanations from the same forward pass	Tests produce Grad-CAM and Grad-CAM++ maps and attention heatmaps for both directions from a dataset sample (Sections 4.7, 5.6)	Supported (artefacts produced); faithfulness not assessed
H3: single shape contract	Shape tests across all modules; one configuration module (Section 4.8)	Supported
RQ5 / H4: when cross-attention fusion helps	Fusion baselines and ablations (Sections 3.9.1, 4.4). Fusion beats the better single modality by 10.7 points ($p < 0.001$); gain largest at intermediate informativeness, vanishing when one modality dominates; CAFN versus concatenation +0.7 points ($p = 0.63$) and versus element-wise sum −1.4 points ($p = 0.47$); no single ablation reliable (Sections 5.9, 5.10)	H4 partly supported: fusion helps over either modality alone only in the complementary regime, as hypothesised; the advantage of cross-attention over concatenation or sum is not supported
RQ6 / H5: generalisation across real cohorts	Classical 5 × 5-fold cross-validation: ROC-AUC 0.52–0.59, only spectrum and band-power logistic regression reach permutation $p < 0.05$ (only band powers after Bonferroni correction), with bootstrap intervals that include 0.5; leave-one-cohort-out ROC-AUC 0.40–0.63; neural cross-validation: CAFN 0.51, concatenation 0.53, MRI only 0.48, spectrum only 0.62 (0.50–0.72); CAFN pooled and leave-one-cohort-out test accuracy 0.45–0.52 (balanced accuracy 0.46–0.54); cohort identity recoverable from the spectrum with 100% accuracy (Sections 5.11, 5.12)	H5 supported: signal weak at most, no consistent transfer; CAFN and the MRI stream at chance
Diagnostic accuracy for glaucoma or optic neuritis	No labelled eye-disease data; the real cohorts are other disorders	Not assessed

Verdicts on the hypotheses. H1 to H3 are supported at the level described in Table 6.1. H4 is only partly supported. Fusion beat the better single modality by 10.7

percentage points (paired over 5 seeds, $p < 0.001$) when both modalities carried complementary but individually imperfect evidence, and the gain vanished when one modality dominated, as hypothesised. The part of H4 that concerns cross-attention is not supported: CAFN was within seed noise of concatenation (+0.7 points, $p = 0.63$) and element-wise sum (−1.4 points, $p = 0.47$), and none of seven component ablations was reliably different. H5 is supported: on the real cohorts the signal was weak at most and did not transfer between cohorts.

6.4 DESIGN OBSERVATIONS

6.4.1 COARSE TEMPORAL TOKENS

With 16 temporal tokens over a 2.0 s record, each token nominally covers 125 ms. The synthetic P100 peaks of the Healthy Control (100 ms) and Glaucoma (108 ms) classes both fall in the first token, while the delayed peak of Optic Neuritis / CVI (140 ms) falls in the second (Figure 4.3). The latency difference between the first two classes (8 ms) is therefore far below the nominal token resolution. This is not necessarily a defect, because the tokens are learned, overlapping features with a large receptive field and not literal time bins, but it means that the temporal tokens should not be read as calibrated time axes, and the attention heatmap cannot be read as pointing at a millisecond (Section 4.7).

6.4.2 SYNTHETIC SEPARABILITY

In the original generator the synthetic classes differ in effects that are large relative to the noise (a 60% intensity reduction, an 80% intensity increase, shifts of up to 40 ms and a 2.4-fold amplitude change). A model that separates them has shown that the pipeline can learn from signals of this type, not that it can resolve the subtle, overlapping and heterogeneous changes present in patients. The same applies to the test thresholds. It is also why the earlier training demonstrations, which used the default generator, could not show that fusion helps: in those runs the VEP alone was almost perfectly separable (the VEP-only head reached 99.0%, the MRI-only head 70.8%), so a fused model could reach a high accuracy simply by following the VEP. That observation motivated the controllable generator and the benchmark of Study A, in which the informativeness of each modality is set so that neither is sufficient alone.

6.4.3 MULTI-TASK SUPERVISION AND ATTENTION

The auxiliary unimodal heads give each encoder its own gradient signal and so guard against the fused head ignoring one modality, a risk that is known for multimodal networks; the ablation without the auxiliary loss in Study A tests whether they help in practice (Section 5.9). Likewise, the cross-attention weights are returned for inspection, but attention weights do not automatically constitute an explanation (Jain & Wallace, 2019; Wiegreffe & Pinter, 2019), and gradient-based maps can look plausible even when insensitive to the model's parameters (Adebayo et al., 2018). The explainability outputs of this project are accordingly presented as inspection aids whose faithfulness has not been tested.

6.4.4 PITFALLS DISCOVERED DURING THE PROJECT

- **Checkpoint selection.** With validation loss as the selection criterion, short runs could restore an untrained first-epoch model whose loss happened to be lowest; the smoke and demonstration runs ended at 33% accuracy for that reason. Selecting the checkpoint on validation balanced accuracy (`select_metric`) fixes this.
- **Centre crop versus whole heads.** A centre crop to 64^3 is adequate for the small synthetic volumes but sees only a sliver of a real head about 170 mm across at 1 mm spacing; bounding-box resizing is needed for real scans.
- **Preprocessing can erase the signal.** In a single-seed diagnostic on the synthetic generator, switching the MRI pipeline from a percentile mask with min–max scaling to an Otsu mask with foreground z-scoring dropped the MRI-only accuracy from 73% to 41%, so Study A uses the original percentile-mask pipeline. The mask and normalisation therefore have to be checked for each data type.
- **Cohort identity.** The log spectrum alone identifies the cohort of a subject with 100% accuracy (majority baseline 50.5%), so a model could use cohort identity as a shortcut whenever the label balance differs between cohorts. Permutation tests that shuffle labels within cohorts and leave-one-cohort-out training are the safeguards used here.
- **Instability of the original design.** In an earlier comparison on 192 synthetic subjects and 30 epochs, the original (v1) and the v2 models both reached 98–100% accuracy once the best checkpoint was restored, but the v1 model was

unstable (its validation accuracy ended at 48%), whereas the v2 model stayed at 94–100% (synthetic data).

- **Platform-specific defects.** Three latent defects were visible only on the second platform (Section 4.9).

6.5 COMPARISON WITH THE LITERATURE

The design follows the established practice of intermediate fusion of learned features (Baltrušaitis et al., 2019; Ramachandram & Taylor, 2017) and uses cross-attention in the sense of the multimodal transformer and co-attentional models (Tsai et al., 2019; Lu et al., 2019); in this project each stream queries the other, with learned gates. The closest published studies are CAMF, which fuses structural and functional MRI for schizophrenia and reports that cross-attention beats concatenation and element-wise sum (Zhou et al., 2024), and DRIFA-Net, which applies dual attention to five medical-imaging datasets (Dhar et al., 2025); Table 2.4 summarises them. This project differs from them in the modalities (a volume and a one-dimensional signal), in scale (about 2 million parameters, compared with the large pre-trained models of the multimodal-transformer literature) and, above all, in evidence: those studies report one headline result per task on benchmark or clinical datasets, whereas this project reports a controlled synthetic benchmark and a candid test on two real cohorts that are not eye-disease data. The comparison with concatenation and element-wise sum follows the design of CAMF, which reports that cross-attention beats both by roughly 2 to 4 accuracy points on real schizophrenia data. In the controlled benchmark of this project the paired differences between CAFN and concatenation and element-wise sum are +0.7 and −1.4 points (p = 0.63 and 0.47), of the order of the seed-to-seed standard deviation (about 4.3 points), so the finding of CAMF is neither confirmed nor refuted here; at this model and sample size the extra machinery of cross-attention did not buy accuracy. What cross-attention does provide, and concatenation does not, is attention maps in both directions that can be inspected; their faithfulness is not established. No numerical comparison with published classification results is made, since it would not be like-for-like.

The preprocessing choices are in line with the recommendations of the VEP standard on the importance of latency (Odom et al., 2016) and with the signal-processing literature on zero-phase filtering (Gustafsson, 1996; Oppenheim &

Schafer, 2010). The use of an intensity-percentile mask or of an Otsu threshold with morphological clean-up, by contrast, is below the standard of the neuroimaging literature, where dedicated tools such as HD-BET and SynthStrip are used (Isensee et al., 2019; Hoopes et al., 2022); the project acknowledges this explicitly.

6.6 CONSTRAINTS AND THREATS TO VALIDITY

Table 6.2: Constraints of the study and their consequence for the conclusions.

Constraint	Consequence
No eye-disease cohort; the three-class experiments are synthetic	No statement about diagnostic accuracy, sensitivity, specificity or generalisation to patients with glaucoma or optic neuritis is possible.
The real cohorts are other disorders, with a pan-disorder label and a spectrum in place of the VEP	The real-cohort results test the pipeline, not the clinical hypothesis; they say nothing about MRI–VEP fusion in visual-pathway disease.
Small real sample (95 subjects; per-cohort test splits of about seven)	Single-split metrics are very noisy and bootstrap intervals are wide (for example 0.45–0.68); a weak effect cannot be separated from chance with confidence.
Cohort identity is recoverable from the spectrum (100% accuracy)	A model could use cohort identity as a shortcut; within-cohort permutation tests and leave-one-cohort-out training limit, but do not remove, this risk.
The synthetic modalities are independent given the label	The benchmark shows when fusion can help in that setting, not how often real modalities behave that way; one generator and a limited number of seeds were used.
Analytic complexity counts	Counts of parameters and MACs are derived from the layer definitions, not profiled; memory use and run time were not systematically measured.
Coverage not re-measured for the current code	The 95.1% figure applies to the original code base; no coverage claim is made for the code added since, and test counts and pass status are reported instead.
Placeholder foreground masks	Conclusions about the spatial encoder depend on the mask and cannot be transferred to skull-stripped real scans.

CHAPTER 7
CONCLUSION, RECOMMENDATIONS AND FUTURE WORK

7.1 CONCLUSION

This project designed, implemented and tested a research prototype of a cross-attention fusion network (CAFN) that combines a 3D structural MRI volume with a 1D electrophysiological signal, classifies the pair, and produces 3D Grad-CAM maps and spatial–temporal attention heatmaps for each prediction. The final design fuses 64 spatial and 16 temporal tokens by gated bidirectional cross-attention, with learned attention pooling, positional embeddings, squeeze-and-excitation and multi-scale temporal convolutions (1,955,140 parameters), and is supervised by a fused head and two auxiliary unimodal heads. The pipeline can be exercised from raw NIfTI and CSV files, and from two public OpenNeuro cohorts, through to an optimiser step and the explainability outputs; it is covered by 340 automated tests that pass on two platforms and by a one-command, seeded and resumable reproduction script. Beyond the prototype, the project built what is needed to evaluate the fusion design honestly: fusion baselines (concatenation, element-wise sum, unimodal), a synthetic generator with controllable modality informativeness, and a real-cohort protocol with leave-one-cohort-out training, repeated cross-validation, bootstrap intervals and permutation tests.

What the project does not establish is equally important. No public dataset provides paired MRI, VEP and the required diagnosis labels, so the three-class experiments use a synthetic benchmark whose results are a controlled comparison of architectures and not clinical performance. On the two real public cohorts, which are not eye-disease data, the proposed model is at chance level (test accuracy 0.45 to 0.52 in pooled and leave-one-cohort-out runs), a weak spectral signal is detectable (cross-validated ROC-AUC about 0.56 to 0.62, the best being a spectrum-only network), nothing transfers consistently between cohorts, and the spectrum alone identifies the cohort perfectly. The project therefore makes no claim about diagnostic accuracy, detection of glaucoma or optic neuritis, generalisation across cohorts or state-of-the-art performance. Its value is as a transparent, tested and candidly evaluated foundation on which an evaluation with real, labelled data can be built.

On when fusion helps, the controlled synthetic benchmark gives a clear but limited answer. Fusion improved test accuracy by about 10.7 percentage points over the better single modality (paired over 5 seeds, $p < 0.001$; chance 33.3%), and the gain was largest at intermediate informativeness and vanished when one modality dominated. However, gated bidirectional cross-attention (78.1%) was statistically indistinguishable from concatenation (77.5%) and element-wise sum (79.6%), and none of seven component ablations produced a reliable change. The project therefore does not show that cross-attention is more accurate than simpler fusion; what it adds is inspectable attention maps in both directions.

7.2 CONCLUSIONS AGAINST THE OBJECTIVES

Table 7.1: Status of each objective.

Objective	Outcome	Evidence
O1 Data interface	Achieved	Manifest-driven dataset yielding (1, 64, 64, 64) (or 32^3) and (1, 500) tensors from synthetic files and from the two real cohorts; end-to-end tests from raw files (Sections 3.6, 5.5)
O2 Signal conditioning	Achieved for the VEP and the spectrum; MRI masking is a placeholder	Zero-phase pipeline keeps P100 latency within one sample in 149 of 150 trials; notch −6 dB band 1.65 Hz; percentile and Otsu masks are not skull-stripping (Sections 5.2, 5.3)
O3 Dual encoders	Achieved	3D residual encoder with SE (894,504 parameters) and multi-scale 1D CNN (785,936) produce (B, 64, 128) and (B, 16, 128) tokens (Sections 4.2, 4.3, Table 5.5)
O4 Cross-attention fusion	Achieved at the implementation level; no accuracy advantage over simpler fusion was found	Gated bidirectional cross-attention, attention pooling, fusion baselines and the multi-task loss implemented and tested; synthetic benchmark: CAFN 78.1% against concatenation 77.5% and element-wise sum 79.6% (indistinguishable); real cohorts at chance (Sections 5.9, 5.12)
O5 Explainability	Achieved at the implementation level	3D Grad-CAM and Grad-CAM++, attention heatmaps for both directions and per head, P100 estimator; faithfulness not evaluated (Section 4.7)
O6 Verification	Achieved	340 tests passing on macOS and on Ubuntu with CUDA; filter, data and complexity measurements; 95.1% coverage for the original code only (Chapter 5)
O7 Honest scoping	Achieved	Limitations stated in Sections 1.6 and 6.6 and in Chapter 5; synthetic and real results labelled; negative real-cohort findings reported
O8 Controlled benchmark	Achieved	12 configurations on 4 to 7 seeds and an informativeness sweep: fusion beats the better single modality by 10.7 points; the fusion mechanism and the individual components make no reliable difference (Sections 5.9, 5.10)
O9 Real-data protocol	Achieved; real-cohort results at chance	One-command pipeline and protocol run on 95 subjects; neural fusion at chance (CAFN ROC-AUC 0.51); weak spectral signal (ROC-AUC 0.56–0.62); no consistent transfer between cohorts (Sections 5.11, 5.12)

7.3 RECOMMENDATIONS

The recommendations below are ordered by priority. Priority 1 items are prerequisites for any claim about the usefulness of the system, Priority 2 items raise the scientific quality of such a claim, and Priority 3 items improve robustness and usability. Effort estimates are indicative for a small research team.

Table 7.2: Prioritised recommendations.

Priority	Recommendation	Basis in the findings	Indicative effort
1	Obtain a labelled, paired MRI–VEP cohort (or a clinical partner to label a subset), with ethics approval, diagnostic criteria and patient-level identifiers	Public data lack the diagnosis labels (Section 2.8); the real cohorts used are other disorders	Months; dependent on partners
1	Replace the pan-disorder label by a disease-specific one and enlarge the real cohort; keep leave-one-cohort-out training and within-cohort permutation tests	Real-cohort results are at chance and 95 subjects give wide intervals (Sections 5.11, 5.12)	Months; dependent on data
1	Guard against cohort-identity shortcuts (cohort-stratified splits, within-cohort permutation tests as here; consider harmonisation)	The spectrum identifies the cohort with 100% accuracy (Section 6.4.4)	1–2 weeks
2	Replace the percentile and Otsu masks with validated skull-stripping (for example HD-BET or SynthStrip) as an optional step, and add registration or quality-control checks	The masks are placeholders; the percentile mask zeroed the atrophy region of the original generator (Section 6.2.2)	1–2 weeks
2	On any clinical cohort, report sensitivity, specificity, calibration and confidence intervals alongside ROC-AUC and balanced accuracy	Only ROC-AUC and balanced accuracy are reported here (Section 3.9.3)	1–2 weeks
2	Test the explanation outputs: parameter-randomisation sanity checks for Grad-CAM, and comparison of the attention and saliency maps with clinician-marked regions	Attention and saliency are not automatically faithful (Section 6.4.3)	2–4 weeks
3	Add handling and evaluation of a missing modality beyond training-time modality dropout, and re-measure statement coverage for the current code	Coverage was measured for the original code only (Section 5.6)	1 week
3	Reduce the cost of the early 3D stages (lower input resolution, mixed precision, gradient checkpointing) if memory or time limits training	97% of the MACs are in the spatial encoder (Table 5.5)	1 week
3	Prepare a model card and data statement, and consider regulatory requirements early if clinical use is intended	The system is a research prototype	Ongoing

7.4 FUTURE WORK

- **Real-data training and external validation.** Train on a labelled eye-disease cohort, validate on an independent site, and report performance with uncertainty.

- **Anatomically grounded tokens.** Register spatial features to named structures of the visual pathway (optic nerve, lateral geniculate nucleus, visual cortex) so that attention can be reported against anatomy, and align temporal tokens with the VEP landmarks N75, P100 and N135.
- **Richer electrophysiology.** Extend the temporal stream to multi-channel VEP, pattern electroretinography or full-field stimuli, which could allow localisation along the pathway.
- **Uncertainty and abstention.** Add calibrated probabilities and the option to defer ambiguous cases to a clinician.
- **Longitudinal modelling.** Use repeated MRI and VEP over time to model progression in glaucoma and recovery in optic neuritis.
- **Alternative fusion mechanisms.** Compare cross-attention with bilinear pooling and other gating schemes under the same protocol, now that concatenation, element-wise sum and unimodal baselines are part of the benchmark.

7.5 CLOSING REMARKS

The prototype shows that a clean interface between a volumetric and a one-dimensional modality, a fusion module that makes the interaction explicit, and a testing discipline that reaches from file readers to explanation plots can be assembled in a compact, readable code base, and that it can be evaluated honestly: the controlled benchmark asks when fusion helps, and the real-cohort protocol shows that, on the cohorts available, the proposed model finds no signal beyond chance. The next stage of work is not another architecture but data: with a labelled eye-disease cohort, the protocol, baselines and sanity checks built here would turn this prototype into an experiment whose results can be believed.

REFERENCES

Adebayo, J., Gilmer, J., Muelly, M., Goodfellow, I., Hardt, M., & Kim, B. (2018). Sanity checks for saliency maps. *Advances in Neural Information Processing Systems, 31*, 9505–9515.

Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). *Layer normalization* (arXiv:1607.06450). arXiv. https://arxiv.org/abs/1607.06450

Baltrušaitis, T., Ahuja, C., & Morency, L.-P. (2019). Multimodal machine learning: A survey and taxonomy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *41*(2), 423–443. https://doi.org/10.1109/TPAMI.2018.2798607

Benson, N. C., Jamison, K. W., Arcaro, M. J., Vu, A. T., Glasser, M. F., Coalson, T. S., Van Essen, D. C., Yacoub, E., Ugurbil, K., Winawer, J., & Kay, K. (2018). The Human Connectome Project 7 Tesla retinotopy dataset: Description and population receptive field analysis. *Journal of Vision, 18*(13), Article 23. https://doi.org/10.1167/18.13.23

Butterworth, S. (1930). On the theory of filter amplifiers. *Experimental Wireless and the Wireless Engineer, 7*, 536–541.

Calhoun, V. D., & Sui, J. (2016). Multimodal fusion of brain imaging data: A key to finding the missing link(s) in complex mental illness. *Biological Psychiatry: Cognitive Neuroscience and Neuroimaging*, *1*(3), 230–244. https://doi.org/10.1016/j.bpsc.2015.12.005

Cardoso, M. J., Li, W., Brown, R., Ma, N., Kerfoot, E., Wang, Y., Murrey, B., Myronenko, A., Zhao, C., Yang, D., Nath, V., He, Y., Xu, Z., Hatamizadeh, A., Zhu, W., Liu, Y., Zheng, M., Tang, Y., Yang, I., . . . Feng, A. (2022). *MONAI: An open-source framework for deep learning in healthcare* (arXiv:2211.02701). arXiv. https://arxiv.org/abs/2211.02701

Çiçek, Ö., Abdulkadir, A., Lienkamp, S. S., Brox, T., & Ronneberger, O. (2016). 3D U-Net: Learning dense volumetric segmentation from sparse annotation. In S. Ourselin, L. Joskowicz, M. R. Sabuncu, G. Unal, & W. Wells (Eds.), *Medical image computing and computer-assisted intervention – MICCAI 2016* (pp. 424–432). Springer. https://doi.org/10.1007/978-3-319-46723-8_49

Dhar, J., Zaidi, N., Haghighat, M., Roy, S., Goyal, P., Alavi, A., & Kumar, V. (2025). Multimodal fusion learning with dual attention for medical imaging. *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision* (WACV), 4362–4371. https://doi.org/10.1109/WACV61041.2025.00428

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). An image is worth 16×16 words: Transformers for image

recognition at scale. *International Conference on Learning Representations*. https://arxiv.org/abs/2010.11929

Goldberger, A. L., Amaral, L. A. N., Glass, L., Hausdorff, J. M., Ivanov, P. C., Mark, R. G., Mietus, J. E., Moody, G. B., Peng, C.-K., & Stanley, H. E. (2000). PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals. *Circulation, 101*(23), e215–e220. https://doi.org/10.1161/01.CIR.101.23.e215

Gong, Z., Zhou, M., Dai, Y., Wen, Y., Liu, Y., & Zhen, Z. (2023). *A large-scale fMRI dataset for the visual processing of naturalistic scenes* (Version 2.1.2) [Data set]. OpenNeuro (ds004496). https://doi.org/10.18112/openneuro.ds004496.v2.1.2

Gramfort, A., Luessi, M., Larson, E., Engemann, D. A., Strohmeier, D., Brodbeck, C., Goj, R., Jas, M., Brooks, T., Parkkonen, L., & Hämäläinen, M. (2013). MEG and EEG data analysis with MNE-Python. *Frontiers in Neuroscience, 7,* Article 267. https://doi.org/10.3389/fnins.2013.00267

Gupta, N., Ang, L.-C., Noël de Tilly, L., Bidaisee, L., & Yücel, Y. H. (2006). Human glaucoma and neural degeneration in intracranial optic nerve, lateral geniculate nucleus, and visual cortex. *British Journal of Ophthalmology, 90*(6), 674–678. https://doi.org/10.1136/bjo.2005.086769

Gustafsson, F. (1996). Determining the initial states in forward-backward filtering. *IEEE Transactions on Signal Processing, 44*(4), 988–992. https://doi.org/10.1109/78.492552

Halliday, A. M., McDonald, W. I., & Mushin, J. (1972). Delayed visual evoked response in optic neuritis. *The Lancet, 299*(7758), 982–985. https://doi.org/10.1016/S0140-6736(72)91155-5

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., . . . Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 770–778. https://doi.org/10.1109/CVPR.2016.90

Himmelberg, M., Kurzawski, J., Benson, N. C., Pelli, D., Carrasco, M., & Winawer, J. (2026). *NYU retinotopy dataset* (Version 1.0.2) [Data set]. OpenNeuro (ds003787). https://doi.org/10.18112/openneuro.ds003787.v1.0.2

Hoopes, A., Mora, J. S., Dalca, A. V., Fischl, B., & Hoffmann, M. (2022). SynthStrip: Skull-stripping for any brain image. *NeuroImage, 260,* Article 119474. https://doi.org/10.1016/j.neuroimage.2022.119474

Hu, J., Shen, L., & Sun, G. (2018). Squeeze-and-excitation networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 7132–7141. https://doi.org/10.1109/CVPR.2018.00745

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of the 32nd International Conference on Machine Learning, 37*, 448–456.

Isensee, F., Schell, M., Pflueger, I., Brugnara, G., Bonekamp, D., Neuberger, U., Wick, A., Schlemmer, H.-P., Heiland, S., Wick, W., Bendszus, M., Maier-Hein, K. H., & Kickingereder, P. (2019). Automated brain extraction of multisequence MRI using artificial neural networks. *Human Brain Mapping, 40*(17), 4952–4964. https://doi.org/10.1002/hbm.24750

Jain, S., & Wallace, B. C. (2019). Attention is not explanation. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 3543–3556. https://doi.org/10.18653/v1/N19-1357

Kamnitsas, K., Ledig, C., Newcombe, V. F. J., Simpson, J. P., Kane, A. D., Menon, D. K., Rueckert, D., & Glocker, B. (2017). Efficient multi-scale 3D CNN with fully connected CRF for accurate brain lesion segmentation. *Medical Image Analysis, 36,* 61–78. https://doi.org/10.1016/j.media.2016.10.004

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations*. https://arxiv.org/abs/1412.6980

LaMontagne, P. J., Benzinger, T. L. S., Morris, J. C., Keefe, S., Hornbeck, R., Xiong, C., Grant, E., Hassenstab, J., Moulder, K., Vlassenko, A. G., Raichle, M. E., Cruchaga, C., & Marcus, D. (2019). *OASIS-3: Longitudinal neuroimaging, clinical, and cognitive dataset for normal aging and Alzheimer disease* [Preprint]. medRxiv. https://doi.org/10.1101/2019.12.13.19014902

Li, L., Han, Q., Bai, H., Zhang, X., Liu, Y., & He, C. (2026). *Structural MRI, resting-state fMRI, and PSG/EEG dataset of zoster-associated neuralgia* (Version 1.0.1) [Data set]. OpenNeuro (ds007181). https://doi.org/10.18112/openneuro.ds007181.v1.0.1

Liuzzi, L., Chang, K., Keren, H., Zheng, C., Saha, D., Nielson, D., & Stringaris, A. (2023). *Mood induction in MDD and healthy adolescents* (Version 1.0.4) [Data

set]. OpenNeuro (ds003568). https://doi.org/10.18112/openneuro.ds003568.v1.0.4

Lu, J., Batra, D., Parikh, D., & Lee, S. (2019). ViLBERT: Pretraining task-agnostic visiolinguistic representations for vision-and-language tasks. *Advances in Neural Information Processing Systems, 32*, 13–23.

NiBabel Developers. (n.d.). *NiBabel: Access a cacophony of neuro-imaging file formats* [Computer software documentation]. https://nipy.org/nibabel/

Odom, J. V., Bach, M., Brigell, M., Holder, G. E., McCulloch, D. L., Mizota, A., & Tormene, A. P. (2016). ISCEV standard for clinical visual evoked potentials: (2016 update). *Documenta Ophthalmologica, 133*(1), 1–9. https://doi.org/10.1007/s10633-016-9553-y

Oppenheim, A. V., & Schafer, R. W. (2010). *Discrete-time signal processing* (3rd ed.). Pearson.

Parisi, V., Miglior, S., Manni, G., Centofanti, M., & Bucci, M. G. (2006). Clinical ability of pattern electroretinograms and visual evoked potentials in detecting visual dysfunction in ocular hypertension and glaucoma. *Ophthalmology, 113*(2), 216–228. https://doi.org/10.1016/j.ophtha.2005.10.044

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., . . . Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems, 32*, 8024–8035.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Ramachandram, D., & Taylor, G. W. (2017). Deep multimodal learning: A survey on recent advances and trends. *IEEE Signal Processing Magazine, 34*(6), 96–108. https://doi.org/10.1109/MSP.2017.2738401

Sakki, H. E. A., Dale, N. J., Sargent, J., Perez-Roche, T., & Bowman, R. (2018). Is there consensus in defining childhood cerebral visual impairment? A systematic review of terminology and definitions. *British Journal of Ophthalmology, 102*(4), 424–432. https://doi.org/10.1136/bjophthalmol-2017-310694

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *Proceedings of the IEEE International Conference on Computer Vision*, 618–626. https://doi.org/10.1109/ICCV.2017.74

Tham, Y.-C., Li, X., Wong, T. Y., Quigley, H. A., Aung, T., & Cheng, C.-Y. (2014). Global prevalence of glaucoma and projections of glaucoma burden through 2040: A systematic review and meta-analysis. *Ophthalmology*, *121*(11), 2081–2090. https://doi.org/10.1016/j.ophtha.2014.05.013

Tsai, Y.-H. H., Bai, S., Liang, P. P., Kolter, J. Z., Morency, L.-P., & Salakhutdinov, R. (2019). Multimodal transformer for unaligned multimodal language sequences. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 6558–6569. https://doi.org/10.18653/v1/P19-1656

Van Essen, D. C., Smith, S. M., Barch, D. M., Behrens, T. E. J., Yacoub, E., & Ugurbil, K. (2013). The WU-Minn Human Connectome Project: An overview. *NeuroImage*, *80*, 62–79. https://doi.org/10.1016/j.neuroimage.2013.05.041

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, *30*, 5998–6008.

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., . . . SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Wiegreffe, S., & Pinter, Y. (2019). Attention is not not explanation. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 11–20. https://doi.org/10.18653/v1/D19-1002

Zhou, Z., Orlichenko, A., Qu, G., Fu, Z., Calhoun, V. D., Ding, Z., & Wang, Y.-P. (2024). *An interpretable cross-attentive multi-modal MRI fusion framework for schizophrenia diagnosis* (arXiv:2404.00144). arXiv. https://arxiv.org/abs/2404.00144

APPENDIX 1

DEFAULT CONFIGURATION

A1.1 CONFIGURATION PARAMETERS

All shapes and hyperparameters are defined in `src/config.py`. Table A1.1 summarises the default values of the current version of this file, which are used throughout the report unless a study overrides them (Sections 3.9 and 4.6). Appendix 2 reproduces the original version of the file.

Table A1.1: Default configuration values (current `src/config.py`).

Group	Parameter	Default value
MRI	target_shape / target_spacing / orientation / foreground_percentile	(64, 64, 64) / (1.0, 1.0, 1.0) mm / RAS / 60
MRI	mask_method / mask_morphology / normalization / fit	"percentile" / True / "minmax" / "crop"
VEP	length / sample_rate / bandpass / notch_freq / notch_quality	500 / 250 Hz / 1–100 Hz / 50 Hz / 30
VEP	baseline_window_ms / signal_mode	None / "waveform"
Model	d_model / num_heads / spatial_tokens / temporal_tokens	128 / 4 / 64 / 16
Model	ffn_hidden / dropout / num_classes / spatial_pool	256 / 0.1 / 3 / (4, 4, 4)
Model	fusion / pos_encoding / pooling	"bidirectional" / True / "attn"
Model	modality_dropout / fusion_blocks / se_reduction	0.0 / 1 / 4
Training	batch_size / epochs / lr / weight_decay	8 / 20 / 1e-3 / 1e-4
Training	aux_loss_weight / val_split / seed / checkpoint_dir	0.3 / 0.2 / 42 / checkpoints
Training	optimizer / scheduler / warmup_fraction / warmup_epochs	"adamw" / "cosine" / 0.1 / None
Training	max_grad_norm / patience / num_workers / amp	1.0 / None / 0 / False
Training	balance_classes / class_weights / label_smoothing / deterministic	False / None / 0.0 / False
Training	select_metric / augment / device	"val_loss" / True / None (automatic)
Augmentation	enabled / mri_flip_prob / mri_intensity_scale / mri_intensity_shift	True / 0.5 / (0.95, 1.05) / 0.02
Augmentation	mri_noise_std / vep_amplitude_scale / vep_noise_std	0.01 / (0.9, 1.1) / 0.02

APPENDIX 2
SOURCE CODE LISTINGS

A2.1 OVERVIEW

The listings below reproduce the source files of the `src/` package verbatim from the repository (commit `87362ec`). Line wrapping is introduced by the page width. The test suite is available in the repository under `tests/`. They are the original (v1) versions of the twelve modules, on which the measurements of Sections 5.2 to 5.4 and the coverage figure of Section 5.6 were made. The modules added or changed since (Table 4.6) are not reproduced here; they are available in the repository.

Table A2.1: Source files reproduced in this appendix (original versions).

File	Purpose
`src/config.py`	Central configuration (shape contract and hyperparameters).
`src/data/synthetic.py`	Class-conditioned synthetic MRI and VEP generators.
`src/data/dataset.py`	Manifest-driven and synthetic PyTorch datasets.
`src/preprocessing/mri_transforms.py`	MRI loading, resampling, masking, scaling and padding or cropping.
`src/preprocessing/signal_cleaner.py`	VEP validation, zero-phase filtering, resampling and normalisation.
`src/models/spatial_encoder.py`	3D residual CNN producing spatial tokens.
`src/models/temporal_encoder.py`	1D CNN producing temporal tokens.
`src/models/cross_attention.py`	Multi-head cross-attention fusion module.
`src/models/cafn.py`	The complete CAFN model and the multi-task loss.
`src/train.py`	Training loop, evaluation, checkpointing and command-line entry point.
`src/explainability/grad_cam_3d.py`	3D Grad-CAM on the fused logit.
`src/explainability/plot_attention.py`	Spatial–temporal attention heatmap.

A2.2 LISTINGS

Listing A2.1: src/config.py

```
"""Central configuration for shapes, hyperparameters, and class labels.

Keeping these in one place guarantees the preprocessing stream, both
encoders, the fusion module, and the training loop all agree on tensor
shapes without hardcoding magic numbers in multiple files.
"""

from dataclasses import dataclass, field
```

```
CLASS_NAMES = ("Healthy Control", "Glaucoma", "Optic Neuritis / CVI")


@dataclass(frozen=True)
class MRIConfig:
    target_shape: tuple[int, int, int] = (64, 64, 64)
    target_spacing: tuple[float, float, float] = (1.0, 1.0, 1.0)
    orientation: str = "RAS"
    foreground_percentile: float = 60.0  # intensity-threshold skull-strip approx


@dataclass(frozen=True)
class VEPConfig:
    length: int = 500
    sample_rate: float = 250.0  # Hz -> 500 samples spans 2.0s per ISCEV sweep
    bandpass_low: float = 1.0
    bandpass_high: float = 100.0
    notch_freq: float = 50.0
    notch_quality: float = 30.0


@dataclass(frozen=True)
class ModelConfig:
    d_model: int = 128
    num_heads: int = 4
    spatial_tokens: int = 64   # 4x4x4 adaptive pool
    temporal_tokens: int = 16
    spatial_pool: tuple[int, int, int] = (4, 4, 4)
    ffn_hidden: int = 256
    dropout: float = 0.1
    num_classes: int = len(CLASS_NAMES)


@dataclass(frozen=True)
class TrainConfig:
    batch_size: int = 8
    epochs: int = 20
    lr: float = 1e-3
    weight_decay: float = 1e-4
    aux_loss_weight: float = 0.3
    val_split: float = 0.2
    seed: int = 42
    checkpoint_dir: str = "checkpoints"


@dataclass(frozen=True)
class Config:
    mri: MRIConfig = field(default_factory=MRIConfig)
    vep: VEPConfig = field(default_factory=VEPConfig)
    model: ModelConfig = field(default_factory=ModelConfig)
    train: TrainConfig = field(default_factory=TrainConfig)


DEFAULT_CONFIG = Config()
```

Listing A2.2: src/data/synthetic.py

```
"""Synthetic MRI + VEP generators.

The datasets named in Project.md (OpenNeuro, HCP Retinotopy, OASIS, MNE
sample, PhysioNet) provide structural/functional scans and generic
electrophysiology, but none carry the clinical Healthy/Glaucoma/Optic
Neuritis-CVI labels this model targets -- that requires a curated clinical
cohort. This module generates class-conditioned synthetic MRI volumes and
VEP waveforms with the same interface real data would have, so the rest of
the system (preprocessing, model, training, explainability) can be built,
exercised, and tested end-to-end while a labeled cohort is sourced
separately.

Class-conditioning is deliberately simplistic but structured so a model can
learn from it: label 0 (Healthy) is unstructured background; label 1
(Glaucoma) has focal MRI intensity loss (atrophy proxy) and reduced VEP
amplitude; label 2 (Optic Neuritis / CVI) has focal MRI hyperintensity
(lesion proxy) and delayed VEP P100 latency (the clinical hallmark of optic
neuritis).
"""
```

```
import csv
import os

import nibabel as nib
import numpy as np

from src.config import Config, DEFAULT_CONFIG

NUM_CLASSES = DEFAULT_CONFIG.model.num_classes


def synthetic_mri_volume(
    label: int, shape: tuple[int, int, int] = (80, 96, 64), rng: np.random.Generator |
   None = None
) -> np.ndarray:
    """Generate a class-conditioned synthetic 3D MRI-like volume."""
    rng = rng or np.random.default_rng()
    volume = rng.normal(loc=100.0, scale=15.0, size=shape).clip(min=0)
    d, h, w = shape
    if label == 1:  # Glaucoma: focal atrophy (intensity loss)
        volume[d // 2 :, : h // 2, : w // 2] *= 0.4
    elif label == 2:  # Optic Neuritis / CVI: focal lesion (hyperintensity)
        volume[: d // 2, h // 2 :, w // 2 :] *= 1.8
    return volume.astype(np.float32)


def synthetic_vep_signal(
    label: int, fs: float = 250.0, duration_s: float = 2.0, rng: np.random.Generator |
   None = None
) -> np.ndarray:
    """Generate a class-conditioned synthetic VEP waveform (N75-P100-N145)."""
    rng = rng or np.random.default_rng()
    t = np.arange(0.0, duration_s, 1.0 / fs)

    if label == 0:  # Healthy
        latency, amplitude = 0.100, 6.0
    elif label == 1:  # Glaucoma: reduced amplitude, near-normal latency
        latency, amplitude = 0.108, 2.5
    else:  # Optic Neuritis / CVI: delayed P100 latency
        latency, amplitude = 0.140, 4.0

    p100 = amplitude * np.exp(-((t - latency) ** 2) / (2 * 0.012**2))
    n75 = -0.5 * amplitude * np.exp(-((t - (latency - 0.025)) ** 2) / (2 * 0.010**2))
    n145 = -0.4 * amplitude * np.exp(-((t - (latency + 0.045)) ** 2) / (2 * 0.015**2))
    waveform = p100 + n75 + n145

    drift = 0.3 * np.sin(2 * np.pi * 0.3 * t)
    mains = 0.15 * np.sin(2 * np.pi * 50.0 * t)
    emg_noise = rng.normal(0.0, 0.3, size=t.shape)

    return (waveform + drift + mains + emg_noise).astype(np.float64)


def write_synthetic_dataset(
    root: str,
    samples_per_class: int = 4,
    cfg: Config = DEFAULT_CONFIG,
    seed: int = 0,
) -> str:
    """Write synthetic NIfTI + VEP CSV files and a manifest.csv under `root`.

    Mirrors the data/raw/{mri,vep} layout described in Project.md. Returns
    the manifest CSV path, with columns: subject_id, mri_path, vep_path, fs, label.
    """
    rng = np.random.default_rng(seed)
    mri_dir = os.path.join(root, "mri")
    vep_dir = os.path.join(root, "vep")
    os.makedirs(mri_dir, exist_ok=True)
    os.makedirs(vep_dir, exist_ok=True)

    manifest_path = os.path.join(root, "manifest.csv")
    rows = []
    subject_idx = 0
    for label in range(NUM_CLASSES):
        for _ in range(samples_per_class):
            subject_id = f"sub-{subject_idx:04d}"
```

```
            volume = synthetic_mri_volume(label, rng=rng)
            affine = np.diag([1.0, 1.0, 1.0, 1.0])
            img = nib.Nifti1Image(volume, affine)
            mri_path = os.path.join(mri_dir, f"{subject_id}_mri.nii.gz")
            nib.save(img, mri_path)

            signal = synthetic_vep_signal(label, fs=cfg.vep.sample_rate, rng=rng)
            vep_path = os.path.join(vep_dir, f"{subject_id}_vep.csv")
            with open(vep_path, "w", newline="") as f:
                writer = csv.writer(f)
                writer.writerow(["amplitude"])
                for value in signal:
                    writer.writerow([value])

            rows.append(
                {
                    "subject_id": subject_id,
                    "mri_path": mri_path,
                    "vep_path": vep_path,
                    "fs": cfg.vep.sample_rate,
                    "label": label,
                }
            )
            subject_idx += 1

    with open(manifest_path, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["subject_id", "mri_path", "vep_path",
   "fs", "label"])
        writer.writeheader()
        writer.writerows(rows)

    return manifest_path
```

Listing A2.3: src/data/dataset.py

```
"""PyTorch Dataset classes for paired MRI + VEP samples.

`PairedNeuroDataset` reads a manifest CSV (subject_id, mri_path, vep_path,
fs, label) and applies the real preprocessing pipeline -- this is what a
labeled clinical cohort would plug into. `SyntheticNeuroDataset` generates
class-conditioned synthetic samples in-memory (no disk I/O) through the same
preprocessing pipeline, used for fast unit tests, integration tests, and
development/training-loop validation ahead of real labeled data.
"""

import csv

import numpy as np
import torch
from torch.utils.data import Dataset

from src.config import Config, DEFAULT_CONFIG
from src.data.synthetic import synthetic_mri_volume, synthetic_vep_signal
from src.preprocessing.mri_transforms import preprocess_mri_file,
   preprocess_mri_volume
from src.preprocessing.signal_cleaner import clean_vep_signal


class PairedNeuroDataset(Dataset):
    """Loads (MRI volume, VEP signal, label) triples from a manifest CSV."""

    def __init__(self, manifest_path: str, cfg: Config = DEFAULT_CONFIG):
        self.cfg = cfg
        with open(manifest_path, newline="") as f:
            self.rows = list(csv.DictReader(f))
        if not self.rows:
            raise ValueError(f"Manifest at {manifest_path} has no rows")

    def __len__(self) -> int:
        return len(self.rows)

    def __getitem__(self, idx: int) -> dict:
        row = self.rows[idx]
        mri = preprocess_mri_file(row["mri_path"], self.cfg.mri)

        raw_signal = np.loadtxt(row["vep_path"], delimiter=",", skiprows=1)
        fs = float(row.get("fs") or self.cfg.vep.sample_rate)
```

```
        vep = clean_vep_signal(raw_signal, fs, self.cfg.vep)

        return {
            "mri": torch.from_numpy(mri),
            "vep": torch.from_numpy(vep).unsqueeze(0),
            "label": torch.tensor(int(row["label"]), dtype=torch.long),
            "subject_id": row["subject_id"],
        }


class SyntheticNeuroDataset(Dataset):
    """In-memory class-conditioned synthetic dataset, real preprocessing applied."""

    def __init__(
        self,
        num_samples: int,
        cfg: Config = DEFAULT_CONFIG,
        seed: int = 0,
        raw_mri_shape: tuple[int, int, int] = (80, 96, 64),
    ):
        self.cfg = cfg
        self.num_samples = num_samples
        self.raw_mri_shape = raw_mri_shape
        self.rng = np.random.default_rng(seed)
        num_classes = cfg.model.num_classes
        # Balanced labels, deterministically shuffled.
        base_labels = [i % num_classes for i in range(num_samples)]
        self.rng.shuffle(base_labels)
        self.labels = base_labels

    def __len__(self) -> int:
        return self.num_samples

    def __getitem__(self, idx: int) -> dict:
        label = self.labels[idx]
        raw_volume = synthetic_mri_volume(label, shape=self.raw_mri_shape,
   rng=self.rng)
        mri = preprocess_mri_volume(raw_volume, zooms=(1.0, 1.0, 1.0),
   cfg=self.cfg.mri)

        raw_signal = synthetic_vep_signal(label, fs=self.cfg.vep.sample_rate,
   rng=self.rng)
        vep = clean_vep_signal(raw_signal, self.cfg.vep.sample_rate, self.cfg.vep)

        return {
            "mri": torch.from_numpy(mri),
            "vep": torch.from_numpy(vep).unsqueeze(0),
            "label": torch.tensor(label, dtype=torch.long),
            "subject_id": f"synthetic-{idx:04d}",
        }
```

Listing A2.4: src/preprocessing/mri_transforms.py

```
"""3D sMRI spatial standardization: RAS reorientation, 1mm isotropic
resampling, intensity-threshold foreground masking, and normalization.

Note on skull-stripping: true skull-stripping (FSL BET / HD-BET / SynthStrip)
requires an external binary or a pretrained segmentation network that is not
pip-installable. This module ships a lightweight intensity-percentile
foreground mask as a prototype-stage approximation -- it zeroes low-intensity
background/skull voxels without needing an external dependency. Swap in
HD-BET or SynthStrip for clinical-grade deployment; the rest of the pipeline
is agnostic to which foreground mask produced its input.
"""

import nibabel as nib
import numpy as np
from monai.transforms import Resize, ResizeWithPadOrCrop, ScaleIntensity

from src.config import MRIConfig, DEFAULT_CONFIG


def load_nifti(path: str) -> tuple[np.ndarray, np.ndarray, tuple[float, float,
   float]]:
    """Load a NIfTI volume, reoriented to RAS (nibabel's closest-canonical).

    Returns (data, affine, voxel_zooms).
```

```
    """
    img = nib.load(path)
    img = nib.as_closest_canonical(img)
    data = np.asarray(img.get_fdata(), dtype=np.float32)
    if data.ndim != 3:
        raise ValueError(f"Expected a 3D volume, got shape {data.shape}")
    zooms = tuple(float(z) for z in img.header.get_zooms()[:3])
    if any(z <= 0 for z in zooms):
        raise ValueError(f"Invalid voxel spacing in header: {zooms}")
    return data, img.affine, zooms


def foreground_mask_threshold(volume: np.ndarray, percentile: float) -> np.ndarray:
    """Zero out background/skull voxels below an intensity percentile.

    Percentile is computed over strictly-positive voxels so that an
    already-large air/background region does not collapse the threshold to
    zero.
    """
    nonzero = volume[volume > 0]
    if nonzero.size == 0:
        return volume
    threshold = np.percentile(nonzero, percentile)
    masked = volume.copy()
    masked[masked < threshold] = 0.0
    return masked


def resample_isotropic(
    volume: np.ndarray,
    zooms: tuple[float, float, float],
    target_spacing: tuple[float, float, float] = (1.0, 1.0, 1.0),
) -> np.ndarray:
    """Resample a volume to isotropic spacing by scaling voxel counts.

    MONAI's shape-based `Resize` combined with a zoom-derived target shape
    is mathematically equivalent to physical-spacing resampling and avoids
    MetaTensor/affine bookkeeping for a standalone array pipeline.
    """
    new_shape = tuple(
        max(1, round(dim * zoom / target))
        for dim, zoom, target in zip(volume.shape, zooms, target_spacing)
    )
    resize = Resize(spatial_size=new_shape, mode="trilinear", align_corners=False)
    resized = resize(volume[np.newaxis, ...])  # add channel dim
    return np.asarray(resized[0])


def preprocess_mri_volume(
    volume: np.ndarray,
    zooms: tuple[float, float, float],
    cfg: MRIConfig = DEFAULT_CONFIG.mri,
) -> np.ndarray:
    """Full spatial standardization pipeline for an already-RAS volume.

    Returns a float32 array of shape (1, *cfg.target_shape) scaled to [0, 1].
    """
    if volume.ndim != 3:
        raise ValueError(f"Expected a 3D volume, got shape {volume.shape}")

    isotropic = resample_isotropic(volume, zooms, cfg.target_spacing)
    masked = foreground_mask_threshold(isotropic, cfg.foreground_percentile)

    if masked.max() > masked.min():
        scaler = ScaleIntensity(minv=0.0, maxv=1.0)
        normalized = np.asarray(scaler(masked[np.newaxis, ...]))
    else:
        normalized = masked[np.newaxis, ...].astype(np.float32)

    pad_crop = ResizeWithPadOrCrop(spatial_size=cfg.target_shape)
    standardized = np.asarray(pad_crop(normalized))
    return standardized.astype(np.float32)


def preprocess_mri_file(path: str, cfg: MRIConfig = DEFAULT_CONFIG.mri) -> np.ndarray:
    """Convenience wrapper: load a NIfTI file and run full standardization."""
    data, _, zooms = load_nifti(path)
```

```
    return preprocess_mri_volume(data, zooms, cfg)
```

Listing A2.5: src/preprocessing/signal_cleaner.py

```
"""1D VEP signal conditioning: zero-phase Butterworth bandpass + mains notch.

Filtering order matters for VEP work: bandpass first removes drift and
high-frequency EMG jitter, then the notch removes the narrowband 50Hz mains
hum. Both filters use zero-phase (filtfilt / sosfiltfilt) application so the
P100 peak latency -- the clinically meaningful measurement -- is not
shifted in time by the filter's group delay.
"""

import numpy as np
from scipy.signal import butter, iirnotch, sosfiltfilt, filtfilt

from src.config import VEPConfig, DEFAULT_CONFIG


def _validate_signal(signal: np.ndarray) -> np.ndarray:
    signal = np.asarray(signal, dtype=np.float64)
    if signal.ndim != 1:
        raise ValueError(f"Expected a 1D signal, got shape {signal.shape}")
    if signal.size == 0:
        raise ValueError("Signal is empty")
    if not np.all(np.isfinite(signal)):
        raise ValueError("Signal contains NaN or Inf values")
    return signal


def butterworth_bandpass(
    signal: np.ndarray,
    fs: float,
    lowcut: float = 1.0,
    highcut: float = 100.0,
    order: int = 4,
) -> np.ndarray:
    """Zero-phase 4th-order Butterworth bandpass filter."""
    signal = _validate_signal(signal)
    nyquist = fs / 2.0
    if lowcut <= 0:
        raise ValueError("lowcut must be > 0")
    if highcut >= nyquist:
        raise ValueError(
            f"highcut ({highcut} Hz) must be below the Nyquist frequency "
            f"({nyquist} Hz) for fs={fs} Hz"
        )
    if lowcut >= highcut:
        raise ValueError("lowcut must be < highcut")

    sos = butter(order, [lowcut, highcut], btype="bandpass", fs=fs, output="sos")
    # Zero-phase filtering needs enough samples relative to the filter's
    # padding requirement; short synthetic signals can trip this up.
    min_len = 3 * (2 * len(sos) + 1)
    if signal.size <= min_len:
        raise ValueError(
            f"Signal too short ({signal.size} samples) for zero-phase "
            f"filtering; need > {min_len} samples"
        )
    return sosfiltfilt(sos, signal)


def notch_filter(
    signal: np.ndarray,
    fs: float,
    freq: float = 50.0,
    quality: float = 30.0,
) -> np.ndarray:
    """Zero-phase IIR notch filter to remove AC power-line hum."""
    signal = _validate_signal(signal)
    nyquist = fs / 2.0
    if freq >= nyquist:
        raise ValueError(
            f"notch freq ({freq} Hz) must be below Nyquist ({nyquist} Hz)"
        )
    b, a = iirnotch(freq, quality, fs)
    return filtfilt(b, a, signal)
```

```
def zscore_normalize(signal: np.ndarray, eps: float = 1e-8) -> np.ndarray:
    """Zero mean, unit variance normalization."""
    signal = _validate_signal(signal)
    std = signal.std()
    return (signal - signal.mean()) / (std + eps)


def resample_to_length(signal: np.ndarray, target_length: int) -> np.ndarray:
    """Linear-interpolation resample to a fixed number of samples."""
    signal = _validate_signal(signal)
    if signal.size == target_length:
        return signal
    x_old = np.linspace(0.0, 1.0, num=signal.size)
    x_new = np.linspace(0.0, 1.0, num=target_length)
    return np.interp(x_new, x_old, signal)


def clean_vep_signal(
    raw_signal: np.ndarray,
    fs: float,
    cfg: VEPConfig = DEFAULT_CONFIG.vep,
) -> np.ndarray:
    """Full 1D conditioning pipeline: bandpass -> notch -> resample -> z-score.

    Returns a float32 array of shape (cfg.length,).
    """
    signal = _validate_signal(raw_signal)
    filtered = butterworth_bandpass(
        signal, fs, lowcut=cfg.bandpass_low, highcut=cfg.bandpass_high
    )
    denotched = notch_filter(filtered, fs, freq=cfg.notch_freq,
   quality=cfg.notch_quality)
    resampled = resample_to_length(denotched, cfg.length)
    normalized = zscore_normalize(resampled)
    return normalized.astype(np.float32)
```

Listing A2.6: src/models/spatial_encoder.py

```
"""3D-CNN / ResNet backbone: structural MRI volume -> spatial tokens."""

import torch
import torch.nn as nn

from src.config import ModelConfig, DEFAULT_CONFIG


class ResidualBlock3D(nn.Module):
    def __init__(self, in_channels: int, out_channels: int, stride: int = 1):
        super().__init__()
        self.conv1 = nn.Conv3d(in_channels, out_channels, 3, stride=stride, padding=1,
   bias=False)
        self.bn1 = nn.BatchNorm3d(out_channels)
        self.conv2 = nn.Conv3d(out_channels, out_channels, 3, stride=1, padding=1,
   bias=False)
        self.bn2 = nn.BatchNorm3d(out_channels)
        self.relu = nn.ReLU(inplace=True)

        self.shortcut = None
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv3d(in_channels, out_channels, 1, stride=stride, bias=False),
                nn.BatchNorm3d(out_channels),
            )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        identity = x if self.shortcut is None else self.shortcut(x)
        out = self.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return self.relu(out + identity)


class SpatialEncoder3D(nn.Module):
    """(B, 1, D, H, W) -> (B, spatial_tokens, d_model) spatial token sequence.

    `self.last_conv` exposes the final residual block so Grad-CAM3D can hook
```

```
    its output activation map.
    """

    def __init__(self, cfg: ModelConfig = DEFAULT_CONFIG.model, in_channels: int = 1):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv3d(in_channels, 16, 3, stride=1, padding=1, bias=False),
            nn.BatchNorm3d(16),
            nn.ReLU(inplace=True),
        )
        self.layer1 = ResidualBlock3D(16, 32, stride=2)
        self.layer2 = ResidualBlock3D(32, 64, stride=2)
        self.layer3 = ResidualBlock3D(64, cfg.d_model, stride=2)
        self.last_conv = self.layer3

        self.pool = nn.AdaptiveAvgPool3d(cfg.spatial_pool)
        self.d_model = cfg.d_model

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.stem(x)
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.pool(x)  # (B, d_model, *spatial_pool)
        tokens = x.flatten(2).transpose(1, 2)  # (B, tokens, d_model)
        return tokens
```

Listing A2.7: src/models/temporal_encoder.py

```
"""1D-CNN latency backbone: VEP waveform -> temporal tokens."""

import torch
import torch.nn as nn

from src.config import ModelConfig, DEFAULT_CONFIG


class TemporalEncoder1D(nn.Module):
    """(B, 1, L) -> (B, temporal_tokens, d_model) temporal token sequence."""

    def __init__(self, cfg: ModelConfig = DEFAULT_CONFIG.model, in_channels: int = 1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv1d(in_channels, 16, kernel_size=7, stride=1, padding=3,
  bias=False),
            nn.BatchNorm1d(16),
            nn.ReLU(inplace=True),
            nn.Conv1d(16, 32, kernel_size=5, stride=2, padding=2, bias=False),
            nn.BatchNorm1d(32),
            nn.ReLU(inplace=True),
            nn.Conv1d(32, 64, kernel_size=5, stride=2, padding=2, bias=False),
            nn.BatchNorm1d(64),
            nn.ReLU(inplace=True),
            nn.Conv1d(64, cfg.d_model, kernel_size=3, stride=2, padding=1,
  bias=False),
            nn.BatchNorm1d(cfg.d_model),
            nn.ReLU(inplace=True),
        )
        self.pool = nn.AdaptiveAvgPool1d(cfg.temporal_tokens)
        self.d_model = cfg.d_model

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.net(x)          # (B, d_model, L')
        x = self.pool(x)         # (B, d_model, temporal_tokens)
        tokens = x.transpose(1, 2)  # (B, temporal_tokens, d_model)
        return tokens
```

Listing A2.8: src/models/cross_attention.py

```
"""Multi-Head Cross-Attention Fusion Module (CAFM).

Spatial 3D tokens are projected as Queries; temporal 1D latency tokens are
projected as Keys and Values, so each spatial brain region dynamically
queries the electrophysiological latency bins:

    Attention(Q, K, V) = Softmax(Q K^T / sqrt(d_k)) V
"""
```

```
import math

import torch
import torch.nn as nn

from src.config import ModelConfig, DEFAULT_CONFIG


class MultiHeadCrossAttention(nn.Module):
    """Q from `query_tokens`, K/V from `kv_tokens`. Sequence lengths may differ."""

    def __init__(self, cfg: ModelConfig = DEFAULT_CONFIG.model):
        super().__init__()
        if cfg.d_model % cfg.num_heads != 0:
            raise ValueError("d_model must be divisible by num_heads")
        self.num_heads = cfg.num_heads
        self.head_dim = cfg.d_model // cfg.num_heads
        self.d_model = cfg.d_model

        self.w_q = nn.Linear(cfg.d_model, cfg.d_model)
        self.w_k = nn.Linear(cfg.d_model, cfg.d_model)
        self.w_v = nn.Linear(cfg.d_model, cfg.d_model)
        self.out_proj = nn.Linear(cfg.d_model, cfg.d_model)
        self.dropout = nn.Dropout(cfg.dropout)

    def _split_heads(self, x: torch.Tensor) -> torch.Tensor:
        b, n, _ = x.shape
        return x.view(b, n, self.num_heads, self.head_dim).transpose(1, 2)  # (B, H,
   N, Dh)

    def forward(self, query_tokens: torch.Tensor, kv_tokens: torch.Tensor):
        b = query_tokens.shape[0]
        q = self._split_heads(self.w_q(query_tokens))
        k = self._split_heads(self.w_k(kv_tokens))
        v = self._split_heads(self.w_v(kv_tokens))

        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.head_dim)
        attn = torch.softmax(scores, dim=-1)  # (B, H, Nq, Nk)
        attn = self.dropout(attn)
        context = torch.matmul(attn, v)  # (B, H, Nq, Dh)

        context = context.transpose(1, 2).contiguous().view(b, -1, self.d_model)
        out = self.out_proj(context)
        return out, attn


class CrossAttentionFusion(nn.Module):
    """CAFM: cross-attention -> residual + LayerNorm -> feed-forward block."""

    def __init__(self, cfg: ModelConfig = DEFAULT_CONFIG.model):
        super().__init__()
        self.cross_attn = MultiHeadCrossAttention(cfg)
        self.norm1 = nn.LayerNorm(cfg.d_model)
        self.ffn = nn.Sequential(
            nn.Linear(cfg.d_model, cfg.ffn_hidden),
            nn.ReLU(inplace=True),
            nn.Dropout(cfg.dropout),
            nn.Linear(cfg.ffn_hidden, cfg.d_model),
        )
        self.norm2 = nn.LayerNorm(cfg.d_model)

    def forward(self, spatial_tokens: torch.Tensor, temporal_tokens: torch.Tensor):
        attn_out, attn_weights = self.cross_attn(spatial_tokens, temporal_tokens)
        fused = self.norm1(spatial_tokens + attn_out)
        ffn_out = self.ffn(fused)
        fused = self.norm2(fused + ffn_out)
        return fused, attn_weights
```

Listing A2.9: src/models/cafn.py

```
"""Cross-Attention Fusion Network (CAFN): full model wiring.

Dual-stream encoders feed the cross-attention fusion module; a fused
classification head and two auxiliary unimodal heads train jointly
(multi-task supervision) so the network cannot collapse onto a single
dominant modality.
```

```
"""

import torch
import torch.nn as nn
import torch.nn.functional as F

from src.config import ModelConfig, DEFAULT_CONFIG
from src.models.spatial_encoder import SpatialEncoder3D
from src.models.temporal_encoder import TemporalEncoder1D
from src.models.cross_attention import CrossAttentionFusion


class CAFN(nn.Module):
    def __init__(self, cfg: ModelConfig = DEFAULT_CONFIG.model):
        super().__init__()
        self.cfg = cfg
        self.spatial_encoder = SpatialEncoder3D(cfg)
        self.temporal_encoder = TemporalEncoder1D(cfg)
        self.fusion = CrossAttentionFusion(cfg)

        self.fused_head = nn.Linear(cfg.d_model, cfg.num_classes)
        self.spatial_head = nn.Linear(cfg.d_model, cfg.num_classes)
        self.temporal_head = nn.Linear(cfg.d_model, cfg.num_classes)

    def forward(self, mri_volume: torch.Tensor, vep_signal: torch.Tensor) -> dict:
        spatial_tokens = self.spatial_encoder(mri_volume)    # (B, S, d)
        temporal_tokens = self.temporal_encoder(vep_signal)  # (B, T, d)

        fused_tokens, attn_weights = self.fusion(spatial_tokens, temporal_tokens)

        fused_embedding = fused_tokens.mean(dim=1)
        spatial_embedding = spatial_tokens.mean(dim=1)
        temporal_embedding = temporal_tokens.mean(dim=1)

        return {
            "logits_fused": self.fused_head(fused_embedding),
            "logits_spatial": self.spatial_head(spatial_embedding),
            "logits_temporal": self.temporal_head(temporal_embedding),
            "attn_weights": attn_weights,  # (B, num_heads, S, T)
            "fused_tokens": fused_tokens,
            "spatial_tokens": spatial_tokens,
            "temporal_tokens": temporal_tokens,
        }

    def multi_task_loss(self, outputs: dict, labels: torch.Tensor, aux_weight: float =
  0.3):
        loss_fused = F.cross_entropy(outputs["logits_fused"], labels)
        loss_spatial = F.cross_entropy(outputs["logits_spatial"], labels)
        loss_temporal = F.cross_entropy(outputs["logits_temporal"], labels)
        total = loss_fused + aux_weight * (loss_spatial + loss_temporal)
        return total, {
            "loss_fused": loss_fused.item(),
            "loss_spatial": loss_spatial.item(),
            "loss_temporal": loss_temporal.item(),
            "loss_total": total.item(),
        }
```

Listing A2.10: src/train.py

```
"""Multi-task training loop for CAFN.

Trains the fused cross-attention head jointly with the two auxiliary
unimodal heads (`CAFN.multi_task_loss`). Callable as a library function
(`train_model`) for tests/notebooks, or as a script:

    python -m src.train --epochs 20 --synthetic-samples 64
"""

import argparse
import os

import torch
from torch.utils.data import DataLoader, Subset

from src.config import Config, DEFAULT_CONFIG
from src.data.dataset import SyntheticNeuroDataset
from src.models.cafn import CAFN
```

```
def train_val_split(dataset, val_split: float, seed: int) -> tuple[Subset, Subset]:
    n = len(dataset)
    n_val = max(1, int(n * val_split)) if n > 1 else 0
    generator = torch.Generator().manual_seed(seed)
    perm = torch.randperm(n, generator=generator).tolist()
    val_idx, train_idx = perm[:n_val], perm[n_val:]
    if not train_idx:
        train_idx, val_idx = perm, []
    return Subset(dataset, train_idx), Subset(dataset, val_idx)


@torch.no_grad()
def evaluate(model: CAFN, loader: DataLoader, device: torch.device) -> dict:
    model.eval()
    total, correct, loss_sum = 0, 0, 0.0
    for batch in loader:
        mri = batch["mri"].to(device)
        vep = batch["vep"].to(device)
        labels = batch["label"].to(device)
        outputs = model(mri, vep)
        loss, _ = model.multi_task_loss(outputs, labels)
        preds = outputs["logits_fused"].argmax(dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)
        loss_sum += loss.item() * labels.size(0)
    if total == 0:
        return {"loss": float("nan"), "accuracy": float("nan")}
    return {"loss": loss_sum / total, "accuracy": correct / total}


def train_model(
    model: CAFN,
    train_dataset,
    val_dataset=None,
    cfg: Config = DEFAULT_CONFIG,
    device: torch.device | None = None,
    verbose: bool = False,
) -> dict:
    """Runs the multi-task training loop. Returns per-epoch history."""
    device = device or torch.device("cpu")
    model.to(device)

    train_loader = DataLoader(train_dataset, batch_size=cfg.train.batch_size,
   shuffle=True)
    val_loader = (
        DataLoader(val_dataset, batch_size=cfg.train.batch_size, shuffle=False)
        if val_dataset is not None and len(val_dataset) > 0
        else None
    )

    optimizer = torch.optim.Adam(
        model.parameters(), lr=cfg.train.lr, weight_decay=cfg.train.weight_decay
    )

    history = {"train_loss": [], "val_loss": [], "val_accuracy": []}
    for epoch in range(cfg.train.epochs):
        model.train()
        epoch_loss, n_samples = 0.0, 0
        for batch in train_loader:
            mri = batch["mri"].to(device)
            vep = batch["vep"].to(device)
            labels = batch["label"].to(device)

            optimizer.zero_grad()
            outputs = model(mri, vep)
            loss, _ = model.multi_task_loss(outputs, labels,
   aux_weight=cfg.train.aux_loss_weight)
            loss.backward()
            optimizer.step()

            epoch_loss += loss.item() * labels.size(0)
            n_samples += labels.size(0)

        train_loss = epoch_loss / max(n_samples, 1)
        history["train_loss"].append(train_loss)
```

```
        if val_loader is not None:
            metrics = evaluate(model, val_loader, device)
            history["val_loss"].append(metrics["loss"])
            history["val_accuracy"].append(metrics["accuracy"])
            if verbose:
                print(
                    f"epoch {epoch + 1}/{cfg.train.epochs} "
                    f"train_loss={train_loss:.4f} val_loss={metrics['loss']:.4f} "
                    f"val_acc={metrics['accuracy']:.4f}"
                )
        elif verbose:
            print(f"epoch {epoch + 1}/{cfg.train.epochs} train_loss={train_loss:.4f}")

    return history


def save_checkpoint(model: CAFN, path: str, cfg: Config = DEFAULT_CONFIG) -> None:
    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
    torch.save({"model_state_dict": model.state_dict(), "model_config": cfg.model},
   path)


def load_checkpoint(path: str, cfg: Config = DEFAULT_CONFIG) -> CAFN:
    checkpoint = torch.load(path, map_location="cpu", weights_only=False)
    model = CAFN(checkpoint.get("model_config", cfg.model))
    model.load_state_dict(checkpoint["model_state_dict"])
    return model


def main():
    parser = argparse.ArgumentParser(description="Train CAFN on synthetic or manifest
   data")
    parser.add_argument("--epochs", type=int, default=DEFAULT_CONFIG.train.epochs)
    parser.add_argument("--batch-size", type=int,
   default=DEFAULT_CONFIG.train.batch_size)
    parser.add_argument("--lr", type=float, default=DEFAULT_CONFIG.train.lr)
    parser.add_argument("--synthetic-samples", type=int, default=64)
    parser.add_argument("--checkpoint-name", type=str, default="cafn.pt")
    parser.add_argument("--seed", type=int, default=DEFAULT_CONFIG.train.seed)
    args = parser.parse_args()

    torch.manual_seed(args.seed)

    cfg = Config(
        train=DEFAULT_CONFIG.train.__class__(
            batch_size=args.batch_size,
            epochs=args.epochs,
            lr=args.lr,
            weight_decay=DEFAULT_CONFIG.train.weight_decay,
            aux_loss_weight=DEFAULT_CONFIG.train.aux_loss_weight,
            val_split=DEFAULT_CONFIG.train.val_split,
            seed=args.seed,
            checkpoint_dir=DEFAULT_CONFIG.train.checkpoint_dir,
        )
    )

    dataset = SyntheticNeuroDataset(num_samples=args.synthetic_samples, cfg=cfg,
   seed=args.seed)
    train_ds, val_ds = train_val_split(dataset, cfg.train.val_split, cfg.train.seed)

    model = CAFN(cfg.model)
    history = train_model(model, train_ds, val_ds, cfg=cfg, verbose=True)

    checkpoint_path = os.path.join(cfg.train.checkpoint_dir, args.checkpoint_name)
    save_checkpoint(model, checkpoint_path, cfg)
    print(f"Saved checkpoint to {checkpoint_path}")
    print(f"Final train loss: {history['train_loss'][-1]:.4f}")


if __name__ == "__main__":
    main()
```

Listing A2.11: src/explainability/grad_cam_3d.py

```
"""Grad-CAM for the 3D spatial stream of CAFN.
```

```
Backprop starts from the *fused* classification logit by default, so the
resulting activation map reflects which anatomical zones drove the
multimodal decision (not just the unimodal spatial head) -- gradients flow
from `logits_fused` back through the cross-attention fusion module into the
spatial encoder's last residual block.
"""

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F


class GradCAM3D:
    def __init__(self, model: nn.Module, target_layer: nn.Module | None = None):
        self.model = model
        self.target_layer = target_layer or model.spatial_encoder.last_conv
        self._activations = None
        self._gradients = None
        self._fwd_handle =
   self.target_layer.register_forward_hook(self._save_activation)
        self._bwd_handle =
   self.target_layer.register_full_backward_hook(self._save_gradient)

    def _save_activation(self, module, inp, out):
        self._activations = out.detach()

    def _save_gradient(self, module, grad_input, grad_output):
        self._gradients = grad_output[0].detach()

    def remove_hooks(self) -> None:
        self._fwd_handle.remove()
        self._bwd_handle.remove()

    def generate(
        self,
        mri_volume: torch.Tensor,
        vep_signal: torch.Tensor,
        target_class: int | torch.Tensor | None = None,
        logits_key: str = "logits_fused",
    ) -> tuple[torch.Tensor, torch.Tensor]:
        """Returns (cam, target_class): cam is (B, D, H, W) in [0, 1]."""
        was_training = self.model.training
        self.model.eval()
        self.model.zero_grad(set_to_none=True)

        outputs = self.model(mri_volume, vep_signal)
        logits = outputs[logits_key]

        if target_class is None:
            target_class = logits.argmax(dim=1)
        elif isinstance(target_class, int):
            target_class = torch.full(
                (logits.shape[0],), target_class, dtype=torch.long,
   device=logits.device
            )

        selected = logits.gather(1, target_class.view(-1, 1)).sum()
        selected.backward()

        if was_training:
            self.model.train()

        activations = self._activations  # (B, C, d, h, w)
        gradients = self._gradients  # (B, C, d, h, w)
        if activations is None or gradients is None:
            raise RuntimeError("Hooks did not capture activations/gradients; check
   target_layer")

        weights = gradients.mean(dim=(2, 3, 4), keepdim=True)
        cam = F.relu((weights * activations).sum(dim=1, keepdim=True))  # (B, 1, d, h,
   w)
```

```
        target_size = mri_volume.shape[2:]
        cam = F.interpolate(cam, size=target_size, mode="trilinear",
   align_corners=False)
        cam = cam.squeeze(1)  # (B, D, H, W)

        cam_min = cam.amin(dim=(1, 2, 3), keepdim=True)
        cam_max = cam.amax(dim=(1, 2, 3), keepdim=True)
        denom = (cam_max - cam_min).clamp(min=1e-8)
        cam = (cam - cam_min) / denom

        return cam.detach(), target_class.detach()


def compute_gradcam(
    model: nn.Module,
    mri_volume: torch.Tensor,
    vep_signal: torch.Tensor,
    target_class: int | torch.Tensor | None = None,
) -> tuple[torch.Tensor, torch.Tensor]:
    """One-shot convenience wrapper around GradCAM3D that cleans up its hooks."""
    cam_engine = GradCAM3D(model)
    try:
        return cam_engine.generate(mri_volume, vep_signal, target_class)
    finally:
        cam_engine.remove_hooks()


def plot_gradcam_slices(
    volume: np.ndarray,
    cam: np.ndarray,
    save_path: str,
    title: str = "Grad-CAM: Visual Cortex Activation",
) -> None:
    """Overlay a Grad-CAM heatmap on the mid axial/coronal/sagittal slices."""
    if volume.shape != cam.shape:
        raise ValueError(f"volume shape {volume.shape} != cam shape {cam.shape}")

    d, h, w = volume.shape
    slices = [
        (volume[d // 2, :, :], cam[d // 2, :, :], "Axial"),
        (volume[:, h // 2, :], cam[:, h // 2, :], "Coronal"),
        (volume[:, :, w // 2], cam[:, :, w // 2], "Sagittal"),
    ]

    fig, axes = plt.subplots(1, 3, figsize=(12, 4))
    for ax, (vol_slice, cam_slice, name) in zip(axes, slices):
        ax.imshow(vol_slice, cmap="gray")
        ax.imshow(cam_slice, cmap="jet", alpha=0.5)
        ax.set_title(name)
        ax.axis("off")
    fig.suptitle(title)
    fig.tight_layout()
    fig.savefig(save_path, dpi=150)
    plt.close(fig)
```

Listing A2.12: src/explainability/plot_attention.py

```
"""2D spatial-temporal cross-attention correlation heatmaps.

Reduces the multi-head cross-attention weights (B, H, S, T) produced by
`CrossAttentionFusion` down to a single (S, T) matrix -- spatial cortex
tokens against VEP temporal latency bins -- so clinicians can see which
brain regions the model correlated with which millisecond latency window.
"""

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import torch


def cross_attention_heatmap(attn_weights: torch.Tensor) -> np.ndarray:
    """Average (B, H, S, T) attention weights over batch and heads -> (S, T)."""
    if attn_weights.dim() != 4:
        raise ValueError(
```

```
            f"Expected (B, H, S, T) attention weights, got shape
   {tuple(attn_weights.shape)}"
        )
    reduced = attn_weights.mean(dim=(0, 1))
    return reduced.detach().cpu().numpy()


def plot_cross_attention(
    attn_weights: torch.Tensor,
    save_path: str,
    xlabel: str = "Temporal latency bin",
    ylabel: str = "Spatial cortex token",
    title: str = "Spatial-Temporal Cross-Attention",
) -> np.ndarray:
    """Render and save the cross-attention heatmap; returns the plotted array."""
    heatmap = cross_attention_heatmap(attn_weights)

    fig, ax = plt.subplots(figsize=(6, 8))
    im = ax.imshow(heatmap, aspect="auto", cmap="viridis")
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.set_title(title)
    fig.colorbar(im, ax=ax, label="Attention weight")
    fig.tight_layout()
    fig.savefig(save_path, dpi=150)
    plt.close(fig)

    return heatmap
```

APPENDIX 3

TEST INVENTORY

A3.1 TEST FILES

Table A3.1 and Listing A3.1 list the original suite of 86 test functions in 12 files, as recorded in the repository's pytest node-identifier file at the time of the first prototype. The suite now contains 340 tests in 29 files (310 unit and 30 integration tests; Table 4.7). The inventory below was not regenerated.

Table A3.1: Test files of the original suite, number of test functions and the component each exercises (86 tests; the current suite has 340, see Table 4.7).

File (under `tests/`)	Tests	Component or behaviour tested
`unit/test_signal_cleaner.py`	18	VEP conditioning functions (`signal_cleaner.py`)
`unit/test_mri_transforms.py`	12	MRI preprocessing functions (`mri_transforms.py`)
`unit/test_cross_attention.py`	8	Cross-attention module (`cross_attention.py`)
`unit/test_grad_cam_3d.py`	8	3D Grad-CAM (`grad_cam_3d.py`)
`unit/test_cafn_model.py`	7	Complete model and multi-task loss (`cafn.py`)
`unit/test_dataset.py`	7	Dataset classes and synthetic generators (`dataset.py`, `synthetic.py`)
`unit/test_plot_attention.py`	5	Attention heatmap (`plot_attention.py`)
`unit/test_spatial_encoder.py`	5	Spatial encoder (`spatial_encoder.py`)
`unit/test_temporal_encoder.py`	4	Temporal encoder (`temporal_encoder.py`)
`unit/test_train_utils.py`	4	Training utilities (`train.py`)
`integration/test_pipeline_end_to_end.py`	4	Raw files through preprocessing, model, loss, optimiser step and explainability
`integration/test_training_loop.py`	4	Multi-epoch training, loss reduction, accuracy and checkpoint round-trip
Total	**86**	**78 unit tests and 8 integration tests**

A3.2 TEST FUNCTIONS

Listing A3.1 lists every collected test function by file, test class and name, as recorded in the repository's pytest node-identifier file.

Listing A3.1: Collected test functions.

```
unit/test_signal_cleaner.py
    TestButterworthBandpass::test_output_shape_preserved
    TestButterworthBandpass::test_attenuates_low_frequency_drift
    TestButterworthBandpass::test_preserves_inband_content
```

```
    TestButterworthBandpass::test_rejects_highcut_above_nyquist
    TestButterworthBandpass::test_rejects_lowcut_above_highcut
    TestButterworthBandpass::test_rejects_too_short_signal
    TestButterworthBandpass::test_rejects_non_1d_input
    TestButterworthBandpass::test_rejects_nan_input
    TestNotchFilter::test_attenuates_mains_hum
    TestNotchFilter::test_rejects_freq_above_nyquist
    TestZscoreNormalize::test_mean_zero_std_one
    TestZscoreNormalize::test_constant_signal_does_not_explode
    TestResampleToLength::test_upsample_length
    TestResampleToLength::test_downsample_length
    TestResampleToLength::test_identity_when_same_length
    TestCleanVepSignal::test_output_shape_dtype
    TestCleanVepSignal::test_output_is_finite_and_normalized
    TestCleanVepSignal::test_does_not_shift_p100_like_peak_latency
unit/test_mri_transforms.py
    TestLoadNifti::test_returns_ras_oriented_data
    TestLoadNifti::test_returns_voxel_zooms
    TestLoadNifti::test_rejects_non_3d_volume
    TestForegroundMaskThreshold::test_zeroes_low_intensity_voxels
    TestForegroundMaskThreshold::test_all_zero_volume_is_noop
    TestResampleIsotropic::test_anisotropic_spacing_upsamples_correct_axis
    TestResampleIsotropic::test_isotropic_input_shape_unchanged
    TestPreprocessMriVolume::test_output_shape_and_range
    TestPreprocessMriVolume::test_output_shape_default_config
    TestPreprocessMriVolume::test_constant_volume_does_not_produce_nans
    TestPreprocessMriVolume::test_rejects_non_3d_volume
    TestPreprocessMriFile::test_full_file_pipeline
unit/test_cross_attention.py
    TestMultiHeadCrossAttention::test_output_and_attention_shapes
    TestMultiHeadCrossAttention::test_attention_weights_sum_to_one
    TestMultiHeadCrossAttention::test_handles_mismatched_query_key_lengths
    TestMultiHeadCrossAttention::test_rejects_non_divisible_heads
    TestCrossAttentionFusion::test_output_shape_matches_query_tokens
    TestCrossAttentionFusion::test_output_is_finite
    TestCrossAttentionFusion::test_gradients_flow_to_all_parameters
    TestCrossAttentionFusion::test_temporal_signal_actually_influences_output
unit/test_grad_cam_3d.py
    TestGradCAM3D::test_cam_shape_matches_input_volume
    TestGradCAM3D::test_cam_values_in_unit_range
    TestGradCAM3D::test_explicit_target_class_is_respected
    TestGradCAM3D::test_hooks_are_removed_after_generate
    TestGradCAM3D::test_model_left_in_original_training_mode
    TestGradCAM3D::test_manual_engine_generate_and_remove
    TestPlotGradcamSlices::test_saves_png_file
    TestPlotGradcamSlices::test_rejects_mismatched_shapes
unit/test_cafn_model.py
    TestCAFNForward::test_output_keys_and_shapes
    TestCAFNForward::test_batch_size_one
    TestCAFNForward::test_logits_are_finite
    TestCAFNMultiTaskLoss::test_loss_is_finite_scalar
    TestCAFNMultiTaskLoss::test_loss_equals_weighted_sum_of_parts
    TestCAFNMultiTaskLoss::test_backward_reaches_every_parameter
    TestCAFNMultiTaskLoss::test_optimizer_step_changes_weights
unit/test_dataset.py
    TestSyntheticNeuroDataset::test_item_shapes_and_types
    TestSyntheticNeuroDataset::test_length
    TestSyntheticNeuroDataset::test_labels_are_balanced_across_classes
    TestSyntheticNeuroDataset::test_dataloader_batches_correctly
    TestSyntheticNeuroDataset::test_different_seeds_change_labels
    TestPairedNeuroDataset::test_reads_manifest_and_preprocesses
    TestPairedNeuroDataset::test_raises_on_empty_manifest
unit/test_plot_attention.py
    TestCrossAttentionHeatmap::test_output_shape_averages_batch_and_heads
    TestCrossAttentionHeatmap::test_values_are_nonnegative
    TestCrossAttentionHeatmap::test_rejects_wrong_dimensionality
    TestPlotCrossAttention::test_saves_png_and_returns_heatmap
    TestPlotCrossAttention::test_heatmap_matches_manual_reduction
unit/test_spatial_encoder.py
    TestSpatialEncoder3D::test_output_shape
    TestSpatialEncoder3D::test_batch_size_one
    TestSpatialEncoder3D::test_gradients_flow_to_all_parameters
    TestSpatialEncoder3D::test_last_conv_hook_target_participates_in_forward
    TestSpatialEncoder3D::test_deterministic_in_eval_mode
unit/test_temporal_encoder.py
    TestTemporalEncoder1D::test_output_shape
    TestTemporalEncoder1D::test_batch_size_one
```

```
    TestTemporalEncoder1D::test_gradients_flow_to_all_parameters
    TestTemporalEncoder1D::test_robust_to_varying_input_length
unit/test_train_utils.py
    TestTrainValSplit::test_normal_split_partitions_dataset
    TestTrainValSplit::test_single_sample_dataset_falls_back_to_all_train
    TestTrainValSplit::test_split_is_deterministic_for_fixed_seed
    TestEvaluate::test_empty_loader_returns_nan
integration/test_pipeline_end_to_end.py
    TestFullPipelineFromRawFiles::test_raw_files_to_trained_step

   TestFullPipelineFromRawFiles::test_preprocessing_output_feeds_model_without_shape_e
   rrors

   TestExplainabilityOnRealPipelineOutput::test_gradcam_and_attention_plot_from_datase
   t_sample

   TestClassConditionedSignalIsLearnable::test_glaucoma_vs_neuritis_vep_signals_are_di
   stinguishable
integration/test_training_loop.py
    TestTrainModel::test_training_loss_decreases_substantially
    TestTrainModel::test_model_learns_to_separate_classes
    TestTrainModel::test_runs_without_validation_set
    TestCheckpointRoundTrip::test_save_and_load_preserves_behavior
```

APPENDIX 4
REPRODUCTION NOTES

A4.1 SOFTWARE ENVIRONMENT

The repository lists its dependencies in `requirements.txt`: torch, monai, nibabel, scipy, numpy, matplotlib, scikit-learn, mne, nilearn, pytest and pytest-cov, with version ranges; the versions installed at the time of a run are recorded in the environment snapshot that `run_all.sh` writes. The results were produced on an NVIDIA RTX 3050 Laptop GPU (4 GB, thermally throttled during long runs), with some synthetic-benchmark runs on an Apple M3 Max, using Python 3.14 on macOS and Python 3.12 on Ubuntu 22.04 under WSL2 with CUDA. The model needs less than 1 GB of memory at batch size 8.

A4.2 COMMANDS

Listing A4.1: Commands used with the repository.

```
# install the dependencies
pip install -r requirements.txt

# run the test suite (340 tests), optionally with coverage
pytest
pytest --cov=src

# one-command real-cohort pipeline: download -> preprocess -> train -> validate ->
   test
./run_all.sh              # full run (downloads about 68 GB, trains overnight;
   resumable)
./run_all.sh --smoke      # 1 subject per group, 2 epochs: plumbing check

# the stages individually
python scripts/download_datasets.py          # dry run: lists files and exact sizes
python scripts/download_datasets.py --yes    # download the two core cohorts
python -m src.data.cohorts --raw data/raw --out data/processed
python -m src.cohort_experiments --manifest data/processed/manifest.csv --out
   results/cohorts

# Study B: repeated cross-validation, baselines, bootstrap, permutation tests
python -m src.cv_real --manifest data/processed/manifest.csv --out results/real_cv

# Study A: controlled synthetic benchmark (pilot, then the main setting on five seeds)
python -m src.ablation_synthetic --pilot --out results/synth_pilot
python -m src.ablation_synthetic --out results/synth --settings 1.0:0.3 --seeds 0 1 2
   3 4

# general training command line: synthetic data, or a manifest
python -m src.train --epochs 20 --batch-size 8 --lr 1e-3 \
    --synthetic-samples 64 --seed 42 --run-name demo
python -m src.train --manifest data/processed/manifest.csv --amp --device auto --test-
   split 0.15
```

A4.3 MEASUREMENTS REPORTED IN CHAPTER 5

The signal-conditioning and synthetic-data measurements of Sections 5.2 and 5.3 were produced by a stand-alone script, `report_experiments.py`, which imports the repository's own `clean_vep_signal`, `synthetic_vep_signal`,

`synthetic_mri_volume` and `foreground_mask_threshold` functions (in their original versions), generates 50 seeded records per class and writes figures and a JSON file of results. A second script, `model_complexity.py`, counts parameters and multiply-accumulate operations analytically from the layer definitions; its result for the final model is stored in `docs/results_raw/model_complexity_v2.json`. These scripts and the resulting data files are supplied with this report in the folder `report_assets/`. The scripts can be re-run with `NEURO_REPO=<path to repository> python report_experiments.py <output directory>`; they were written for the original code base, and because the synthetic generator has since changed (Section 3.5.4) they will not reproduce Tables 5.2 and 5.4 exactly with the current generator. The results of Sections 5.9 to 5.12 were produced by `src/ablation_synthetic.py`, `src/cv_real.py` and `src/cohort_experiments.py` (Listing A4.1); their raw outputs (JSON, CSV and summaries) are in `docs/results_raw/`.

A4.4 ITEMS NOT REPRODUCED HERE

- The full real-cohort run downloads about 68 GB and trains overnight on a laptop GPU, so it is not reproduced inside the report; `./run_all.sh --smoke` gives a quick plumbing check, and the raw outputs of the runs summarised in Chapter 5 are supplied in `docs/results_raw/`.
- Statement coverage was not re-measured for the current code; the 95.1% figure applies to the original twelve modules (Section 5.6), and Appendices 2 and 3 reproduce the original source listings and the original 86-test inventory. The 340-test suite and the modules added since are available in the repository.